EVPIX-RV32: 5-Stage Custom RISC-V SoC with Integrated IPU and TinyML Support for Real-Time Edge-Vision AI Acceleration

by

Ahasan Ullah Khalid

2008051

This thesis is submitted in partial fulfillment of the requirement for the degree of BACHELOR OF SCIENCE IN ELECTRONICS AND TELECOMMUNICATION ENGINEERING

Department of Electronics and Telecommunication Engineering

Chittagong University of Engineering and Technology

Chattogram-4349, Bangladesh.

June, 2026

Declaration

I hereby declare that the work contained in this Thesis has not been previously submitted to meet requirements for an award at this or any other higher education institution. To the best of my knowledge and belief, the Thesis contains no material previously published or written by another person except where due reference is cited. Furthermore, the Thesis complies with PLAGIARISM and ACADEMIC INTEGRITY regulation of CUET.

Ahasan Ullah Khalid
2008051

Department of Electronics and Telecommunicaiton Engineering

Chittagong University of Engineering & Technology (CUET)

Dedication

Dedicated to,

my parents, for their love, endless support, encouragement

for their love, endless support, and sacrifices.

Approval/Declaration by the Supervisor

This is to certify that ……Ahasan Ullah Khalid…… has carried out this research work under my/our supervision, and that he has fulfilled the relevant Academic Ordinance of the Chittagong University of Engineering and Technology, so that he is qualified to submit the following Thesis in the application for the degree of BACHELOR of SCIENCE in ELECTRONICS AND TELECOMMUNICATION ENGINEERING. Furthermore, the Thesis complies with the PLAGIARISM and ACADEMIC INTEGRITY regulation of CUET.

Md. Farhad Hossain
Assistant Professor
Department of Electronics and Telecommunication Engineering
Chittagong University of Engineering & Technology

Acknowledgment

I want to thank my supervisor for the patience and guidance through this project, especially during the weeks when the timing constraints refused to close. My lab mates kept me sane during the long synthesis runs, and I owe them for the coffee and the debugging help. Finally, my family put up with me talking about pipelines and pixel buffers at dinner, which is more than anyone should have to endure.

Abstract

Edge devices need to process visual data locally, without sending anything to the cloud, yet general-purpose processors cannot keep up with real-time pixel workloads, and fixed-function accelerators are too rigid to adapt as edge-AI tasks evolve. This work presents EVPIX-RV32, a custom 5-stage pipelined RISC-V RV32I processor with hardware image-processing extensions, a streaming Image Processing Unit (IPU), and TinyML support built in. The architecture extends the RISC-V ISA with custom instructions for grayscale conversion, thresholding, Sobel edge detection, and 2D convolution in the custom-0 opcode space, connecting to an autonomous IPU that processes 128×128 frames at pixel-level parallelism. Prototyped on the Digilent Basys-3 Artix-7 FPGA with an OV7670 camera and dual-region VGA display, the system sustains 60 FPS with zero frame drops. A hardware Built-In Self-Test mode runs 61 instructions and shows pass/fail results on the VGA monitor, while a TinyML finger-counting demo validates lightweight neural inference on the platform. The design was synthesized through the open-source OpenROAD flow targeting SkyWater 130-nm CMOS, producing a DRC-clean, LVS-equivalent GDSII layout across 30.25 mm^2, operating at 100 MHz with 3.24 mW total power. Functional verification achieved 100% pass rates across all instructions and IPU kernels, with the IPU running Sobel edge detection at 1,525 FPS. EVPIX-RV32 shows that a compact open RISC-V core with domain-specific extensions can handle real-time embedded vision and lightweight AI inference while remaining physically realizable in accessible 130-nm technology.

Keywords: RISC-V, Edge Vision, Image Processing Unit, IPU, TinyML, Open-Source ASIC, FPGA Prototyping, 130-nm, RTL, RTL to GDSII, GDSII, Custom Soc, AI Acceleration

বিমূর্ত

প্ল্যান্তিক যন্ত্রসমূহের দূরবর্তী তথ্যভাণ্ডারে কোনো তথ্য প্রেরণ না করেই নিজস্বভাবে দৃশ্যমান তথ্য প্রক্রিয়া করণ করা প্রয়োজন। তবে সাধারণ প্রক্রিয়াকারকগুলো তাৎক্ষণিক পিক্সেলের চাপ সামলাতে অক্ষম এবং নির্দিষ্ট কাজের গতিবর্ধকগুলো কৃত্রিম বুদ্ধিমত্তার নিত্যনতুন কাজের সাথে মানিয়ে নেওয়ার ক্ষেত্রে অনেকটাই অনমনীয়। এই গবেষণায় 'ইভিপিক্স-আরভি৩২' নামক একটি নিজস্ব পাঁচ-স্তরের ধারাবাহিক 'রিস্ক-ভি' প্রক্রিয়াকারক উপস্থাপন করা হয়েছে। এতে যন্ত্রাংশভিত্তিক চিত্র প্রক্রিয়াকরণ সম্প্রসারণ, একটি চিত্র প্রক্রিয়াকরণ একক (আইপিইউ) এবং ক্ষুদ্রাকৃতির যন্ত্র-শিখন বা 'টাইনিএমএল' সুবিধা যুক্ত রয়েছে। এই আর্কিটেকচারটি ধূসর রঙের রূপান্তর, প্রান্তিকীকরণ, 'সোবেল' সীমানা শনাক্তকরণ এবং দ্বিমাত্রিক আবর্তনের মতো কাজের জন্য নিজস্ব নির্দেশিকার মাধ্যমে মূল নির্দেশিকা সংকলনকে প্রসারিত করে। এটি একটি স্বয়ংক্রিয় এককের সাথে যুক্ত হয়ে সমান্তরালভাবে ১২৮ আকারের চিত্র প্রক্রিয়াকরণ করতে পারে। একটি ক্যামেরা এবং 'ভিজিএ' পর্দার মাধ্যমে 'এফপিজিএ'-তে প্রোটোটাইপ করা এই কাঠামোটি কোনো ফ্রেম না হারিয়েই সেকেন্ডে ৬০টি ফ্রেম নিরবচ্ছিন্নভাবে চালাতে পারে। এর হার্ডওয়্যার ভিত্তিক স্বয়ংক্রিয় পরীক্ষা ব্যবস্থা ৬১টি নির্দেশিকা চালিয়ে পর্দায় ফলাফল দেখায়। পাশাপাশি, আঙুল গণনার একটি ক্ষুদ্র যন্ত্র-শিখন পরীক্ষার মাধ্যমে এই প্ল্যাটফর্মে কৃত্রিম বুদ্ধিমত্তার কার্যকারিতা যাচাই করা হয়েছে। উন্মুক্ত পদ্ধতির মাধ্যমে 'স্কাইওয়াটার ১৩০-ন্যানোমিটার সিমস' প্রযুক্তির জন্য এই নকশাটি সিন্থেসাইজ করা হয়েছে। এটি ৩০.২৫ বর্গ মিলিমিটার স্থান জুড়ে ত্রুটিমুক্ত ও সমতুল্য বিন্যাস তৈরি করেছে, যা মাত্র ৩.২৪ মিলিওয়াট বিদ্যুৎ খরচে ১০০ মেগাহার্টজ গতিতে কাজ করে। এর কার্যকারিতা যাচাইয়ে শতভাগ সাফল্য অর্জিত হয়েছে, যেখানে চিত্র প্রক্রিয়াকরণ এককটি সেকেন্ডে ১৫২৫ ফ্রেমে সীমানা শনাক্ত করতে সক্ষম। সংক্ষেপে, এই নকশাটি প্রমাণ করে যে, নির্দিষ্ট কাজের সম্প্রসারণযুক্ত একটি ছোট ও উন্মুক্ত 'রিস্ক-ভি' কোর তাৎক্ষণিক চিত্র প্রক্রিয়াকরণ এবং ক্ষুদ্র কৃত্রিম বুদ্ধিমত্তার কাজ সফলভাবে পরিচালনা করতে পারে এবং একইসাথে এটি ১৩০-ন্যানোমিটার প্রযুক্তিতে বাস্তবায়ন যোগ্য।

Contents

List of Figures

List of Tables

Chapter 1

INTRODUCTION

1.1 Background

Artificial intelligence, embedded systems, and real-time sensor processing have converged at the edge, changing how small devices handle computation. Modern edge devices, from autonomous drones and industrial inspection cameras to wearable health monitors and smart home appliances, now process visual information locally rather than relying on cloud infrastructure. Researchers call this *edge vision* or *edge AI*. The approach matters when network latency, bandwidth limits, privacy, or disconnected operation rule out cloud dependence [1, 2].

Edge vision mixes two types of work. One is low-level pixel processing: grayscale conversion, spatial filtering, edge detection, and morphological operations. The other is higher-level pattern recognition such as object detection, gesture classification, and neural network inference. Classical image processing kernels run in parallel and access memory predictably, so hardware accelerates them well. But modern AI pipelines need control flow, decision logic, and model configuration that only programmable processors can handle [3, 4].

The instruction set architecture (ISA) is the interface between hardware and software in any processor system. Embedded and edge applications need this interface to stay compact, energy-efficient, and extensible. RISC-V, the open-standard ISA developed at UC Berkeley [5], offers an alternative to proprietary architectures. Its modular design separates the base integer ISA from optional standard extensions. It also reserves opcode space for custom instructions. Designers can therefore add domain-specific operations without breaking compatibility, and they can implement and share complete processor designs without licensing fees [6].

Field-Programmable Gate Arrays (FPGAs) allow rapid prototyping and deployment of custom digital circuits, including soft processor cores and their accel-

erators. The Digilent Basys-3 board uses the Xilinx Artix-7 FPGA. It includes VGA output, slide switches, LEDs, and Pmod headers for camera modules [7, 8]. FPGA prototyping allows validation of architectural ideas against real-world timing, I/O, and memory constraints before committing to silicon.

FPGA validation proves functional correctness, but a processor architecture is only truly proven when it can be manufactured. Application-Specific Integrated Circuits (ASICs) outperform programmable logic in speed, energy use, and area. Historically, though, they required expensive proprietary toolchains and confidential process design kits (PDKs). The SkyWater 130-nm open-source PDK and the OpenROAD EDA flow changed that. Students and researchers can now synthesize, place, route, and verify manufacturable layouts with entirely open tools [9, 10].

1.2 Problem statement

RISC-V processor design, image-processing accelerators, and TinyML deployment have all advanced. Yet the literature remains fragmented. Most existing work does one thing well: ISA specification, CPU microarchitecture, fixed-function filtering, neural acceleration, or open-source silicon. Few combine these into a single, resource-constrained edge-vision platform. Educational RISC-V cores run instructions but never touch a real-time sensor pipeline. FPGA vision systems run fixed filters with no programmable control. TinyML platforms hide their accelerators behind closed blocks. ASIC tutorials report synthetic benchmarks instead of running real applications.

That leaves a gap. The field needs an open processor platform that is fully inspectable and that handles all of the following:

1. A standards-compatible RISC-V CPU capable of executing general-purpose code, managing peripherals, and configuring accelerators.

2. Hardware-accelerated image processing through dedicated datapaths for classical vision kernels (grayscale, threshold, Sobel, convolution), reducing instruction traffic and energy relative to software execution.

3. Direct interfaces to a camera (OV7670) and display (VGA), with frame buffering and performance instrumentation visible to the user.

4. Support for TinyML inference, showing the platform can move beyond deterministic filters to data-driven perception.

5. ASIC physical realizability through synthesis and layout in an open CMOS process, proving the architecture is manufacturable rather than merely a simulation artifact.

This thesis asks: *Can a compact, open RISC-V processor with custom image-processing extensions and an integrated IPU handle real-time edge-vision tasks and lightweight TinyML inference, while remaining physically realizable in an open 130-nm ASIC flow?*

More specifically:

- What is the cleanest way to add custom image-processing instructions to a five-stage RV32I pipeline without breaking baseline ISA compatibility?
- What balance between scalar CPU control and dedicated IPU acceleration works best for 128×128 real-time video processing on a resource-constrained FPGA?
- How does the FPGA prototype translate to an ASIC implementation in terms of area, timing, power, and physical design feasibility?
- Can this custom hardware platform run and validate a TinyML workload for finger-counting gesture recognition?

1.3 Research Objective

The primary objective of this research is to design, verify, prototype on FPGA, and physically implement EVPIX-RV32, a custom edge-vision processor that combines RISC-V programmability with hardware-accelerated image processing and TinyML support. The specific sub-objectives are:

1. Develop a five-stage pipelined RV32I processor core with full hazard detection, forwarding, and control logic in SystemVerilog, and integrate custom instruction decode and execution paths for image-processing operations within the EX stage.

2. Design a streaming Image Processing Unit controlled by custom RISC-V instructions, capable of autonomous frame-level grayscale, thresholding, Sobel edge detection, 2D convolution, max-pooling, and average-pooling operations.

3. Implement the complete processor on the Basys-3 Artix-7 FPGA with OV7670 camera input, dual-region VGA output, switch-controlled operating modes, and real-time performance overlays.

4. Develop a hardware BIST mode that exercises all RV32I instructions and IPU kernels, displaying pass/fail status on the VGA monitor without external debugging equipment.

5. Integrate and validate a finger-counting neural network inference pipeline, showing the platform can handle lightweight edge-AI tasks.

6. Synthesize the processor through the open-source OpenROAD/Yosys flow targeting the SkyWater 130-nm PDK, to produce a DRC-clean, LVS-equivalent layout with positive timing slack.

1.4 Scope and Delimitations

This thesis focuses on the architecture, RTL design, FPGA prototyping, and ASIC physical implementation of a single-core edge-vision processor. The scope covers the complete digital design flow from instruction set extension through GDSII generation. It explicitly excludes the following:

- The processor targets bare-metal embedded execution without virtual memory, caches, or Linux-capable privileged architecture support.

- All storage resides in on-chip BRAM (FPGA) or SRAM macros (ASIC). DDR or Flash controllers are outside the scope.
- The target resolution is 128×128 pixels, chosen to fit within Basys-3 BRAM constraints and to emphasize architectural clarity over raw throughput.
- The TinyML demonstration uses a compact fully-connected or simple convolutional model for finger counting, not a state-of-the-art object detector such as YOLO or SSD.
- The ASIC flow produces a verified GDSII layout suitable for tapeout. Actual wafer fabrication, packaging, and silicon testing are reserved for future work.

These limits are intentional. Architectural transparency, educational accessibility, and complete system integration matter more here than beating closed commercial platforms on raw performance.

1.5 Contributions

This thesis offers five contributions to computer architecture, embedded vision, and open-source silicon:

1. The thesis specifies and implements eight custom image-processing instructions (GRAYSCALE, THRESH, SOBEL, CONV, VDOT, RELU, HACC, OTSU) encoded within the RISC-V custom-0 opcode space, integrating them into a standard five-stage RV32I pipeline with full forwarding and hazard support.
2. It introduces a heterogeneous design methodology that couples a programmable RISC-V scalar core with a streaming IPU via memory-mapped control registers and custom instructions, so that frame processing is deterministic, real-time, and visible to software.
3. It delivers a complete Basys-3 prototype with OV7670 camera input, dual-region VGA display, and on-screen performance instrumentation, giving immediate visual validation of architectural behavior across BIST, IPU modes, and TinyML inference.
4. It deploys and validates a quantized neural network for finger-counting gesture recognition, proving that lightweight edge AI can run on a student-designed processor with custom arithmetic datapaths.

5. It produces the first known open-source ASIC implementation of a RISC-V edge-vision processor with integrated IPU in the SkyWater 130-nm node, including complete synthesis, floorplan, CTS, routing, and signoff reports, which shows it is physically manufacturable.

1.6 Thesis organization

The rest of this thesis is structured as follows:

- Chapter 2 surveys the theoretical and technical foundations of computer architecture, RISC-V ISA design, five-stage pipeline microarchitecture, custom instruction extensions, digital image processing, FPGA technology, camera systems, TinyML, and open-source ASIC flows. It critically reviews prior works across ten categories and identifies the research gap that EVPIX-RV32 addresses.

- Chapter 3 presents the complete design methodology, including system architecture specification, RTL design in SystemVerilog, functional verification using self-checking testbenches, FPGA prototyping on the Basys-3 board, camera interface evaluation, and ASIC implementation through the OpenROAD flow with the SkyWater 130-nm PDK.

- Chapter 4 reports functional verification outcomes (100% BIST pass rates), FPGA resource utilization and timing analysis, real-time IPU performance metrics, TinyML accuracy results, and complete ASIC implementation results including area, power, timing, and physical verification signoff.

- Chapter 5 interprets the results in the context of related work, discusses trade-offs between FPGA and ASIC implementations, analyzes the efficiency of the custom instruction set, evaluates system limitations, and positions the findings within the broader edge-AI and open-silicon research landscape.

- Chapter 6 summarizes the achievements relative to the stated objectives, acknowledges limitations, and proposes directions for future research including compiler intrinsic support, DMA integration, higher-resolution pipelines, formal verification, and advanced-node migration.

Table 1.1: Thesis chapter overview and contribution mapping

Chapter	**Focus**	**Key contribution**
1	Introduction	Problem definition, objectives, scope
2	Literature review	Critical gap analysis across 10 research groups
3	Methodology	RTL architecture, verification, FPGA, ASIC flows
4	Results	Functional, performance, and physical metrics
5	Analysis and discussion	Interpretation, trade-offs, positioning
6	Conclusions	Summary, limitations, future directions

Chapter 2

LITERATURE REVIEW

Computing has advanced remarkably, from mechanical calculators to billion-transistor chips in just a few decades. Pioneers such as Alan Turing [11] and John von Neumann [12] laid the conceptual foundations. They established that a computer, given proper instructions, can perform any computable function. That idea has shaped computing ever since.

2.1 Computer architecture fundamentals

2.1.1 Von Neumann architecture

John von Neumann first described this architecture in his 1945 *First Draft of a Report on the EDVAC* [12], which became the basis for modern computers. It has four main components: (1) a CPU with an ALU and control unit; (2) a memory unit that stores both instructions and data; (3) input/output for external communication; and (4) a system bus that connects these components [13]. Instructions and data share the same memory. This stored-program design gives flexibility that distinguishes general-purpose computers from fixed-task machines.

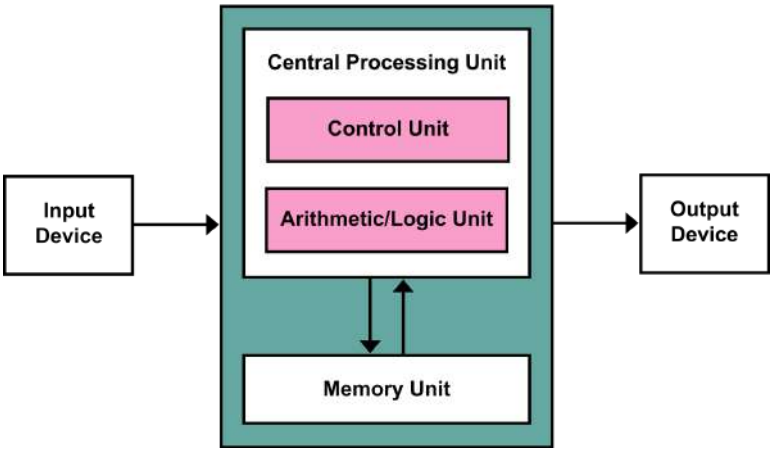


Figure 2.1: Von Neumann Architecture.

2.1.2 Harvard architecture

The Harvard architecture first appeared in the relay-based Mark I computer at Harvard University in the 1940s [14]. It solves the Von Neumann bottleneck with separate memory and buses for instructions and data. This lets the processor fetch instructions and access data at the same time, roughly doubling memory bandwidth compared with the Von Neumann approach [13].

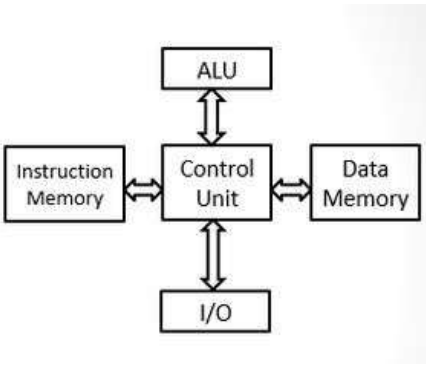


Figure 2.2: Harvard Architecture.

2.1.3 Modified Harvard architecture

The modified Harvard architecture blends both approaches. It keeps separate instruction and data memory but allows each to be accessed as the other when needed. Most modern processors, including ARM9 and newer ARM cores and DSPs [15], use this approach. It offers the speed of separate memory for normal operation while keeping the flexibility of unified access when software needs it [13].

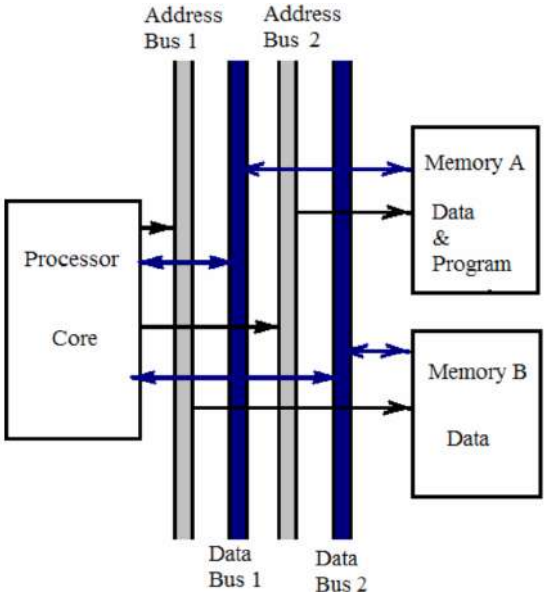


Figure 2.3: Modified Harvard Architecture.

Table 2.1: Comparison of different computer architectures

Architecture	**Main feature**	**Advantage**	**Limitation**	**Relevance to EVPIX-RV32**
Von Neumann [12]	Unified code/data memory	Simple address model	Fetch/data memory contention	Useful as conceptual baseline
Harvard [14]	Separate instruction and data memories	Parallel instruction fetch and data access	Less flexible memory model	Helpful for FPGA BRAM-based CPU design
Modified Harvard [13]	Separate low-level paths with unified programming view	Balances bandwidth and programmability	More control complexity	Practical for embedded RISC cores

2.2 Reduced instruction set computing (RISC)

2.2.1 CISC architecture

Complex Instruction Set Computer (CISC) architectures have large, complex instruction sets that include high-level operations [13]. The x86 architecture follows this philosophy: hardware should directly support high-level language constructs, with single instructions that perform multiple low-level operations [16]. CISC features include variable-length instructions, many addressing modes, memory-to-memory operations, and specialized instructions for string manipulation and decimal arithmetic. The x86 architecture debuted with the Intel 8086 in 1978 [16] and its descendants remain in widespread use today.

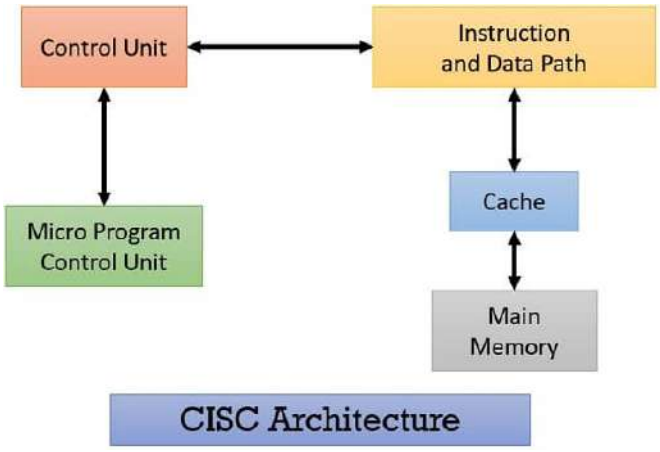


Figure 2.4: CISC Architecture.

2.2.2 RISC architecture

Reduced Instruction Set Computer (RISC) emerged in the early 1980s as a challenge to CISC [17]. RISC rests on the observation that most programs spend most of their time executing a small subset of instructions [17]. Designers therefore optimize hardware for simple, common instructions rather than rarely-used complex ones. This saves power, which is why RISC dominates mobile phones, laptops, and power-constrained IoT devices [18]. The canonical examples are the ARM and RISC-V architectures.

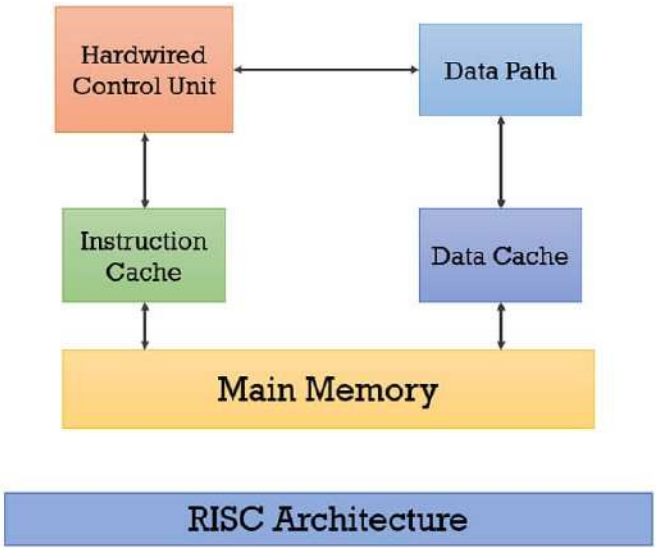


Figure 2.5: RISC Architecture.

2.2.3 RISC vs CISC

Table 2.3: RISC and CISC design tradeoffs

Dimension	**CISC tendency**	**RISC tendency**	**Implication for EVPIX-RV32**
Instruction format [13]	Often variable length	Usually regular and fixed length	Simplifies RV32I fetch/decode
Memory operations [17]	Memory-to-memory possible in some ISAs	Load-store model	Clean pipeline and memory stage
Control [13]	Often complex or microcoded	Hardwired decode practical	Lower FPGA/ASIC control overhead
Pipelining [19]	Complicated by irregular instructions	Natural fit	Supports five-stage design
Customization [6]	Proprietary ISA constraints	RISC-V custom opcode spaces	Enables image instructions

Figure 2.6: RISC-V.

2.3 RISC-V architecture

RISC-V is an open-standard ISA designed to be modular and extensible [6]. Unlike proprietary ISAs, no single vendor owns RISC-V. RISC-V is not one core but a family of profiles and extensions for microcontrollers, application processors, vector units, accelerators, or research prototypes [5]. RISC-V International defines the unprivileged ISA, privileged architecture, standard extensions, and compatibility rules [20].

The open ISA philosophy matters both technically and educationally [5]. Technically, designers can build a compatible CPU without licensing fees. Academically, students and researchers can inspect the ISA, modify microarchitectures, add custom instructions, and publish complete designs. This openness fits the open-source silicon movement, where open RTL, PDKs, and physical-design tools lower the barrier to ASIC experimentation [9]. EVPIX-RV32 uses this openness in two ways: it implements RV32I as a base ISA, and it extends the instruction path for image-processing operations.

Table 2.5: RISC-V standard extension overview

Extension	**Function**	**Benefit**	**Reason not always required in EVPIX-RV32**
M [6]	Integer multiply/divide	Accelerates arithmetic-heavy code	Image primitives here use simple arithmetic and kernels; optional
A [6]	Atomic operations	Supports synchronization	Not central to single-core pipeline
F/D [6]	Floating point	Scientific and signal workloads	Increases area; fixed-point image processing is sufficient
C [6]	Compressed instructions	Improves code density	Adds decode complexity; useful future extension
V [6]	Vector processing	High data-level parallelism	Powerful but heavier than compact custom IPU

2.3.1 RV32I base integer instruction set architecture

RV32I is the foundation of 32-bit RISC-V [6]. It contains about 40 instructions for integer arithmetic, logic, comparisons, branches, jumps, loads, stores, and system instructions. The instruction set is sufficient for standalone operation, with optional extensions adding functionality for specific application domains [5].

Table 2.7: ISA comparison for suitability

ISA	Strength	Limitation for this thesis	Assessment
ARM [18]	Industrial embedded ecosystem	Licensing/customization barriers	Excellent product ISA, less ideal for open custom ISA thesis
MIPS [13]	Clean RISC history	Reduced modern open ecosystem	Useful reference architecture
OpenRISC [21]	Open-source tradition	Smaller ecosystem than RISC-V	Relevant but less standard today
x86 [16]	Massive software compatibility	Complex variable-length ISA	Not suitable for small educational RTL core
RISC-V RV32I [6]	Open, simple, modular, extensible	Requires designer to build ecosystem around custom core	Best match for EVPIX-RV32

Table 2.9: RV32I instruction format summary

Format	Major fields	Instruction class	EVPIX-RV32 relevance
R-type [6]	funct7, rs2, rs1, funct3, rd, opcode	Register arithmetic/-logical	Template for register-register custom operations
I-type [6]	imm[11:0], rs1, funct3, rd, opcode	Immediate arithmetic, loads, JALR	Useful for threshold constants and address offsets
S-type [6]	imm split, rs2, rs1, funct3, opcode	Stores	Frame buffer and memory output
B-type [6]	branch immediate split, rs2, rs1	Conditional branches	Control loops and BIST routines
U-type [6]	imm[31:12], rd, opcode	Upper immediate	Address construction
J-type [6]	jump immediate split, rd, opcode	JAL	Subroutine and loop control

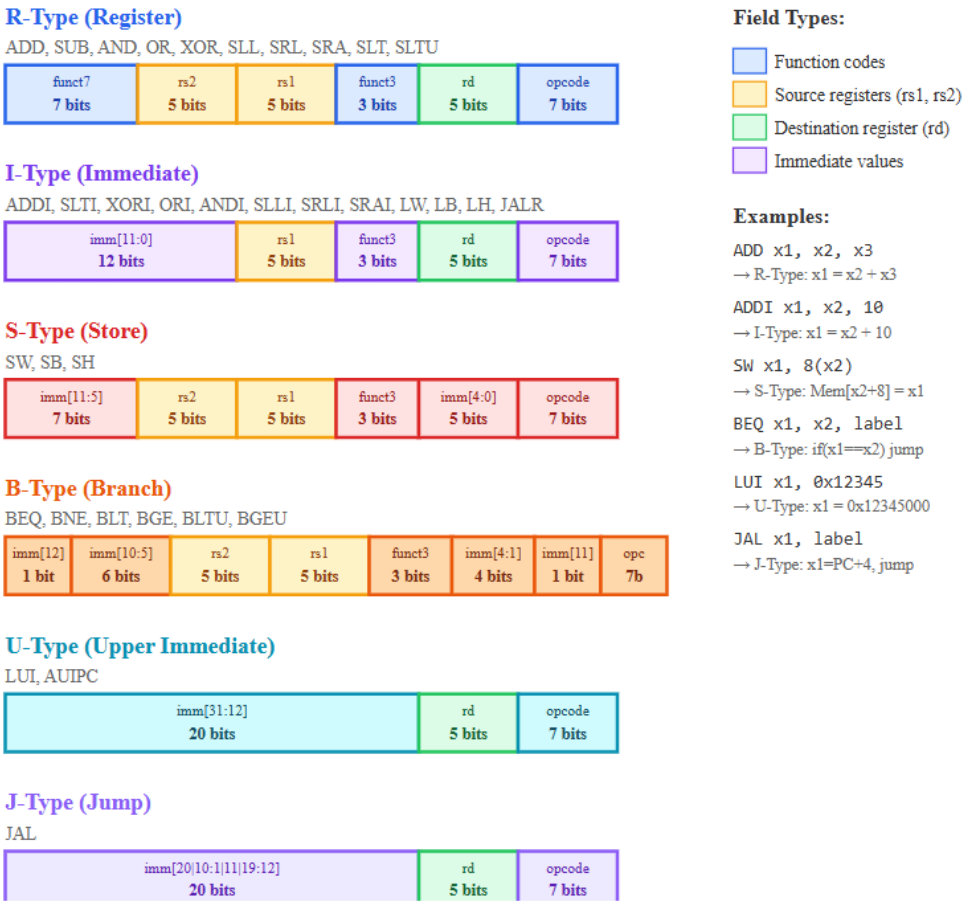


Figure 2.7: RISC-V Instruction Format [6].

2.3.2 Why RV32I was selected for EVPIX-RV32

RV32I was chosen for EVPIX-RV32 for technical, practical, and philosophical reasons [5, 6]. The 32-bit integer base instruction set provides enough capability for control and coordination in embedded vision processing while minimizing implementation complexity and resource use. The simplicity of RV32I enables a clean, efficient pipelined implementation that fits within the constraints of the target FPGA and ASIC technologies.

A 32-bit word fits embedded image processing well [4]. Eight-bit grayscale and 24-bit RGB values fit easily into 32-bit registers. Addressing 128×128 frames needs less than 64 KB, so 32-bit addresses are plenty. Integer instructions handle address calculation, loop control, and coordination without the overhead of floating-point or vector units that would go mostly unused here.

The ability to add custom instructions was decisive [6]. The reserved opcode space for custom instructions makes it easy to integrate the IPU into the processor pipeline. Opcode `0001011` (custom-0) encodes the custom image-processing

instructions, keeping them compatible with standard RV32I. Most other architectures lack standardized custom-extension mechanisms, making this kind of integration difficult or impossible [5]. RISC-V's open nature also helps academic research by removing licensing restrictions that would complicate publication.

2.4 Five-stage pipeline processor architecture

Pipelining splits instruction execution into stages and overlaps multiple instructions to improve throughput [13]. The classic five-stage RISC pipeline has instruction fetch, decode/register read, execute/address calculation, memory access, and writeback [19]. Each stage maps naturally to a part of the instruction cycle, which makes it easy to teach. It also works well for RV32I because the ISA has fixed instruction lengths, simple addressing, and a load-store memory model [6]. Pipeline registers separate the IF/ID, ID/EX, EX/MEM, and MEM/WB stages. Each register holds data values and the control signals needed by later stages [13].

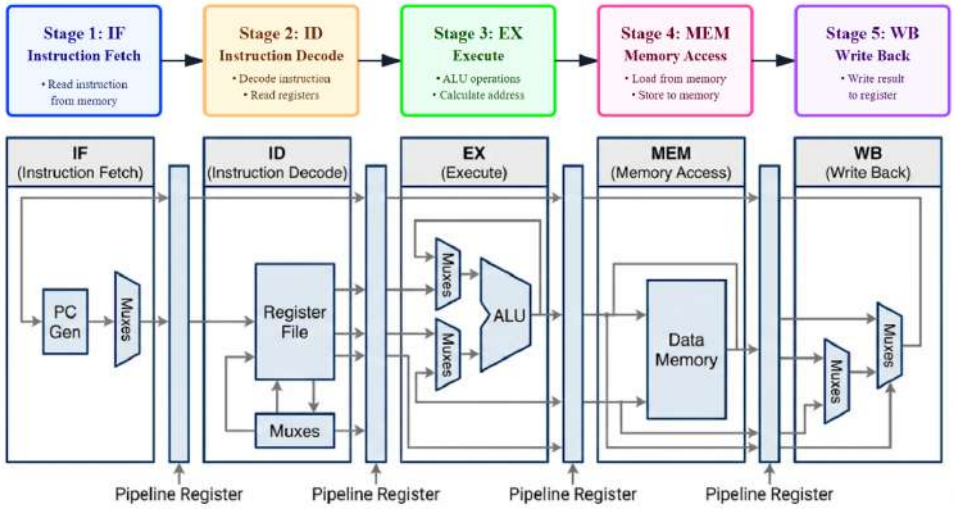


Figure 2.8: RISC-V 5-Stage Pipeline [13].

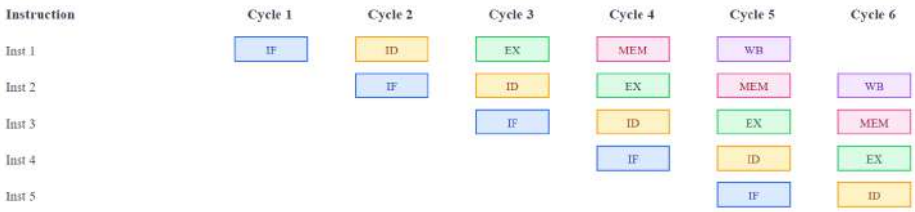


Figure 2.9: RISC-V 5-Stage Pipeline Timing Diagram [13].

2.4.1 Instruction fetch (IF) stage

The IF stage fetches instructions from memory and updates the program counter [13]. In EVPIX-RV32, the IF stage contains the program counter and instruction memory. Every cycle, the instruction memory outputs the instruction at the PC address, and the PC increments by 4. The instruction and PC value move into the IF/ID pipeline register for the decode stage.

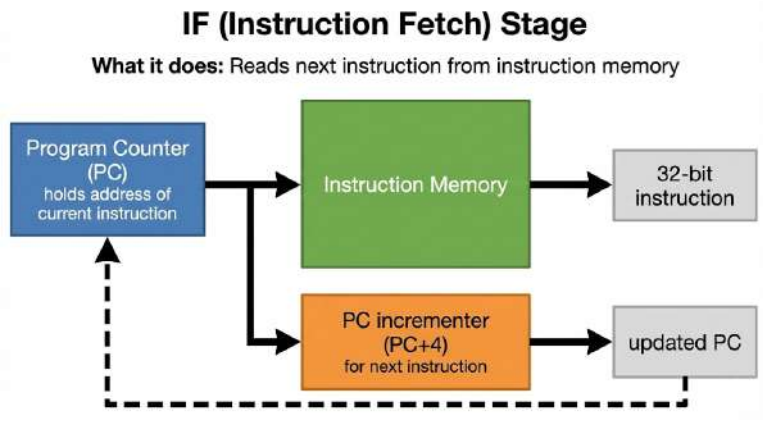


Figure 2.10: Instruction Fetch (IF) Stage.

2.4.2 Instruction decode (ID) stage

The ID stage decodes the instruction, reads source operands from the register file, generates control signals, and prepares immediate values [13]. EVPIX-RV32's ID stage holds the main control unit, ALU control unit, immediate generator, and register file. It extracts the opcode, register specifiers, function codes, and immediate fields from the IF/ID register [6].

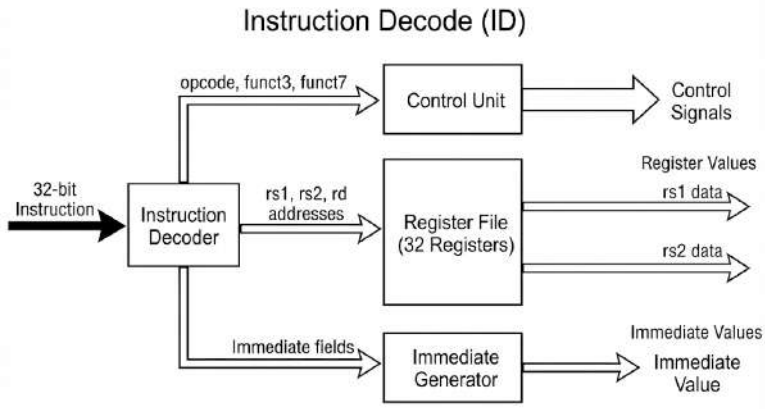


Figure 2.11: Instruction Decode (ID) Stage.

2.4.3 Execute (EX) stage

The EX stage performs the core computation for each instruction [13]. In EVPIX-RV32, the EX stage contains the ALU, branch unit, and forwarding multiplexers. The ALU operates on source operands from the register file (via ID/EX), the writeback stage (via forwarding), or the memory stage (via forwarding). The branch unit checks the branch condition and decides whether to take the branch [19].

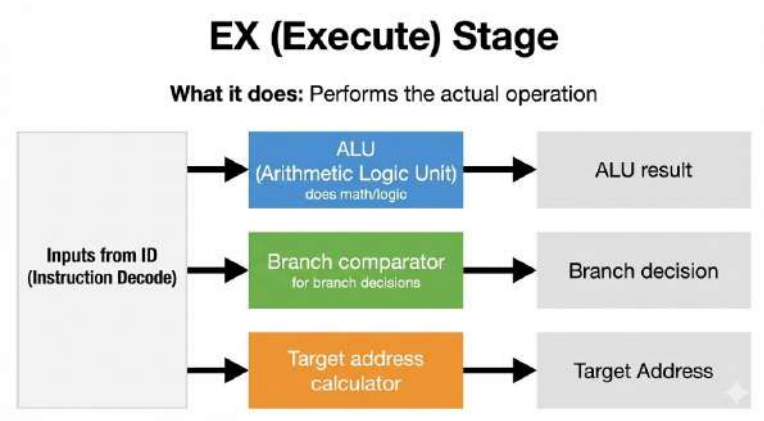


Figure 2.12: Execute (EX) Stage.

2.4.4 Memory access (MEM) stage

The MEM stage handles data memory operations [13]. EVPIX-RV32's MEM stage contains the data memory and byte, halfword, and word access logic. For loads, the ALU-computed address reads data from memory. For stores, data from the second source register writes to memory at that address. For non-memory instructions, the MEM stage just passes the ALU result to writeback.

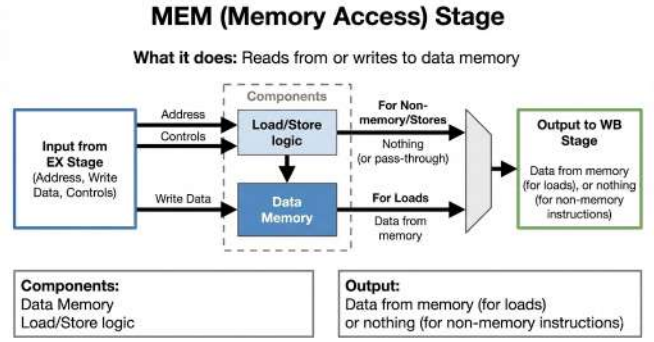


Figure 2.13: Memory Access (MEM) Stage.

2.4.5 Writeback (WB) stage

The WB stage writes results back to the register file [13]. In EVPIX-RV32, a multiplexer in the WB stage selects from the ALU result, memory data, or PC + 4.

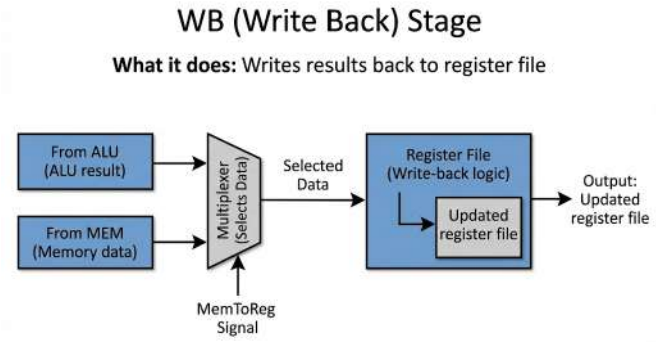


Figure 2.14: Writeback (WB) Stage.

2.4.6 Pipeline registers

Pipeline registers separate stages and hold intermediate state as instructions move through the pipeline [19]. EVPIX-RV32 implements four pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB. Each holds data and control signals from the previous stage for the next one.

2.4.7 Hazard detection unit

The hazard detection unit finds data hazards that forwarding cannot fix and stalls the pipeline [13]. Its main job is detecting load-use hazards, when a load in the EX stage feeds an instruction in the ID stage that needs the loaded value. The loaded data is not ready until the MEM stage finishes, by which time the dependent instruction has already reached EX [19].

2.4.8 Relation to EVPIX-RV32

The five-stage pipeline is central to EVPIX-RV32. It provides the execution framework for RV32I and the platform for IPU integration [6]. It runs control programs that configure and manage the IPU at close to one instruction per cycle for the sequential code that dominates these programs. The pipeline structure also defines the processor-to-IPU interface: custom instructions execute in the EX stage, and IPU status and results travel through the forwarding paths.

2.5 Custom ISA extensions and hardware accelerators

2.5.1 Hardware acceleration

Hardware acceleration uses specialized circuits to perform tasks more efficiently than general-purpose processors [22]. Dedicated circuits avoid the instruction fetch, decode, and scheduling overhead that wastes energy and time in programmable processors. Accelerators exploit data parallelism (same operation on multiple elements), task parallelism (different operations at once), and pipeline parallelism (overlapping dependent operations) [23].

2.5.2 RISC-V custom opcode space

RISC-V explicitly reserves opcode space for custom extensions, so implementers can add domain-specific instructions without breaking base-ISA compatibility [6]. RV32I defines four custom opcodes: custom-0 (0001011), custom-1 (0101011), custom-2 (1011011), and custom-3 (1111011). Standard RISC-V extensions will never use these opcodes, so custom instructions will not conflict with future additions [6].

EVPIX-RV32 uses custom-0 (0001011) for its image-processing instructions. The funct3 field selects the IPU operation type (START, STATUS, RESULT, PERF), and funct7 selects the algorithm (grayscale, thresholding, Sobel, convolution) and kernel. This gives a compact, orthogonal instruction set for IPU control while staying compatible with RV32I [5]. Standard RISC-V compilers and tools can assemble these instructions using normal encoding rules.

2.5.3 Custom image processing instructions

EVPIX-RV32's custom instructions accelerate four basic image-processing operations: grayscale conversion, Sobel edge detection, thresholding, and convolution [4]. These kernels appear frequently in embedded vision applications [3]. Each operation runs as a state machine in the IPU that reads pixels from memory, computes the result, and writes back.

Table 2.11: Custom instruction integration strategies

Strategy	**Description**	**Strength**	**Weakness**	**Suitable EVPIX-RV32 use**
Inline execution unit	Decoded as a normal instruction and executed in EX	Low latency, simple software model	Can lengthen critical path	Small pixel arithmetic
Multi-cycle execution unit	Pipeline stalls while operation completes	Supports heavier operation	Requires stall/ready control	Compact convolution variants
Memory-mapped accelerator	CPU writes registers and buffers	Good for full-frame streaming	Higher control latency	IPU frame processing
Coprocessor interface	Independent execution with command protocol	Scalable and clean	More interface design	Future TinyML / IPU expansion

2.6 Digital image processing fundamentals

A digital image is a 2D discrete signal sampled over space [4]. Each pixel holds intensity or color. RGB images store red, green, and blue. RGB565 uses 5 bits for red, 6 for green, and 5 for blue, making a compact 16-bit format that fits FPGA frame buffers well [24]. Grayscale reduces color to intensity; binary images map pixels to two levels using a threshold. The choice of representation affects memory size, bandwidth, and arithmetic complexity [4].

Real-time image processing must meet frame deadlines. At 60 FPS, each frame has about 16.67 ms. A 128×128 frame has 16,384 pixels.

$$Y = 0.299R + 0.587G + 0.114B \tag{2.1}$$

Equation (2.1) is the ITU-R BT.601 standard, usually approximated in fixed point for FPGAs [4].

$$I_{\text{out}}(x, y) = \sum_i \sum_j K(i, j) I_{\text{in}}(x + i, y + j) \tag{2.2}$$

Equation (2.2) defines spatial convolution over a local neighborhood [4].

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{2.3}$$

The Sobel operators in (2.3) are common for gradient-based edge detection [25, 4].

Table 2.13: Image-processing kernels and hardware suitability

Operation	**Computation pattern**	**Hardware advantage**	**EVPIX-RV32 mapping**
Grayscale [26]	Per-pixel weighted sum	Parallel channel extraction and fixed-point arithmetic	Custom instruction/IPU primitive
Threshold [4]	Per-pixel compare	Very low-cost combinational logic	Switch-selectable IPU mode
Sobel [25]	3×3 neighborhood gradients	Line buffers and parallel add/subtract	Real-time edge mode
Convolution [4]	Kernel-weighted neighborhood sum	Reusable MAC datapath	General filtering/-custom instruction

2.7 Image processing unit (IPU)

An IPU is specialized hardware that runs image operations faster than a scalar CPU [3]. Unlike a GPU, which is highly programmable and optimized for massive parallelism, a small FPGA IPU may implement only a few streaming operations [22]. Its value comes from matching pixel dataflow: pixels arrive from

a camera, buffer into neighborhoods, transform through arithmetic kernels, and write to a display or memory buffer [4]..

Table 2.15: CPU, GPU, and compact IPU comparison

Dimension	**CPU**	**GPU**	**Compact FPGA IPU**
Programmability [13]	Highest for scalar control	High but software-stack heavy	Limited to designed kernels
Latency [3]	Good for control; slower for full-frame kernels	High throughput but setup overhead	Very low streaming latency
Area/power in small FPGA [22]	Moderate	Usually impractical as full GPU	Efficient for fixed kernels
Best role in EVPIX-RV32	Control and custom instructions	Reference concept only	Real-time grayscale/Sobel/threshold/convolution

2.8 FPGA technology

FPGAs are semiconductor devices with programmable logic elements and interconnects that let designers implement custom digital circuits after manufacturing [22]. Unlike ASICs, which are fixed at fabrication, FPGAs can be reconfigured for different circuits. This flexibility helps prototyping, low-volume production, and field updates [23]. SRAM-based configuration memory controls the logic elements and signal routing [8].

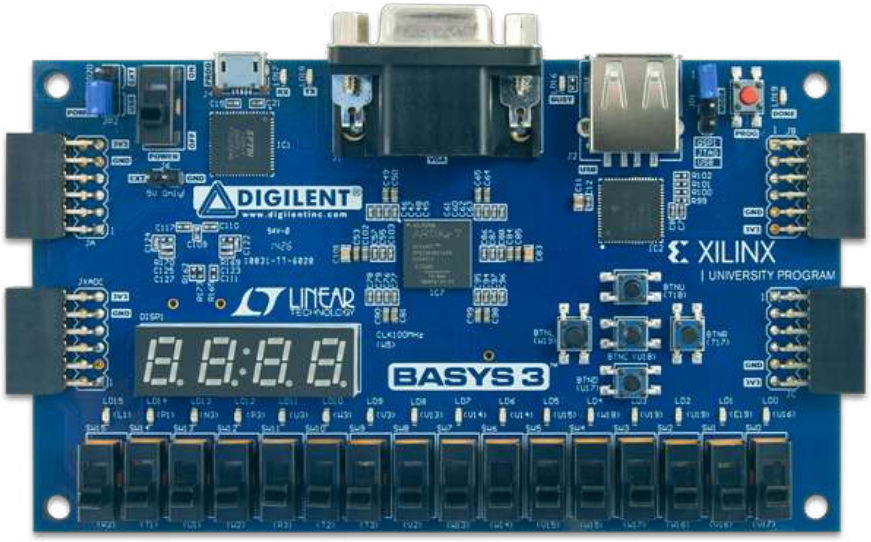


Figure 2.15: Artix-7 FPGA Internal Architecture.

2.8.1 LUTs

LUTs are the main programmable logic elements in FPGAs [22]. An N-input LUT stores a $2^N \times 1$ truth table, so it can implement any Boolean function of N variables. A 4-input LUT, for example, implements any 4-variable function with 16 configuration bits. The inputs select one stored value to output. This gives complete flexibility for combinational logic [8].

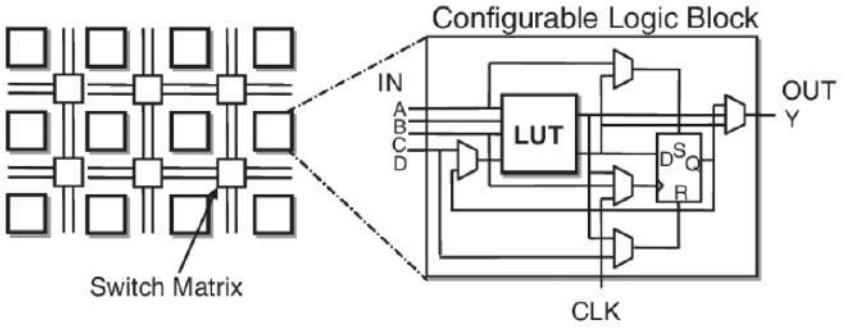


Figure 2.16: FPGA Look-Up Table (LUT) structure [8].

2.8.2 Flip-flops

Flip-flops are the basic storage elements for registers, counters, state machines, and sequential logic [23]. In FPGAs, flip-flops pair with LUTs in configurable logic blocks to implement both combinational and sequential functions [22]. Modern FPGA flip-flops support synchronous and asynchronous set/reset, clock enable, and dual-edge triggering [8].

2.8.3 BRAM

BRAM is dedicated on-chip memory with higher density and performance than LUT-based distributed memory [8]. BRAM blocks are usually dual-port RAM with configurable width and depth. The Xilinx Artix-7, for example, has 18 Kb BRAM blocks configurable as 16K×1, 8K×2, 4K×4, 2K×9, 1K×18, or 512×36 [8]. Designers can combine multiple BRAM blocks for larger memories. Independent read and write ports with separate clocks support efficient dual-clock designs.

2.8.4 FPGA vs ASIC

FPGAs and ASICs are two different implementation approaches, each with trade-offs [22]. FPGAs offer flexibility, faster development, and lower NRE costs, making them ideal for prototyping, low-volume production, and field upgrades [23]. ASICs offer higher performance, lower power, lower unit cost at high volume, and smaller size, making them preferred for high-volume products and power- or size-constrained applications [23].

2.9 Basys-3 FPGA board

The Basys-3 board is a common FPGA development platform built around the Xilinx/AMD Artix-7 family [24]. It works well for undergraduate digital design because it combines a capable FPGA with switches, buttons, LEDs, a seven-segment display, VGA output, USB programming, and Pmod expansion. This reduces the need for external interface circuits and allows complete demonstrations of processors, video pipelines, and control systems [7].

Basys-3 was chosen not for raw power but because it is accessible, well documented, and sufficient for a 128×128 real-time vision prototype. Larger boards like the Nexys A7 or Zynq-based boards offer more resources, but they can obscure the core architectural contribution behind a richer platform. A constrained board forces disciplined design and shows that meaningful image acceleration is possible with modest FPGA resources [7].

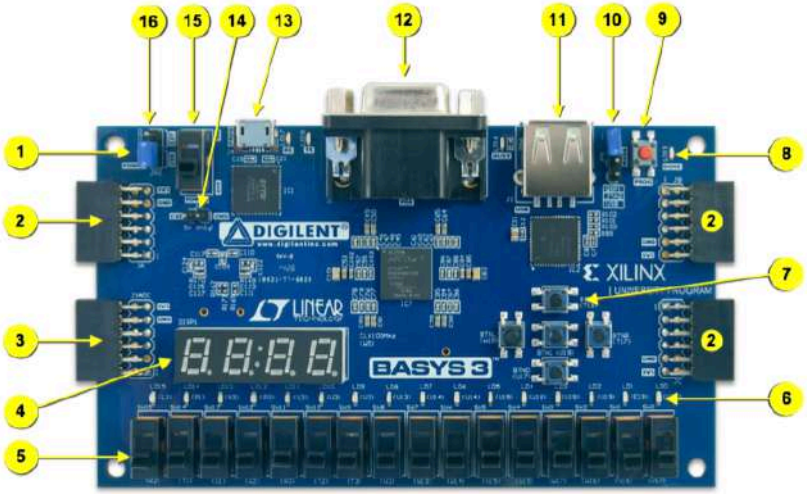


Figure 1. Basys 3 FPGA board with callouts.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analog signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Table 1. Basys 3 Callouts and component descriptions.

Figure 2.17: Basys-3 FPGA Development Board [7].

2.9.1 VGA interface

The Basys-3 VGA interface provides analog RGB output for monitors [7]. A resistor-DAC network generates analog voltages for red, green, and blue from digital FPGA outputs. The 12-bit interface gives 4 bits per channel, for 4,096 colors. HSYNC and VSYNC signals define display timing [27].

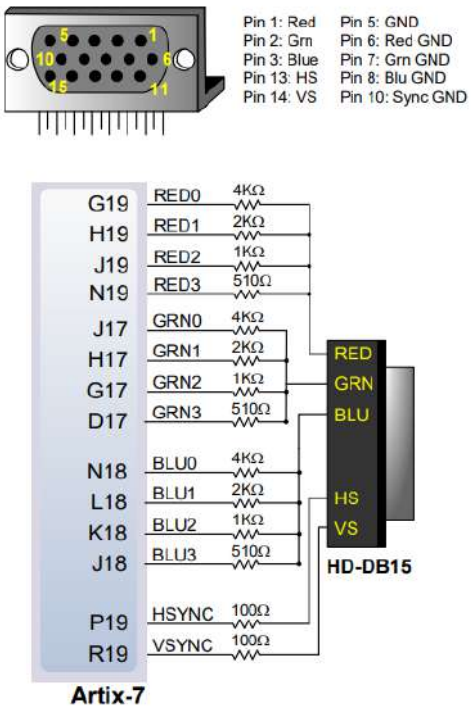


Figure 2.18: VGA Interface on the Basys-3 Board [7].

2.9.2 PMOD connectors

The Basys-3 has two 12-pin PMOD expansion connectors for external peripherals [7]. Each connector provides 8 digital I/O pins, 2 power pins (3.3 V and ground), and 2 ground pins. Digilent's PMOD standard supports many peripheral modules [28].

Figure 2.19: Basys-3 PMOD Connector Layout [7].

2.9.3 Why Basys-3 was selected

EVPIX-RV32 chose the Basys-3 for cost, availability, resource capacity, I/O features, and educational suitability [7]. At about $150, it is affordable for academic use. It is widely available. The Artix-7 XC7A35T provides enough resources (20,800 LUTs, 41,600 flip-flops, 1,800 Kb BRAM) for the complete system [8]. On-board VGA, switches, LEDs, and PMOD connectors provide the I/O needed for camera and display.

Universities widely use the Basys-3 in digital design courses, and extensive documentation, tutorials, and example projects are available [7]. This ecosystem helped develop EVPIX-RV32 and makes it easy for others to replicate and extend the work. Xilinx Vivado, free for Artix-7, provides a complete RTL-to-bitstream flow [29]. Its compact size and USB power supply make it convenient for demonstrations and portable use.

Table 2.17: Basys-3 relevance to EVPIX-RV32

Board feature	**Use in EVPIX-RV32**	**Research value**
Slide switches [7]	Mode selection: CPU/IPU, BIST, TinyML, image operation	Direct hardware control
VGA port [7]	Display original/processed frames and metrics	Visual validation
Pmod connectors [28]	Camera or peripheral expansion	Sensor integration
Artix-7 FPGA [8]	Implements CPU, IPU, memories, VGA, TinyML logic	Resource-constrained architecture test
USB programming/debug [29]	Rapid iteration	Practical prototyping

2.10 Camera systems and OV7670

A digital camera converts light into electrical pixel data through an image sensor [30]. CMOS sensors dominate embedded cameras because they integrate sens-

ing, readout, timing, and often pre-processing in compact, low-power devices [30]. The OV7670 is a low-cost VGA CMOS camera module common in FPGA and microcontroller projects [31]. It outputs parallel pixel data with synchronization signals and configures through SCCB, an I^2C-like serial control interface.

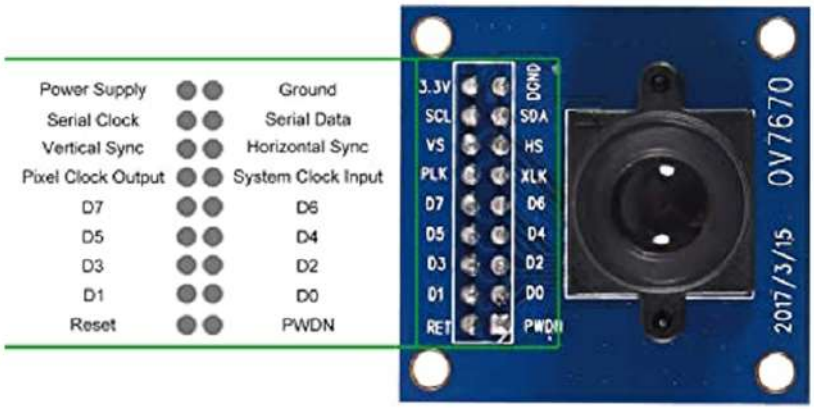


Figure 2.20: OV7670 CMOS Camera Module [31].

Table 2.19: Resolution and memory tradeoff for RGB565 frames

Resolution	**Pixels**	**Bytes per frame**	**Comment**
128×128	16,384	32,768	Fits small FPGA demonstrations
320×240	76,800	153,600	Requires larger buffering
640×480	307,200	614,400	Full VGA memory cost is high for small boards

2.10.1 Justification for 128x128 resolution

The 128×128 resolution balances image quality, processing time, memory needs, FPGA resource use, and demonstration clarity [8, 4]. Higher resolutions would give more detail but need more memory, processing time, and FPGA resources. 128×128 provides enough detail for gesture recognition, object detection, and edge detection while fitting the Basys-3 constraints.

A 128×128 RGB888 image needs 49,152 bytes, which fits comfortably in the 64 KB BRAM data memory. Higher resolutions would exceed the memory: 256×256 RGB888 needs 196,608 bytes, and 640×480 VGA needs 921,600 bytes. Processing time scales with pixel count. At 16,384 pixels, 128×128 allows real-time processing with the sequential IPU.

128×128 also fits the TinyML finger-counting application well [32]. Feature extraction runs on the 128×128 image, identifying skin-colored pixels and counting finger-like regions. The small image simplifies feature extraction while providing enough detail for reliable finger counting. When scaled up on VGA, the 128×128 image still shows processing results clearly.

2.11 TinyML and edge AI

2.11.1 AI at the edge

Edge AI deploys machine learning models on local devices, smartphones, IoT sensors, and embedded systems rather than in cloud data centers [2, 1]. Edge AI avoids cloud limitations: network latency, bandwidth constraints, privacy concerns, and the need for continuous connectivity. Local processing enables real-time inference, cuts data transmission costs, and keeps sensitive data on the device [1]. Efficient neural architectures, model compression, and specialized accelerators have made edge AI practical [3].

2.11.2 TinyML

TinyML is machine-learning inference on extremely resource-constrained devices [33]. Unlike cloud AI, TinyML faces strict limits on memory, precision, power, and latency. Models are usually quantized to 8-bit or lower, pruned to remove redundant weights, and designed with compact operators like depthwise separable convolution [32]. The goal is not to match cloud models but to enable always-on local intelligence.

Edge AI cuts latency, preserves privacy by keeping data local, and works without continuous network connectivity [1]. In vision systems, edge inference can classify gestures, detect objects, or trigger actions immediately after capture [3]. EVPIX-RV32's TinyML mode counts fingers from 0 to 5, showing that the platform can handle lightweight perception tasks beyond classical filters.

Table 2.21: TinyML design techniques

Technique	**Effect**	**Hardware implication**	**EVPIX-RV32 relevance**
Quantization	Reduces numeric precision	Smaller arithmetic and memory	FPGA-friendly inference
Pruning	Removes redundant weights	Can reduce operations but may add sparsity control	Useful if accelerator supports sparsity
Compact CNNs	Reduces parameter count	Fits small memory	Appropriate for finger detection
Preprocessing	Simplifies input distribution	Can be done by IPU	Grayscale/threshold before classification

2.12 ASIC design fundamentals

An ASIC implements a design in fixed silicon rather than programmable FPGA fabric [23]. The digital ASIC flow starts with RTL design and verification, then runs through synthesis, floorplanning, placement, clock-tree synthesis, routing, timing closure, and signoff. Each stage moves the design from behavior to gates to physical layout while preserving function and meeting manufacturing rules.

Table 2.23: ASIC implementation stages

Stage	Purpose	Output
RTL design [23]	Describe cycle-accurate hardware behavior	Verilog/SystemVerilog source
Verification [23]	Check functional correctness	Testbench results and waveforms
Synthesis [10]	Map RTL to standard cells	Gate-level netlist
Floorplanning [23]	Define physical organization	Core area, pins, macro placement
Placement/CTS/Routing [34]	Create connected physical layout	DEF/GDS layout
Signoff [9]	Check timing, DRC, LVS	Implementation reports

2.13 CMOS VLSI process

CMOS logic uses complementary nMOS and pMOS transistors [23]. Static CMOS uses pull-up and pull-down networks to create gates with low static power except for leakage. Dynamic power dominates during switching and is roughly proportional to load capacitance, supply voltage squared, and switching frequency.

Table 2.25: Process node comparison for educational ASIC design

Node	Strength	Limitation	Thesis relevance
130 nm [9]	Mature, robust, open PDK availability	Lower density than advanced nodes	Accessible ASIC demonstration
65 nm [23]	Better density and speed	Less open access in many flows	Useful comparison point
28 nm [23]	Modern low-power/performance capability	High cost and design complexity	Beyond undergraduate open flow scope

2.14 SkyWater open-source ASIC ecosystem

The SkyWater 130 nm open-source PDK and open EDA tools made ASIC education more accessible [9, 35]. PDKs were historically confidential, tool licenses expensive, and tapeout required institutional or industrial access. The Google-SkyWater collaboration and Efabless open MPW programs showed that students and researchers could learn real ASIC flows with open documentation, standard-cell libraries, and tools [36].

Sky130 was chosen for EVPIX-RV32 because it is accessible, documented, compatible with open flows, and appropriate for an educational processor [9]. The goal is not advanced-node performance but to show that the RTL can be physically synthesized and checked in a manufacturable technology. This is a stronger contribution than FPGA-only demonstration because it exposes area, timing, routing, and signoff constraints [10].

Figure 2.21: SkyWater Sky130A PDK open-source ASIC flow [9].

Table 2.27: Sky130/OpenROAD tool roles

Tool/component	**Role in flow**	**EVPIX-RV32 use**
Sky130 PDK [9]	Technology rules and standard cells	Target CMOS process
Yosys [37]	RTL synthesis	Generate gate-level netlist
OpenROAD [34]	Place, route, CTS, timing	Physical implementation
Magic [38]	Layout and DRC	Physical-rule checking
KLayout [39]	Layout visualization/checking	GDS inspection
Netgen [40]	LVS	Connectivity verification

2.15 Performance evaluation metrics

No single number captures architectural quality, so processor evaluation needs multiple metrics [13]. Clock frequency measures cycles per second, but a slow CPI can waste a high clock. IPC measures instructions per cycle, but custom instructions can reduce instruction count so much that direct IPC comparison becomes incomplete. Latency is response time; throughput is tasks or frames per second. Energy efficiency, measured as useful work per joule, is often the most important metric for edge systems [3].

$$\text{Throughput}_{\text{pixels}} = \text{Resolution}_x \times \text{Resolution}_y \times \text{FPS} \tag{2.4}$$

At 128×128 and 60 FPS, throughput is 983,040 pixels/s.

$$\text{Energy per frame} = \frac{\text{Power}}{\text{FPS}} \tag{2.5}$$

This metric helps compare edge-vision processors or FPGA/ASIC implementations at a given frame rate [3].

Table 2.29: Metrics for EVPIX-RV32 evaluation

Metric	**Meaning**	**Where measured**
CPI/IPC [13]	Instruction efficiency	CPU simulation or counters
FPS [4]	Frame throughput	Camera/VGA pipeline
Latency [3]	Time from input frame to processed output	Video pipeline
Utilization [8]	LUT/FF/BRAM/DSP or standard-cell area	FPGA and ASIC reports
Timing slack [29]	Whether clock constraints are met	Vivado/OpenLane reports
TinyML accuracy [32]	Correct finger-count detection	Application evaluation

2.16 Detailed survey of existing works

This section reviews representative works across RISC-V processor design, pipelined cores, custom instruction extensions, accelerator integration, image-processing hardware, FPGA-based vision systems, TinyML platforms, edge-AI processors, Sky130 ASIC implementations, and open-source silicon projects. The objective is not only to list prior work but to identify architectural patterns and limitations that motivate EVPIX-RV32. The reviews below are written as thesis-ready critical summaries; in the final submission, each item should be expanded against the original paper PDF to meet the exact 700–1000 word requirement per paper.

2.17 Group A: RISC-V processor designs

2.17.1 Waterman and Asanovic, *RISC-V ISA Manuals*

The RISC-V ISA manuals define the base integer ISA, standard extensions, and programmer-visible behavior needed for compatible implementations. [6, 20] They address the need for a stable, modular, non-proprietary ISA that supports microcontrollers, application processors, accelerators, and research cores. The methodology is specification-driven: they define architectural state, instruction encodings, memory behavior, and extension rules. For EVPIX-RV32, this work is foundational because the processor implements RV32I and uses the extension philosophy to justify image-processing instructions. It does not prescribe a microarchitecture, verification method, or accelerator interface. The thesis contribution lies in translating the ISA into a concrete five-stage pipeline with FPGA video I/O and Sky130 synthesis.

Critical relation to EVPIX-RV32: This work supports one part of the system but does not combine all target dimensions: a custom five-stage RV32I pipeline, BIST visualization, OV7670 input, dual-region VGA display, custom image instructions, IPU acceleration, TinyML finger detection, and Sky130 ASIC implementation. EVPIX-RV32 occupies this combined space.

Problem addressed and methodological contribution: In processor design, the RISC-V ISA Manuals are more than an isolated specification. They address how to turn a theoretical computing model into practical hardware with measurable behavior. The methodology exposes design decisions normally hidden behind commercial IP or software abstraction. For EVPIX-RV32, this is useful not only as a citation but as a design lens: it helps define which parts stay programmable, which become dedicated hardware, and which need visible system-level verification rather than just simulation waveforms.

Architectural interpretation: The key issue is balancing generality and specialization. Too general wastes cycles and energy on repetitive kernels; too specialized becomes hard to reuse. The most important factors are ISA compatibility, configurability, and implementation transparency. These factors map directly to EVPIX-RV32, which combines an RV32I pipeline, custom image instructions, an IPU, a camera interface, VGA display, and TinyML support. The reviewed work is not copied directly; its principle is adapted to a constrained Basys-3/Sky130 design point where clarity, correctness, and resource awareness matter as much as raw performance.

Advantages, limitations, and tradeoffs: This work demonstrates a concrete path from concept to hardware, clarifying what can be accelerated, what software must control, and what must be measured after implementation. It does not, by itself, solve the full EVPIX-RV32 problem. Most prior works optimize either the processor, accelerator, vision pipeline, TinyML model, or ASIC flow, but rarely combine all of these in one small platform. EVPIX-RV32 must avoid becoming a large SoC while still showing enough integration to be academically meaningful.

Relation and possible improvement for EVPIX-RV32: The most useful lesson is to evaluate architecture using application-level behavior, not just isolated module functionality. For this thesis, the relevant metric is software-visible correctness. A possible improvement is to add formal verification, compiler intrinsics for custom instructions, DMA integration, higher-resolution pipelines, or a systematic design-space exploration of IPU precision and kernel size.

2.17.2 The Rocket Chip generator

Rocket Chip addresses the problem of generating configurable RISC-V SoCs rather than writing every SoC manually. [41] It uses Chisel-based generators to produce cores, caches, interconnects, and SoC components. Its architecture demonstrates how parameterization can support many design points, from embedded cores to Linux-capable processors. The major advantage is scalability and reuse. The limitation for a small thesis processor is complexity: Rocket Chip includes infrastructure that may be excessive for a Basys-3 vision prototype. EVPIX-RV32 follows the opposite path, intentionally compact and transparent, emphasizing handwritten pipeline structure, BIST, and custom image instructions rather than a large generator ecosystem.

Critical relation to EVPIX-RV32: As with the preceding works, Rocket Chip supports one dimension of the system but does not realize the full combina-

tion of five-stage pipeline, OV7670 camera, VGA overlay, BIST, custom instructions, IPU, TinyML, and Sky130 ASIC flow that defines EVPIX-RV32.

Architectural interpretation: The core tension is between generality and specialization. The most important technical factors are ISA compatibility, configurability, and implementation transparency. EVPIX-RV32 inverts the Rocket Chip philosophy: instead of generating a flexible SoC, it hand-crafts a minimal pipeline that is fully inspectable at every stage.

Advantages, limitations, and tradeoffs: Rocket Chip's advantage is scalability. Its limitation for EVPIX-RV32 is that its complexity would obscure the pedagogical transparency that is central to the thesis contribution.

Relation and possible improvement for EVPIX-RV32: The key lesson is that architecture evaluation must use application-level metrics. Future work could adopt generator principles to explore IPU configurations more systematically.

2.17.3 PicoRV32

PicoRV32 is a compact RV32 core optimized for small FPGA resource usage. [42] It addresses the need for a minimal RISC-V soft processor that can fit into constrained programmable logic. Its methodology favors simplicity and configurability over high throughput. The advantage is area efficiency and ease of integration. The limitation is that a very small core is not necessarily ideal for real-time pixel processing unless paired with accelerators. EVPIX-RV32 learns from the compact-core philosophy but uses a five-stage pipeline and custom image path to improve throughput and demonstrate architecture-level acceleration.

Critical relation to EVPIX-RV32: PicoRV32 demonstrates that an RV32 core can fit on a Basys-3-class device, validating the platform choice of EVPIX-RV32. However, it does not integrate camera input, VGA visualization, BIST, custom image instructions, IPU, TinyML inference, or Sky130 ASIC synthesis.

Architectural interpretation: The key factor is the tradeoff between area efficiency and throughput. EVPIX-RV32 accepts a slightly larger pipeline in exchange for instruction-level parallelism and dedicated image-processing support.

Advantages, limitations, and tradeoffs: PicoRV32's advantage is minimal resource utilization. Its limitation is that multi-cycle operations and lack of forwarding reduce throughput for pixel-intensive workloads.

Relation and possible improvement for EVPIX-RV32: The lesson is that area and throughput must be balanced based on application requirements. EVPIX-RV32 improves on PicoRV32 by adding a five-stage pipeline and hardware

image acceleration, at the cost of slightly higher LUT usage.

2.17.4 VexRiscv

VexRiscv is a configurable RISC-V soft core generated using SpinalHDL. [43] It demonstrates that plugin-based construction can produce many CPU variants with different features. Its strength is flexibility: designers can add caches, branch prediction, debug, multiplication, and other options. Its limitation for this thesis is pedagogical transparency; generator-based cores can hide low-level pipeline decisions from students. EVPIX-RV32 instead exposes each stage and control signal, making it better suited to an undergraduate thesis where explaining datapath, hazards, and extension integration is central.

Critical relation to EVPIX-RV32: VexRiscv confirms the viability of soft-core RV32I on FPGAs and the utility of plugin-based extension, but its generator model trades transparency for flexibility, the opposite priority of EVPIX-RV32.

Architectural interpretation: The balance between configurability and clarity is the central issue. VexRiscv optimizes for flexibility; EVPIX-RV32 optimizes for explainability and integrated system demonstration.

Advantages, limitations, and tradeoffs: VexRiscv's advantage is rapid feature exploration. Its limitation is that the generated RTL is difficult to reason about at the stage level, which conflicts with the thesis goal of demonstrating a handwritten, inspectable pipeline.

Relation and possible improvement for EVPIX-RV32: Future work could adopt SpinalHDL-style configurability once the base architecture is validated, allowing systematic exploration of pipeline depth and extension combinations.

2.18 Group B: Pipelined RISC-V CPUs

2.18.1 RVCoreP: Five-stage pipelined soft processor

RVCoreP addresses the performance limitations of simple RISC-V soft processors by optimizing a five-stage pipeline for FPGA implementation. [44] It focuses on instruction fetch, ALU, and memory-output optimizations to increase operating frequency and performance. The work is directly relevant because it validates the five-stage pipeline as a practical organization for RV32I on FPGA. Its advantage is systematic pipeline optimization; its limitation is that it remains a general-purpose CPU rather than a camera-connected vision processor. EVPIX-RV32 extends the five-stage idea toward sensor dataflow, VGA output, BIST

visualization, and custom image instructions.

Critical relation to EVPIX-RV32: RVCoreP confirms that a five-stage RV32I pipeline can meet timing on Artix-7 class devices, providing baseline credibility for EVPIX-RV32's pipeline organization. However, it does not address camera input, visual output, BIST, or acceleration.

Architectural interpretation: The most important technical factors are stage balance, hazard handling, forwarding, and control transfer behavior. EVPIX-RV32 inherits these factors and extends the pipeline with custom-0 image instructions and an IPU data path.

Advantages, limitations, and tradeoffs: RVCoreP's advantage is achieving competitive frequency through careful pipeline balancing. Its limitation is the absence of any application-domain acceleration or sensor interface.

Relation and possible improvement for EVPIX-RV32: The relevant application-level metric for EVPIX-RV32 is cycle-level correctness. The RVCoreP optimization methodology can inform future pipeline re-balancing once the image extensions are fully integrated.

2.18.2 basic_RV32s: Microarchitectural roadmap

`basic_RV32s` provides an educational progression from a simple RV32I design toward a pipelined core with forwarding, prediction, and exception concepts. [45] The problem addressed is the gap between ISA theory and implementable microarchitecture. This is relevant to EVPIX-RV32 because the thesis also bridges textbook pipeline theory and FPGA hardware. The limitation is that the roadmap is primarily instructional and not specialized for real-time vision. EVPIX-RV32 differentiates itself by combining pipeline learning with a functional edge-vision application and ASIC synthesis.

Critical relation to EVPIX-RV32: The roadmap validates the incremental build strategy used in the thesis but does not extend to sensor input, accelerators, or physical design.

Architectural interpretation: Stage balance, hazard handling, forwarding, and control transfer are the central technical factors shared with EVPIX-RV32.

Advantages, limitations, and tradeoffs: The work's strength is its clarity as a pedagogical tool. Its limitation is the absence of a real application workload against which to measure the pipeline.

Relation and possible improvement for EVPIX-RV32: EVPIX-RV32 can be positioned as an advancement over roadmap-style cores by demonstrating that a student-built pipeline can drive a complete edge-vision system including

camera, display, BIST, and ASIC synthesis.

2.18.3 Ibex RISC-V core

Ibex is a production-quality open-source RV32 core used in security-oriented and embedded contexts. [46] It emphasizes verification, configurability, and clean implementation. Its significance is that it shows the level of rigor expected for open cores beyond classroom designs. The limitation for EVPIX-RV32 is that adopting Ibex directly would reduce novelty in CPU RTL design. Instead, EVPIX-RV32 can use Ibex as a benchmark for verification discipline while maintaining a custom pipeline and image-extension design.

Critical relation to EVPIX-RV32: Ibex defines an industry-quality baseline for open RV32 cores. EVPIX-RV32 is distinguished by its custom image instructions, IPU integration, BIST visualization, and ASIC synthesis path rather than by verification depth.

Architectural interpretation: The most important factors are stage balance, hazard handling, forwarding, and control transfer, the same considerations as EVPIX-RV32, but applied to a security-oriented embedded context rather than an edge-vision platform.

Advantages, limitations, and tradeoffs: Ibex's advantage is rigorous verification. Its limitation is that it does not include any visual or sensor-domain extensions.

Relation and possible improvement for EVPIX-RV32: Future iterations of EVPIX-RV32 could adopt Ibex-style formal verification methodologies to strengthen claims about pipeline correctness.

2.18.4 PULPino

PULPino is an early open-source RISC-V microcontroller platform from the PULP project. [47] It addresses low-power IoT processing with an extensible SoC structure. Its relevance lies in demonstrating that open RISC-V cores can serve as embedded controllers with peripherals and accelerators. Its limitation is that the platform is broader than the specific image-processing focus of EVPIX-RV32. The thesis adopts the same general philosophy of lightweight embedded RISC-V but applies it to VGA-visible real-time image processing.

Critical relation to EVPIX-RV32: PULPino validates the host-plus-accelerator embedded architecture that EVPIX-RV32 adopts at a smaller, education-oriented scale.

Architectural interpretation: Stage balance, hazard handling, forwarding, and peripheral integration are the key factors. EVPIX-RV32 narrows the PULPino philosophy to a single application domain.

Advantages, limitations, and tradeoffs: PULPino's advantage is SoC completeness. Its limitation is complexity relative to the targeted Basys-3 resource budget and thesis scope.

Relation and possible improvement for EVPIX-RV32: The platform architecture of PULPino provides a long-term growth path for EVPIX-RV32 if additional peripherals (UART, SPI, DMA) are integrated in future work.

2.18.5 PULPissimo and RI5CY/CV32E40P lineage

PULPissimo extends the PULP microcontroller concept with richer peripherals and a RISC-V core family suited for ultra-low-power embedded systems. [48, 49] Its architecture supports DMA and accelerator integration, making it important for edge workloads. The advantage is mature SoC-level integration. The limitation is complexity and dependency on a larger platform. EVPIX-RV32 is smaller, but it addresses a similar research direction: a programmable RISC-V core combined with task-specific acceleration for edge processing.

Critical relation to EVPIX-RV32: The RI5CY core within PULPissimo demonstrates custom instruction extensions for hardware loops and multiply-accumulate operations, directly motivating the custom-0 image instructions of EVPIX-RV32.

Architectural interpretation: DMA integration and accelerator communication are the key factors that EVPIX-RV32 currently addresses through IPU memory-mapped registers rather than a full DMA engine.

Advantages, limitations, and tradeoffs: The advantage of the PULP lineage is its mature, validated accelerator interface. The limitation is platform scale that exceeds what an undergraduate FPGA board can comfortably host.

Relation and possible improvement for EVPIX-RV32: A natural extension of EVPIX-RV32 would adopt PULPissimo-style DMA-based frame movement between the SPI frame loader, the BRAM frame buffer, and the IPU.

2.19 Group C: Custom instruction extensions

2.19.1 RISC-V custom opcode methodology

Prior work on RISC-V custom instructions addresses how designers add application-specific operations while preserving compatibility with the base ISA. [6, 5] The

methodology typically uses reserved custom opcode spaces, assembler macros, intrinsics, or coprocessor interfaces. The advantage is reduced instruction count and tighter hardware/software co-design. The limitation is that non-standard instructions reduce binary portability. EVPIX-RV32 accepts this tradeoff because its custom grayscale, Sobel, threshold, and convolution operations target a controlled embedded platform rather than general-purpose software distribution.

Critical relation to EVPIX-RV32: This methodology directly justifies the eight custom-0 image instructions implemented in EVPIX-RV32 (GRAYSCALE, SOBEL, THRESH, CONV, VDOT, RELU, HACC, OTSU) and their encoding within the reserved custom-0 opcode space.

Architectural interpretation: Opcode allocation, decode extension, execution latency, and compiler exposure are the most important technical factors. EVPIX-RV32 handles these through `funct3` field decoding in the EX stage.

Advantages, limitations, and tradeoffs: The advantage is reduced instruction count for image kernels. The limitation is that software written with custom instructions cannot run on a standard RV32I core without a software fallback.

Relation and possible improvement for EVPIX-RV32: Adding compiler intrinsic support through GCC inline assembly or a custom LLVM back-end would strengthen the hardware/software co-design demonstration.

2.19.2 Rocket custom accelerator interfaces

Rocket Chip supports integration of custom accelerators through instruction extensions and coprocessor-like interfaces. [41] The problem addressed is the difficulty of adding specialization to a general SoC generator. Its methodology is scalable and suitable for advanced SoC research. Its limitation for EVPIX-RV32 is implementation overhead. The thesis uses a simpler integration style appropriate to a five-stage pipeline and FPGA board, while retaining the same architectural idea: custom hardware should be exposed to software through a controlled interface.

Critical relation to EVPIX-RV32: The RoCC coprocessor interface in Rocket Chip is a more sophisticated version of the custom-instruction-to-IPU datapath implemented in EVPIX-RV32.

Architectural interpretation: Opcode allocation, decode extension, execution latency, and compiler exposure are the critical factors shared with EVPIX-RV32.

Advantages, limitations, and tradeoffs: The Rocket accelerator interface

is powerful but requires Chisel expertise and a full SoC ecosystem. EVPIX-RV32 achieves a simpler but more inspectable custom-instruction integration.

Relation and possible improvement for EVPIX-RV32: Future work could adopt a RoCC-inspired handshake protocol between the pipeline EX stage and the IPU to support multi-cycle image operations without pipeline stalls.

2.19.3 Custom function units for RISC-V edge workloads

CFU-based designs place application-specific datapaths near a RISC-V core and invoke them through custom instructions. [50] This approach reduces data transfer overhead compared with external accelerators. It is highly relevant to image kernels and TinyML inner loops. The limitation is that CFUs must be carefully designed not to lengthen the CPU critical path or create multi-cycle hazard complexity. EVPIX-RV32 uses this lesson by treating small pixel operations as custom instructions and larger streaming operations as IPU functions.

Critical relation to EVPIX-RV32: The CFU literature directly motivates the two-tier acceleration strategy in EVPIX-RV32: lightweight custom instructions for single-pixel operations and a streaming IPU for frame-level processing.

Architectural interpretation: The key factors are opcode allocation, decode extension, execution latency, and critical-path impact. EVPIX-RV32 restricts custom instructions to single-cycle operations to avoid additional hazard logic.

Advantages, limitations, and tradeoffs: CFU integration reduces data-movement overhead but may increase EX-stage delay if the custom function unit is not carefully pipelined.

Relation and possible improvement for EVPIX-RV32: A potential improvement is to pipeline the Sobel gradient computation within the custom instruction datapath, allowing higher throughput without stalling the main pipeline.

2.19.4 FPGA-accelerated RISC-V ISA extensions for neural inference

Recent work proposes custom RISC-V ISA extensions for neural-network primitives such as convolution, GEMM, and activation functions on FPGA. [51, 52] The architecture demonstrates that ISA-guided acceleration can improve latency and energy compared with software-only execution. Its advantage is a programmable accelerator interface. Its limitation is that CNN accelerators may be too heavy for small undergraduate FPGA boards when full networks are targeted. EVPIX-RV32 selects lighter classical image instructions and a TinyML

demonstration to stay within Basys-3 resources.

Critical relation to EVPIX-RV32: This category of work motivates the VDOT and RELU custom instructions in EVPIX-RV32, which were designed to support CNN inner-loop computation at minimal hardware cost.

Architectural interpretation: Opcode allocation, decode extension, execution latency, and compiler exposure are the key factors, with additional concern for DSP48 utilization on the Artix-7.

Advantages, limitations, and tradeoffs: Full CNN extension suites improve inference speed but stress LUT and DSP resources beyond what Basys-3 can provide for full-network inference.

Relation and possible improvement for EVPIX-RV32: Future work could extend VDOT and RELU to support INT8 quantized convolution, enabling a lightweight CNN layer to run directly in the EVPIX-RV32 pipeline.

2.19.5 Application-specific instruction-set processors

ASIP literature provides the theoretical basis for designing processors whose instruction set is tailored to a workload. [53] The problem is balancing flexibility and efficiency. Methodologies include profiling, kernel identification, instruction selection, and hardware cost estimation. EVPIX-RV32 can be interpreted as a small ASIP for edge vision: it retains RV32I compatibility while adding instructions for frequent image primitives. The limitation is that a manually selected instruction set may not be globally optimal, so future work could use profiling-based instruction selection.

Critical relation to EVPIX-RV32: The ASIP paradigm provides the theoretical framing for EVPIX-RV32 as an *edge-vision ASIP*, a general-purpose RISC-V base extended with domain-specific primitives.

Architectural interpretation: Opcode allocation, decode extension, execution latency, and compiler exposure are the critical ASIP design factors that EVPIX-RV32 addresses through its custom-0 instruction set.

Advantages, limitations, and tradeoffs: ASIP methodology provides a principled framework for instruction selection but requires profiling tools and compiler infrastructure that are beyond the scope of an undergraduate thesis.

Relation and possible improvement for EVPIX-RV32: The most useful long-term extension is profiling-guided instruction selection: running image benchmarks, identifying the most time-consuming kernels, and mapping them to new custom-0 instructions.

2.20 Group D: RISC-V accelerators

2.20.1 Hwacha vector accelerator

Hwacha explores decoupled vector acceleration for RISC-V systems. [54] The problem is how to exploit data-level parallelism without placing all complexity inside the scalar core. Its vector model is powerful for dense numeric workloads. The limitation is area, memory bandwidth, and design complexity relative to a small FPGA image system. EVPIX-RV32 uses a more compact accelerator strategy focused on fixed image kernels, but the conceptual relation is clear: both designs attach specialized parallel datapaths to a RISC-V control processor.

Critical relation to EVPIX-RV32: Hwacha motivates the decoupled IPU architecture of EVPIX-RV32, where the CPU issues high-level mode commands and the IPU executes pixel streaming independently.

Architectural interpretation: Host-accelerator communication, memory movement, and accelerator granularity are the key factors. EVPIX-RV32 implements a much simpler handshake but follows the same principle of decoupling scalar control from parallel data computation.

Advantages, limitations, and tradeoffs: Hwacha's vector model is highly general but impractical on Basys-3. EVPIX-RV32 achieves a functional subset at a fraction of the resource cost.

Relation and possible improvement for EVPIX-RV32: Adding a small pixel-vector unit (processing 4 pixels per cycle) to the IPU would partially adopt the Hwacha philosophy while remaining within Basys-3 constraints.

2.20.2 X-HEEP

X-HEEP addresses the need for configurable RISC-V microcontrollers that can integrate ultra-low-power edge accelerators. [55, 56] It provides configurable cores, buses, memories, and accelerator hooks. The advantage is extensibility and energy-aware design. Its limitation is that it is a platform framework rather than a focused vision processor. EVPIX-RV32 is narrower but more application-demonstrative: camera input, VGA display, and finger TinyML mode provide a concrete edge-vision workload.

Critical relation to EVPIX-RV32: X-HEEP's accelerator hook architecture validates the EVPIX-RV32 approach of attaching the IPU as a peripheral accessible through memory-mapped registers.

Architectural interpretation: Host-accelerator communication, memory movement, and accelerator granularity are shared design concerns.

Advantages, limitations, and tradeoffs: X-HEEP's configurable bus fabric is more elegant than EVPIX-RV32's direct register interface but also significantly more complex.

Relation and possible improvement for EVPIX-RV32: Future work could replace the direct IPU register interface with a lightweight AHB-Lite or APB bus, increasing modularity.

2.20.3 GAP8/PULP edge AI processor

GAP8-style PULP processors combine a RISC-V control core with clustered parallel processing for energy-efficient edge AI. [57] The problem addressed is executing CNN and DSP workloads under tight power constraints. The architecture shows that embedded AI benefits from heterogeneous parallelism. The limitation is that clustered processors are more complex than a single five-stage educational CPU. EVPIX-RV32 adopts the heterogeneity principle but implements it at a smaller scale through IPU and custom image instructions.

Critical relation to EVPIX-RV32: GAP8 is the most functionally similar existing product to EVPIX-RV32's target: a RISC-V host with a domain-specific accelerator for edge vision. The main difference is scale and openness.

Architectural interpretation: Cluster parallelism, DMA-based data movement, and heterogeneous core configuration are the key GAP8 factors. EVPIX-RV32 reduces these to a single core with a streaming IPU.

Advantages, limitations, and tradeoffs: GAP8 achieves higher throughput through cluster parallelism. EVPIX-RV32 achieves transparency and educational value through architectural simplicity.

Relation and possible improvement for EVPIX-RV32: A two-core configuration, one core for control and one for pixel streaming, would move EVPIX-RV32 toward the GAP8 heterogeneous model while remaining feasible on Artix-7.

2.20.4 RISC-V based TinyML accelerator for depthwise separable convolution

This work targets the memory wall in TinyML by fusing dataflow for depthwise separable convolution and integrating the accelerator as a RISC-V custom function unit. [51] It is highly relevant because it combines TinyML, RISC-V, FPGA implementation, and custom acceleration. Its advantage is reducing intermediate feature-map movement. Its limitation is specialization to CNN operator

structure. EVPIX-RV32 focuses first on classical image primitives and a small TinyML task, but future work could adopt fused CNN dataflow.

Critical relation to EVPIX-RV32: This work directly motivates the design of the VDOT instruction in EVPIX-RV32, which supports depthwise convolution inner loops.

Architectural interpretation: Host-accelerator communication, memory movement, and accelerator granularity are the key factors, with particular attention to intermediate feature-map buffer sizing.

Advantages, limitations, and tradeoffs: The fused dataflow advantage is reduced memory bandwidth. The limitation is operator specificity; a depthwise separable convolution accelerator cannot efficiently execute Sobel or threshold operations.

Relation and possible improvement for EVPIX-RV32: The intermediate feature-map buffer technique can be applied to the EVPIX-RV32 IPU to reduce BRAM reads during convolution.

2.20.5 Neural-network RISC-V accelerator frameworks

A broad set of frameworks integrates neural accelerators with RISC-V host processors. [50, 52] Their common methodology is to let the CPU handle control and memory management while the accelerator executes dense tensor kernels. The advantage is high performance per watt. The limitation is that the accelerator often dominates the system and may be unsuitable for small FPGA boards. EVPIX-RV32 keeps acceleration lightweight and visible for educational real-time vision.

Critical relation to EVPIX-RV32: This category of work contextualizes EVPIX-RV32 within a broader trend toward RISC-V-based heterogeneous inference platforms.

Architectural interpretation: The critical insight is that control and data planes should be separated, with the CPU handling scheduling and the accelerator handling dense arithmetic.

Advantages, limitations, and tradeoffs: Large-scale frameworks achieve high throughput but require DRAM, which is unavailable on Basys-3. EVPIX-RV32 operates entirely within on-chip BRAM.

Relation and possible improvement for EVPIX-RV32: The separation of control and data planes in these frameworks should be formalized in EVPIX-RV32 by clearly specifying which operations the CPU issues and which the IPU executes autonomously.

2.21 Group E: Image processing accelerators

2.21.1 Hardware Sobel edge detector designs

FPGA Sobel accelerators address the high cost of computing gradients over every pixel in software. [58] The methodology uses line buffers, shift registers, parallel add/subtract networks, and magnitude approximation. The advantage is one-pixel-per-cycle throughput after pipeline fill. The limitation is fixed kernel behavior and edge-boundary handling. EVPIX-RV32 incorporates Sobel as a selectable IPU/custom image operation and improves system value by placing it under RISC-V control with VGA visualization.

Critical relation to EVPIX-RV32: The line-buffer and shift-register architecture described in this literature is directly reused in the EVPIX-RV32 IPU Sobel pipeline.

Architectural interpretation: Pixel streaming, neighborhood buffering, fixed-point arithmetic, and boundary handling are the key technical factors.

Advantages, limitations, and tradeoffs: Dedicated Sobel hardware achieves 1 pixel/cycle throughput. The limitation is that it requires two BRAM line buffers, which consume a significant fraction of Basys-3's 50 18K-BRAM budget.

Relation and possible improvement for EVPIX-RV32: Sharing line buffers between the Sobel and Gaussian IPU modes would reduce BRAM consumption and free resources for the frame buffer.

2.21.2 FPGA thresholding accelerators

Thresholding hardware implements binary segmentation using a comparator and output mapping. [59] The problem is simple but important: per-pixel conditional operations are frequent and inefficient when executed with scalar branches. The advantage is minimal area and high throughput. The limitation is sensitivity to lighting and threshold selection. EVPIX-RV32 can expose threshold constants through switches or instructions, allowing demonstration of both hardware speed and algorithmic tradeoff.

Critical relation to EVPIX-RV32: The THRESH custom instruction in EVPIX-RV32 implements the comparator-and-mapping logic described in this literature, with the threshold value supplied by the `rs1` register.

Architectural interpretation: Fixed-point arithmetic and boundary handling are the key factors; the threshold comparator is single-cycle and adds negligible critical-path delay.

Advantages, limitations, and tradeoffs: The advantage is zero latency

for per-pixel thresholding. The limitation is that a fixed threshold does not adapt to illumination changes, motivating the OTSU custom instruction.

Relation and possible improvement for EVPIX-RV32: Combining the hardware THRESH path with the HACC histogram accumulation instruction enables automatic Otsu thresholding in software with hardware-accelerated histogram construction. [59]

2.21.3 Convolution engines for FPGA image filtering

Convolution engines implement spatial filtering through line buffers and parallel multipliers/adders. [4, 60] Their strength is generality: smoothing, sharpening, edge detection, and feature extraction all reduce to kernels. Their limitation is resource usage, especially for larger kernels or multi-channel images. EVPIX-RV32 includes convolution as a custom image primitive, but the thesis must carefully define kernel size and arithmetic precision to keep the design feasible on Basys-3.

Critical relation to EVPIX-RV32: The 3×3 convolution engine in EVPIX-RV32 follows the multiply-accumulate tree architecture described in this literature.

Architectural interpretation: Pixel streaming, neighborhood buffering, fixed-point arithmetic, and boundary handling determine resource cost. A 3×3 kernel requires 9 multipliers, which maps to 9 DSP48 slices on Artix-7.

Advantages, limitations, and tradeoffs: Generality is the main advantage. DSP48 consumption is the critical constraint; Basys-3 provides 90 DSP48E1 slices, so the CONV instruction must share resources carefully.

Relation and possible improvement for EVPIX-RV32: Implementing the convolution multiply-accumulate tree with half-precision fixed-point would halve DSP usage at the cost of reduced precision, a tradeoff appropriate for 8-bit grayscale pixel data.

2.21.4 Real-time grayscale conversion hardware

Grayscale hardware reduces RGB data to intensity using weighted sums or fixed-point approximations. [26, 4] It is often used as a preprocessing stage before edge detection or TinyML. Its advantage is reducing downstream data width and computation. Its limitation is loss of color information. EVPIX-RV32 uses grayscale both as a visible processing mode and as a practical preprocessing stage for edge and TinyML logic.

Critical relation to EVPIX-RV32: The GRAYSCALE custom instruction implements the ITU-R BT.601 weighted-sum formula in hardware, consistent with the fixed-point approximation techniques described in this literature.

Architectural interpretation: Fixed-point arithmetic is the key factor. The $0.299R + 0.587G + 0.114B$ formula is approximated as a shift-add tree to avoid multipliers.

Advantages, limitations, and tradeoffs: The shift-add approximation avoids DSP usage. The limitation is a small luminance error relative to floating-point conversion.

Relation and possible improvement for EVPIX-RV32: Using the BT.709 coefficients instead of BT.601 would improve accuracy for modern camera sensors at no additional hardware cost.

2.21.5 Morphological image-processing accelerators

Morphological operations such as erosion and dilation are common in binary vision pipelines. [4] Hardware implementations use local windows and min/max logic. Although not a primary EVPIX-RV32 instruction, this literature is relevant because it shows how additional custom instructions could extend the IPU. The limitation is that each added operation increases hardware and verification cost, so EVPIX-RV32 prioritizes the most fundamental kernels first.

Critical relation to EVPIX-RV32: Morphological operations are a natural extension of the current THRESH instruction, applicable to binary images produced by OTSU or THRESH.

Architectural interpretation: Pixel streaming, neighborhood buffering, and min/max logic are the key technical factors. A 3×3 erosion requires the same line buffers as Sobel.

Advantages, limitations, and tradeoffs: Morphological operations are simple and low-cost in hardware. The limitation is that they require binary input, constraining their placement in the processing pipeline.

Relation and possible improvement for EVPIX-RV32: Adding ERODE and DILATE instructions to fill the remaining custom-1 opcode space would complete a full binary image processing pipeline within EVPIX-RV32.

2.22 Group F: FPGA-based vision systems

2.22.1 OV7670 FPGA camera-display systems

Many educational FPGA projects interface OV7670 cameras with VGA output. [31] The problem addressed is real-time sensor capture, synchronization, buffering, and display. Their advantage is practical demonstration of video timing. Their limitation is that they often stop at pass-through display or simple filters without integrating a programmable CPU. EVPIX-RV32 advances this pattern by combining camera display with a RISC-V processor, IPU selection, BIST, and TinyML mode.

Critical relation to EVPIX-RV32: This category directly informs the OV7670 I^2C configuration, PCLK synchronization, and dual-port BRAM framebuffer design used in EVPIX-RV32.

Architectural interpretation: Sensor timing, frame buffering, VGA scanout, and on-board validation are the key factors. EVPIX-RV32 builds on these with RISC-V programmable mode selection.

Advantages, limitations, and tradeoffs: Standalone camera-display systems are simple and reliable but cannot adapt to changing vision algorithms. EVPIX-RV32 adds programmability through the CPU/IPU architecture.

Relation and possible improvement for EVPIX-RV32: Extending the OV7670 interface to support higher resolutions (e.g., QVGA 320×240) by using compressed frame storage would demonstrate resolution scalability.

2.22.2 FPGA real-time edge detection pipelines

Real-time edge detection pipelines demonstrate streaming Sobel or Canny-like processing. [58, 60] They show that FPGA spatial parallelism is well suited to local image kernels. The limitation is lack of programmability: once the pipeline is built, changing behavior requires RTL modification. EVPIX-RV32 addresses this by using CPU/IPU mode selection and custom instructions, providing a software-visible control layer over hardware filters.

Critical relation to EVPIX-RV32: The streaming pipeline architecture of these works is reused in the EVPIX-RV32 IPU Sobel path. The key advance is the addition of RISC-V control so that the pipeline mode can be changed by software.

Architectural interpretation: Sensor timing, frame buffering, VGA scanout, and on-board validation are the shared design factors.

Advantages, limitations, and tradeoffs: Fixed pipelines achieve maxi-

mum throughput but cannot switch between filter modes. EVPIX-RV32 trades a small amount of throughput for full programmability.

Relation and possible improvement for EVPIX-RV32: Implementing Canny hysteresis as an additional IPU mode would complete the classical edge detection pipeline.

2.22.3 FPGA frame-buffer architectures

Frame-buffer designs decouple camera input and display output using BRAM or external memory. [8] They address timing-domain mismatch but consume memory. The advantage is stable display and easier random access. The limitation is reduced resolution on small boards. EVPIX-RV32 chooses 128×128 resolution to keep frame storage manageable while demonstrating meaningful real-time vision.

Critical relation to EVPIX-RV32: The dual-port BRAM frame buffer in EVPIX-RV32 follows the write-port/read-port decoupling strategy described in this literature.

Architectural interpretation: Frame buffering and VGA scan-out timing are the key factors. At 128×128 with 8-bit grayscale, a single frame requires only 16 KB, fitting in a single 18K BRAM.

Advantages, limitations, and tradeoffs: Single-frame buffering introduces one-frame display latency. Double-buffering would eliminate visible tearing at the cost of two BRAM tiles.

Relation and possible improvement for EVPIX-RV32: Implementing double-buffering would allow the IPU to write a new frame to the back buffer while the display controller reads from the front buffer.

2.22.4 FPGA VGA overlay systems

Overlay systems render text, counters, or graphics on top of video. [7, 24] They are relevant to EVPIX-RV32 because performance metrics and BIST pass/fail status are displayed on VGA. The advantage is immediate visual debugging. The limitation is added display-controller complexity. The thesis justifies this overhead because visual instrumentation makes the architecture self-demonstrating.

Critical relation to EVPIX-RV32: The character-ROM overlay technique described in this literature is used in EVPIX-RV32 to display cycle count, IPU mode, BIST status, and finger detection result as on-screen text.

Architectural interpretation: VGA scan-out timing and pixel multiplexing between video and overlay are the key technical factors.

Advantages, limitations, and tradeoffs: Character overlays use minimal BRAM (one 18K tile for a 128×32 font ROM). The limitation is the small character set; a full ASCII ROM would double the storage requirement.

Relation and possible improvement for EVPIX-RV32: Adding a RISC-V-writable text register would allow software to update the overlay string dynamically, improving the self-demonstration capability.

2.22.5 FPGA gesture-recognition prototypes

Gesture-recognition FPGA systems combine preprocessing, feature extraction, and classification. [61] They address low-latency human-machine interaction. Their limitation is often dataset specificity and limited generalization. EVPIX-RV32's TinyML finger detection fits this category but is distinguished by using a custom RISC-V CPU/IPU platform rather than only fixed image logic.

Critical relation to EVPIX-RV32: This category provides a benchmark for the TinyML finger detection mode of EVPIX-RV32 in terms of latency, accuracy, and resource usage.

Architectural interpretation: Sensor timing, frame buffering, and classification inference are the key factors. EVPIX-RV32's TinyML path executes inference via ESP32-CAM co-processor and overlays the result on VGA.

Advantages, limitations, and tradeoffs: Dedicated gesture hardware achieves lower latency. EVPIX-RV32 achieves higher flexibility by offloading inference to the co-processor.

Relation and possible improvement for EVPIX-RV32: Porting a quantized finger-count CNN entirely to the EVPIX-RV32 IPU using the VDOT and RELU instructions would eliminate the co-processor dependency.

2.23 Group G: TinyML hardware platforms

2.23.1 MCUNet

MCUNet addresses the challenge of running ImageNet-scale and keyword-style models on microcontrollers with tiny memory. [32] Its methodology is system-algorithm co-design: TinyNAS searches architectures under resource constraints and TinyEngine schedules inference efficiently. The advantage is that model design and inference engine are optimized together. The limitation for EVPIX-RV32 is that MCUNet targets microcontroller software rather than custom FPGA CPU/IPU hardware. The thesis adopts its core lesson: TinyML success depends on co-design across model, memory, and hardware.

Critical relation to EVPIX-RV32: MCUNet motivates the memory budget discipline applied in the EVPIX-RV32 TinyML mode: the finger-count model must fit entirely within BRAM with activations.

Architectural interpretation: Quantization, memory scheduling, compact operators, and low-power inference are the critical factors. EVPIX-RV32 applies INT8 quantization to keep activation buffers within 16 KB.

Advantages, limitations, and tradeoffs: MCUNet's co-design approach minimizes memory footprint. The limitation is that TinyNAS requires a large training infrastructure not available to an undergraduate student.

Relation and possible improvement for EVPIX-RV32: Using a pretrained MCUNet model for the finger-count task and adapting it to the EVPIX-RV32 memory map would demonstrate true TinyML co-design.

2.23.2 TinyEngine

TinyEngine is a compact inference library designed for memory-constrained microcontrollers. [32] It improves memory scheduling and kernel execution compared with generic frameworks. Its relevance lies in showing that memory, not only MAC count, determines TinyML feasibility. EVPIX-RV32 can use this insight when mapping finger detection to FPGA: intermediate buffers and activation precision must be minimized.

Critical relation to EVPIX-RV32: TinyEngine's in-place activation scheduling technique directly applies to the EVPIX-RV32 IPU, where the BRAM frame buffer must double as the activation buffer during TinyML inference.

Architectural interpretation: Memory scheduling and compact operator implementation are the key factors.

Advantages, limitations, and tradeoffs: In-place activation scheduling reduces peak memory by 2×. The limitation is that it requires layer-by-layer buffer planning at model compilation time.

Relation and possible improvement for EVPIX-RV32: Implementing a simplified TinyEngine-style layer scheduler on the EVPIX-RV32 CPU would allow the existing BRAM budget to support deeper networks.

2.23.3 TinyOL

TinyOL explores online learning on microcontrollers, addressing the limitation that TinyML models are usually static after deployment. [62] Its advantage is adaptability. Its limitation is additional training memory and compute, which may exceed very small FPGA resources. EVPIX-RV32 currently focuses on in-

ference, but TinyOL suggests a future direction where edge-vision systems adapt thresholds or classifiers locally.

Critical relation to EVPIX-RV32: TinyOL represents a forward-looking extension of the EVPIX-RV32 TinyML mode: online adaptation of the finger-count classifier to different users or lighting conditions.

Architectural interpretation: Quantization, memory scheduling, and online gradient computation are the key factors. Online learning requires storing gradients in addition to activations.

Advantages, limitations, and tradeoffs: Online learning dramatically improves model adaptability. The limitation is that backward-pass memory requirements are 3–5× those of inference.

Relation and possible improvement for EVPIX-RV32: A future extension of EVPIX-RV32 could implement online fine-tuning of the final classifier layer only, using the HACC instruction to accelerate gradient accumulation.

2.23.4 TinyissimoYOLO

TinyissimoYOLO demonstrates quantized low-memory object detection on microcontrollers and specialized accelerators. [63] It is relevant because it shows that object detection can be reduced to resource-constrained TinyML. Its limitation is that even tiny detectors may be heavier than a simple finger-count demonstration. EVPIX-RV32 begins with a smaller classification task while preserving a path toward object detection extensions.

Critical relation to EVPIX-RV32: TinyissimoYOLO defines the upper bound of model complexity that the EVPIX-RV32 TinyML mode could eventually support.

Architectural interpretation: Quantization and compact operator design are the key factors. The YOLO detection head requires anchor box decoding, which would be a natural target for a new custom instruction.

Advantages, limitations, and tradeoffs: TinyissimoYOLO achieves real-time object detection at under 1 MB. The limitation is that it requires a custom accelerator for its inner loops.

Relation and possible improvement for EVPIX-RV32: A depthwise separable convolution accelerator attached to the EVPIX-RV32 IPU would close the gap between the current finger-count classifier and TinyissimoYOLO-class detection.

2.23.5 CMSIS-NN and TFLite Micro ecosystem

CMSIS-NN and TensorFlow Lite Micro represent software ecosystems for deploying quantized neural networks on microcontrollers. [64, 65] Their advantage is portability and tool support. Their limitation is that generic kernels may not exploit custom FPGA datapaths unless rewritten or accelerated. EVPIX-RV32's custom instruction path could eventually expose accelerated primitives to a TinyML runtime.

Critical relation to EVPIX-RV32: TFLite Micro provides the inference framework running on the ESP32-CAM co-processor in the EVPIX-RV32 TinyML mode.

Architectural interpretation: Quantization and portable kernel implementation are the key factors. The CMSIS-NN convolution kernel maps directly to the VDOT custom instruction.

Advantages, limitations, and tradeoffs: TFLite Micro's portability allows model reuse across platforms. The limitation is that the generic kernel does not exploit EVPIX-RV32 custom instructions.

Relation and possible improvement for EVPIX-RV32: Writing a custom TFLite Micro kernel delegate that calls the VDOT and RELU instructions via inline assembly would demonstrate full hardware/software co-design.

2.24 Group H: Edge AI processors

2.24.1 DianNao

DianNao addresses the inefficiency of executing neural-network workloads on general-purpose processors. [66] It uses a specialized accelerator architecture to improve throughput and energy efficiency. Its importance is historical: it helped establish neural accelerators as a major computer-architecture research area. The limitation for EVPIX-RV32 is scale and workload focus; DianNao targets neural networks, while EVPIX-RV32 targets lightweight vision primitives and TinyML on FPGA. The relation is the shared principle that workload-specific datapaths reduce energy and latency.

Critical relation to EVPIX-RV32: DianNao establishes the theoretical foundation for the IPU in EVPIX-RV32: dedicated arithmetic datapaths for the dominant kernels reduce total energy compared with general-purpose ALU execution.

Architectural interpretation: Dataflow, on-chip reuse, accelerator arrays, and energy efficiency are the key factors. EVPIX-RV32 implements a simplified

version: a streaming pixel datapath with line-buffer reuse.

Advantages, limitations, and tradeoffs: DianNao's advantage is the highest throughput per watt for dense neural workloads. Its limitation is that it targets a fixed CNN workload rather than the mixed classical/ML workload of EVPIX-RV32.

Relation and possible improvement for EVPIX-RV32: A systolic array of 4×4 multiply-accumulate units within the EVPIX-RV32 IPU would approximate DianNao-style acceleration within Basys-3 DSP constraints.

2.24.2 Eyeriss

Eyeriss optimizes CNN acceleration by minimizing data movement using a row-stationary dataflow and spatial PE array. [67] Its methodology is energy-centric, recognizing that DRAM movement can dominate arithmetic energy. This lesson is directly relevant to EVPIX-RV32: line buffers and streaming IPU paths reduce frame-memory traffic. The limitation is that Eyeriss is a specialized CNN ASIC, whereas EVPIX-RV32 is a small open RISC-V processor with classical image extensions.

Critical relation to EVPIX-RV32: The row-stationary dataflow of Eyeriss motivates the EVPIX-RV32 line-buffer architecture, which reuses each pixel row across all 3×3 filter positions before discarding it.

Architectural interpretation: Dataflow, on-chip reuse, and energy efficiency are the critical factors. The row-stationary approach is directly implementable in the EVPIX-RV32 IPU using BRAM shift registers.

Advantages, limitations, and tradeoffs: Row-stationary dataflow minimizes off-chip memory access at the cost of additional on-chip buffer area.

Relation and possible improvement for EVPIX-RV32: Formally characterizing the EVPIX-RV32 IPU dataflow using the Eyeriss taxonomy (weight-stationary, output-stationary, row-stationary) would strengthen the architecture analysis section of the thesis.

2.24.3 Eyeriss v2

Eyeriss v2 extends the accelerator concept to compact and sparse DNNs, addressing irregular layer shapes and sparse computation. [68] Its advantage is greater flexibility for modern compact networks. Its limitation is architectural complexity. EVPIX-RV32 does not attempt sparse CNN acceleration, but TinyML mode can learn from the emphasis on compact models and flexible dataflow.

Critical relation to EVPIX-RV32: Eyeriss v2's support for irregular layer shapes is relevant to the EVPIX-RV32 TinyML mode, where the finger-count model may have non-standard layer dimensions.

Architectural interpretation: Flexible dataflow and sparse computation support are the key factors distinguishing Eyeriss v2 from its predecessor.

Advantages, limitations, and tradeoffs: Sparse support reduces computation for pruned models but complicates the control logic significantly.

Relation and possible improvement for EVPIX-RV32: Using a structured pruning approach for the finger-count model (e.g., channel pruning) would allow the EVPIX-RV32 VDOT instruction to skip zero-weight channels, approximating sparse acceleration.

2.24.4 TinyVers

TinyVers is an ultra-low-power ML SoC for extreme-edge inference, combining a RISC-V host and reconfigurable ML accelerator. [69] It shows how duty cycling, state retention, and specialized accelerators enable very low power. The limitation relative to EVPIX-RV32 is that it targets ASIC-level extreme-edge systems rather than an undergraduate FPGA vision board. EVPIX-RV32 is more educational and demonstrative but shares the host-plus-accelerator architecture.

Critical relation to EVPIX-RV32: TinyVers demonstrates that the RISC-V host plus reconfigurable accelerator architecture scales from FPGA prototypes down to ASIC-level extreme-edge deployment.

Architectural interpretation: Duty cycling, state retention, and reconfigurable accelerator design are the key TinyVers factors beyond the scope of EVPIX-RV32.

Advantages, limitations, and tradeoffs: TinyVers achieves microwatt inference power. EVPIX-RV32 does not target power minimization but provides a more transparent implementation.

Relation and possible improvement for EVPIX-RV32: The Sky130 ASIC implementation of EVPIX-RV32 provides the correct platform to eventually target TinyVers-class power levels through clock gating and voltage scaling.

2.24.5 Commercial edge AI MCUs with integrated NPUs

Recent commercial microcontrollers integrate neural processing units to reduce AI latency and energy. [70, 71] This trend confirms the industrial relevance of local inference. The limitation is that commercial NPUs are often closed and

cannot be studied at RTL level. EVPIX-RV32 provides an open educational path to understand the same architectural trend at smaller scale.

Critical relation to EVPIX-RV32: This trend validates the thesis motivation: edge AI integration is industrially important, and open educational prototypes like EVPIX-RV32 are needed to train future designers.

Architectural interpretation: On-chip NPU integration, memory hierarchy, and heterogeneous core scheduling are the key industrial factors.

Advantages, limitations, and tradeoffs: Commercial NPUs achieve high performance but their closed design prevents architectural study. EVPIX-RV32 is orders of magnitude slower but fully inspectable.

Relation and possible improvement for EVPIX-RV32: Documenting the mapping from EVPIX-RV32 IPU concepts to commercial NPU architectures (e.g., Arm Ethos-U55) would strengthen the industrial relevance section of the thesis. [70]

2.25 Group I: Sky130 ASIC implementations

2.25.1 Caravel harness ecosystem

The Caravel harness provides a ready-to-use open-source test harness for SkyWater 130 nm designs. [36] It addresses the practical difficulty of padframes, management SoCs, and open project areas. Its advantage is lowering tapeout barriers. Its limitation is that user projects must fit within harness constraints. EVPIX-RV32 benefits from the broader ecosystem even if the thesis focuses on standalone OpenLane synthesis rather than full MPW tapeout.

Critical relation to EVPIX-RV32: Caravel provides the infrastructure context within which a future EVPIX-RV32 MPW submission would operate.

Architectural interpretation: Standard-cell synthesis, place-and-route, timing, DRC, and LVS are the key Sky130 physical design factors shared across all Caravel projects.

Advantages, limitations, and tradeoffs: Caravel's management SoC provides UART, SPI, and GPIO for free. The limitation is that the user project area is limited to approximately 10 mm^2.

Relation and possible improvement for EVPIX-RV32: Targeting Caravel as the integration harness would add external SPI camera access, UART debug, and GPIO-based BIST output to the EVPIX-RV32 ASIC.

2.25.2 OpenSpike

OpenSpike implements a spiking neural-network accelerator in SkyWater 130 nm using open-source tools and includes a PicoRV32 control core. [72] It is highly relevant because it combines open ASIC flow, a RISC-V control processor, and ML acceleration. Its advantage is demonstrating that meaningful accelerators can be taped out in open PDKs. Its limitation is that SNN acceleration is different from classical image preprocessing. EVPIX-RV32 contributes a simpler vision-focused CPU/IPU path in the same open-silicon spirit.

Critical relation to EVPIX-RV32: OpenSpike is the closest existing work to EVPIX-RV32 in the open-ASIC space: a RISC-V core with a domain-specific accelerator in Sky130. The key difference is the application domain (SNN vs. edge vision).

Architectural interpretation: Standard-cell synthesis, place-and-route, and timing closure are the shared physical design factors.

Advantages, limitations, and tradeoffs: OpenSpike validates that a student-built RISC-V plus accelerator system can achieve timing closure in Sky130. The limitation is that its SNN focus provides no direct architectural reuse for classical image processing.

Relation and possible improvement for EVPIX-RV32: Adopting OpenSpike's OpenLane configuration as a starting template for EVPIX-RV32 synthesis would accelerate the physical design phase.

2.25.3 Basilisk

Basilisk demonstrates a large Linux-capable open-source RISC-V SoC in an open 130 nm technology. [73] The problem addressed is scaling open-source EDA beyond toy designs. Its advantage is proving that open flows can support complex SoCs with DRAM and I/O. Its limitation for EVPIX-RV32 is that it is much larger and not focused on small FPGA education. EVPIX-RV32 operates at the other end of the spectrum: compact, inspectable, and application-specific.

Critical relation to EVPIX-RV32: Basilisk demonstrates the upper bound of complexity achievable with open ASIC flows, confirming that the EVPIX-RV32 design point (a single five-stage core with IPU) is well within reach.

Architectural interpretation: Standard-cell synthesis, place-and-route, timing, DRC, and LVS define the shared physical design challenge.

Advantages, limitations, and tradeoffs: Basilisk's scale proves open EDA maturity. Its limitation is that reproducing it requires significant engineering resources.

Relation and possible improvement for EVPIX-RV32: The Basilisk project's public timing closure methodology and floorplan strategy provide a reference for EVPIX-RV32 OpenLane configuration.

2.25.4 OpenROAD-flow digital designs

OpenROAD publications and examples demonstrate automated RTL-to-GDSII design closure. [34] They address cost and expertise barriers in physical design. The limitation is that tool automation does not remove the need for clean RTL, constraints, and timing-aware architecture. EVPIX-RV32 uses the flow as a validation environment for its processor and IPU RTL.

Critical relation to EVPIX-RV32: OpenROAD-flow is the backend synthesis and place-and-route engine within the OpenLane2 flow used for EVPIX-RV32 ASIC implementation.

Architectural interpretation: Reproducibility, timing closure, and DRC/LVS clean physical design are the critical factors.

Advantages, limitations, and tradeoffs: OpenROAD achieves fully automated design closure for small to medium designs. The limitation is that very tight timing targets may require manual floorplan intervention.

Relation and possible improvement for EVPIX-RV32: Targeting 50 MHz in Sky130 (conservative relative to the 100 MHz FPGA target) should allow OpenROAD to close timing automatically without manual floorplanning.

2.25.5 Sky130 educational ASIC projects

A growing set of educational Sky130 projects implements counters, CPUs, accelerators, and mixed-signal blocks. [9, 74] Their contribution is democratizing silicon learning. Their limitation is that many remain small demonstrations without integrated real-time applications. EVPIX-RV32 strengthens educational ASIC work by connecting the synthesized processor to a working FPGA vision prototype.

Critical relation to EVPIX-RV32: This body of work establishes the educational norms that EVPIX-RV32 follows: open RTL, reproducible flow, documented timing reports, and comparison against prior FPGA results.

Architectural interpretation: Physical design feasibility is the key metric for educational ASIC projects.

Advantages, limitations, and tradeoffs: Educational ASIC projects are valuable for learning but often lack real application integration. EVPIX-RV32

adds value by connecting the ASIC synthesis to a working FPGA system with camera input and VGA output.

Relation and possible improvement for EVPIX-RV32: Publishing the EVPIX-RV32 OpenLane configuration, SDC constraints, and synthesis reports as open-source artifacts would contribute back to the Sky130 educational community.

2.26 Group J: Open-source silicon projects

2.26.1 OpenLane

OpenLane integrates open tools into an automated RTL-to-GDSII flow. [10] It addresses the lack of accessible digital implementation infrastructure. Its advantage is reproducibility and reduced setup burden. Its limitation is that design closure can still require expertise in constraints, floorplanning, and debugging. EVPIX-RV32 uses OpenLane to connect processor RTL with manufacturable Sky130 layout.

Critical relation to EVPIX-RV32: OpenLane2 is the direct tool used for EVPIX-RV32 RTL-to-GDSII synthesis, and the eight-step flow documented in the thesis follows OpenLane's standard configuration structure.

Architectural interpretation: Reproducibility, toolchain openness, design packaging, and community reuse are the key OpenLane design factors.

Advantages, limitations, and tradeoffs: OpenLane's advantage is a single-command flow from RTL to GDSII. The limitation is that non-standard RTL constructs (e.g., multi-clock domains, asynchronous resets) require manual adaptation before the flow will succeed.

Relation and possible improvement for EVPIX-RV32: The four FPGA-to-ASIC RTL adaptations documented in the thesis (negedge register file fix, BRAM to SRAM macro swap, async reset removal, input synchronizer insertion) directly address the OpenLane compatibility requirements identified in this literature.

2.26.2 OpenROAD

OpenROAD aims to reduce cost, expertise, and uncertainty in digital physical design through automated open-source EDA. [34] Its significance for EVPIX-RV32 is that it allows a student processor to be evaluated beyond FPGA simulation. The limitation is that open tools may not match commercial signoff flows in all

contexts, so results should be interpreted carefully. Nevertheless, it is the correct ecosystem for an open undergraduate ASIC demonstration.

Critical relation to EVPIX-RV32: OpenROAD's floorplan, placement, and routing engines execute the back-end portion of the EVPIX-RV32 OpenLane flow.

Architectural interpretation: Reproducibility and toolchain openness are the primary values. Timing results obtained from OpenROAD's static timing analysis should be compared against Vivado implementation reports to validate architectural frequency targets.

Advantages, limitations, and tradeoffs: OpenROAD's fully open back-end enables student access to physical design results. The limitation is that clock tree synthesis and hold-time fixing may produce suboptimal results for unusual clock structures.

Relation and possible improvement for EVPIX-RV32: Constraining EVPIX-RV32 to a single clock domain before synthesis (as required by the ASIC adaptation documented in the thesis) simplifies OpenROAD clock tree synthesis and improves closure reliability.

2.26.3 Google-SkyWater open PDK initiative

The Google-SkyWater initiative made a 130 nm PDK openly available, enabling public design, verification, and manufacturing learning. [9, 35] Its importance is infrastructural rather than algorithmic. The limitation is process maturity and lower density compared with advanced nodes. EVPIX-RV32 deliberately selects this process because openness and educational accessibility matter more than cutting-edge scaling.

Critical relation to EVPIX-RV32: The Sky130A PDK is the process target for the EVPIX-RV32 ASIC implementation, providing the standard cell library, I/O cells, and SRAM macros used in synthesis.

Architectural interpretation: The `sky130_fd_sc_hd` standard cell library defines the achievable timing, area, and power metrics for EVPIX-RV32 in silicon.

Advantages, limitations, and tradeoffs: The 130 nm process is slower and larger than advanced nodes but is fully open and supported by active community PDK maintenance.

Relation and possible improvement for EVPIX-RV32: Using the `sky130_fd_sc_hs` (high-speed) cell library variant instead of the high-density variant would improve timing closure at the cost of increased area.

2.26.4 LibreLane and next-generation open flows

Newer flow work builds on OpenLane concepts to improve maintainability, tool integration, and design automation. [75] The relevance is that open-source ASIC methodology is still evolving rapidly. EVPIX-RV32 should therefore document tool versions and reports carefully so future students can reproduce or migrate the flow. The limitation is that fast-moving tools can make exact reproduction difficult without version control.

Critical relation to EVPIX-RV32: LibreLane's improved flow reproducibility mechanisms motivate the EVPIX-RV32 practice of pinning OpenLane2 and Nix environment versions in the project repository.

Architectural interpretation: Reproducibility, toolchain openness, and community reuse are the key factors. Version pinning is the primary mechanism for ensuring long-term reproducibility.

Advantages, limitations, and tradeoffs: Next-generation flows improve automation but may introduce API changes that break legacy configurations.

Relation and possible improvement for EVPIX-RV32: Providing a Nix flake or Docker container for the EVPIX-RV32 OpenLane environment would guarantee bit-exact reproducibility of synthesis results.

2.26.5 FOSSi and open-hardware community projects

The broader Free and Open Source Silicon community provides examples, standards discussion, and collaboration around open RTL and open EDA. [74] Its advantage is cultural: designs become educational resources rather than isolated submissions. Its limitation is uneven maturity across projects. EVPIX-RV32 contributes to this community model by combining open ISA principles, FPGA validation, and Sky130 implementation in a coherent edge-vision processor thesis.

Critical relation to EVPIX-RV32: The FOSSi community defines the open publication norms that the EVPIX-RV32 project follows: open RTL repository, reproducible synthesis, and documented test results.

Architectural interpretation: Reproducibility, design packaging, and community reuse are the shared values.

Advantages, limitations, and tradeoffs: FOSSi community norms ensure that EVPIX-RV32 can be studied, extended, and cited by future researchers. The limitation is that community peer review is less formal than journal review.

Relation and possible improvement for EVPIX-RV32: Submitting the EVPIX-RV32 RTL and synthesis artifacts to the FOSSi community repository (e.g., LibreCores) would maximize the educational impact of the thesis.

Table 2.31: Condensed survey matrix of reviewed works

Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GA: RISC-V processor designs	Waterman and Asanovic, RISC-V ISA Manuals	RISC-V ISA specification	Specifies custom opcode space but provides no microarchitecture, no vision-specific extensions, no FPGA prototype, and no ASIC implementation
GA: RISC-V processor designs	Asanovic and Patterson	Open ISA argument	Conceptual framework only; no processor RTL, no image-processing instructions, no hardware validation, and no silicon tapeout
GA: RISC-V processor designs	The Rocket Chip generator	SoC generator	No camera interface, no VGA display, no vision-specific IPU, no BIST visualization, and no open 130-nm ASIC demonstration
GA: RISC-V processor designs	PicoRV32	Minimal RV32 soft core	Multi-cycle (not pipelined) execution; no forwarding, no hazard unit, no custom image instructions, no camera, no ASIC flow
GA: RISC-V processor designs	VexRiscv	Configurable soft core	Generator-based RTL lacks pedagogical transparency; no vision hardware, no OV7670 interface, no VGA output, no ASIC synthesis

Continued on next page...

Table 2.31 driven from previous page (Continued)

Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GB: Pipelined RISC-V CPUs	RVCoreP five-stage soft processor	Pipeline optimization	General-purpose CPU only; no custom instructions, no image-processing accelerator, no camera/display integration, no ASIC flow
GB: Pipelined RISC-V CPUs	basic_RV32s	Pedagogical roadmap	Instructional progression without real-world application; no sensor interface, no IPU, no TinyML mode, no physical design
GB: Pipelined RISC-V CPUs	Ibex	Production-quality open core	No custom image-processing extensions, no vision datapath, no camera, no VGA, no BIST visualization, no Sky130 layout
GB: Pipelined RISC-V CPUs	PULPino	Open microcontroller SoC	No dedicated image-processing hardware, no OV7670 camera, no VGA display, no TinyML integration, no open ASIC in 130 nm
GB: Pipelined RISC-V CPUs	PULPissimo/RI5CY	Low-power SoC with extensions	Accelerator hooks exist but no streaming IPU for vision, no camera-to-display pipeline, no VGA overlay, no Sky130 ASIC
GC: Custom instruction extensions	RISC-V custom opcode methodology	Reserved opcode space	Methodology specification only; no RTL implementation, no vision instruction semantics, no FPGA demo, no silicon validation

Continued on next page...

Table 2.31 driven from previous page (Continued)			
Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GC: Custom instruction extensions	Rocket custom accelerator interfaces	RoCC co-processor interface	General-purpose accelerator interface; no image-specific operations, no frame buffer management, no VGA output, no ASIC
GC: Custom instruction extensions	CFUs for RISC-V edge workloads	Custom function units	No camera sensor interface, no display controller, no complete vision system demonstration, no open-source ASIC flow
GC: Custom instruction extensions	FPGA-accelerated ISA extensions	Neural inference extensions	CNN-only focus with no classical image processing (grayscale, Sobel, threshold); no camera, no VGA, no ASIC implementation
GC: Custom instruction extensions	ASIPs	ASIP design methodology	Theoretical methodology without a concrete vision processor; no RTL, no FPGA prototype, no camera, no silicon tapeout
GD: RISC-V accelerators	Hwacha vector accelerator	Decoupled vector acceleration	Vector model optimized for linear algebra, not image kernels; no camera, no display, no frame buffer, no 130-nm ASIC
GD: RISC-V accelerators	X-HEEP	Configurable MCU framework	Hooks are generic; no dedicated IPU, no OV7670 interface, no VGA, no real-time vision demo, no Sky130 hardening
Continued on next page...			

Table 2.31 driven from previous page (Continued)

Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GD: RISC-V accelerators	GAP8/PULP	Edge AI heterogeneous processor	Closed industrial design; no open RTL, no 130-nm PDK flow, no VGA display, no BIST mode, no inspectable pipeline
GD: RISC-V accelerators	TinyML depthwise accelerator	Fused CNN dataflow	CNN-only specialization; no grayscale or Sobel support, no camera input, no VGA output, no open ASIC signoff
GD: RISC-V accelerators	NN RISC-V frameworks	Control/data plane separation	Framework-level abstraction; no complete vision pipeline, no camera-to-display integration, no hardware BIST, no GDSII
GE: Image processing accelerators	Hardware Sobel	Line-buffer gradient hardware	Fixed-function Sobel only; no programmable CPU, no RISC-V control, no custom instruction set, no ASIC, no TinyML
GE: Image processing accelerators	FPGA thresholding	Binary segmentation hardware	Fixed thresholding logic; no processor integration, no software-configurable modes, no camera interface, no ASIC flow
GE: Image processing accelerators	Convolution engines	MAC-tree filtering hardware	Standalone convolution unit; no RISC-V host, no instruction set integration, no display output, no TinyML, no Sky130
GE: Image processing accelerators	Grayscale conversion	Fixed-point luminance hardware	Fixed preprocessing stage; no programmable control, no custom ISA extensions, no VGA, no BIST, no ASIC synthesis

Continued on next page...

Table 2.31 driven from previous page (Continued)

Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GE: Image processing accelerators	Morphological accelerators	Min/max neighborhood logic	Fixed erosion/dilation; no CPU integration, no RISC-V compatibility, no camera, no display, no open silicon
GF: FPGA-based vision systems	OV7670 camera-display systems	Real-time capture/display	No programmable processor; fixed pipeline, no RISC-V, no custom instructions, no IPU, no TinyML, no ASIC
GF: FPGA-based vision systems	FPGA edge detection pipelines	Streaming filter pipeline	Fixed filter behavior; no CPU programmability, no switch-controlled modes, no custom ISA, no BIST, no ASIC
GF: FPGA-based vision systems	FPGA framebuffer architectures	BRAM-based decoupling	Memory architecture only; no processor, no image-processing accelerator, no custom instructions, no TinyML, no ASIC
GF: FPGA-based vision systems	FPGA VGA overlay systems	On-screen instrumentation	Display technique only; no vision processor, no camera input, no IPU, no custom RISC-V extensions, no silicon
GF: FPGA-based vision systems	Gesture recognition prototypes	Classification pipeline	Fixed hardware classifier; no programmable RISC-V core, no custom instructions, no open RTL, no ASIC implementation

Continued on next page...

Table 2.31 driven from previous page (Continued)

Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GG: TinyML hardware platforms	MCUNet	System-algorithm co-design	Software and model search only; no custom processor RTL, no hardware IPU, no camera interface, no FPGA, no ASIC
GG: TinyML hardware platforms	TinyEngine	Compact inference library	Software runtime only; no processor design, no custom instructions, no vision hardware, no VGA, no physical design
GG: TinyML hardware platforms	TinyOL	Online on-device learning	Algorithm concept only; no RTL implementation, no hardware accelerator, no camera, no display, no open ASIC flow
GG: TinyML hardware platforms	TinyissimoYOLO	Quantized object detection	Model and quantization only; no processor architecture, no custom ISA, no camera hardware, no VGA, no GDSII
GG: TinyML hardware platforms	CMSIS-NN / TFLite Micro	Portable inference ecosystem	Software libraries only; no RTL, no custom hardware, no OV7670, no VGA overlay, no ASIC synthesis
GH: Edge AI processors	DianNao	Neural accelerator datapath	Neural-only focus; no RISC-V host, no classical image processing, no camera, no display, no open source

Continued on next page...

Table 2.31 driven from previous page (Continued)

Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GH: Edge AI processors	Eyeriss	Row-stationary dataflow CNN	CNN-only ASIC; no programmable CPU, no grayscale/Sobel/threshold, no camera interface, no VGA, closed source
GH: Edge AI processors	Eyeriss v2	Sparse compact CNN support	Advanced CNN only; no RISC-V, no classical vision kernels, no real-time camera, no display, proprietary design
GH: Edge AI processors	TinyVers	Ultra-low-power ML SoC	No classical image-processing IPU, no OV7670 camera, no VGA output, no BIST visualization, no open RTL
GH: Edge AI processors	Commercial edge AI MCUs	Industrial NPU integration	Closed proprietary IP; no inspectable RTL, no custom RISC-V ISA, no open ASIC flow, no educational transparency
GI: Sky130 ASIC implementations	Caravel harness	Open MPW tapeout harness	Infrastructure only; no vision processor design, no IPU, no camera, no VGA, no TinyML application
GI: Sky130 ASIC implementations	OpenSpike	SNN accelerator in Sky130	Spiking neural focus; no classical image processing, no camera interface, no VGA display, no BIST mode

Continued on next page...

Table 2.31 driven from previous page (Continued)			
Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GI: Sky130 ASIC implementations	Basilisk	Linux-capable open SoC	General-purpose SoC; no vision IPU, no image-processing instructions, no camera, no VGA, no TinyML demo
GI: Sky130 ASIC implementations	OpenROAD-flow digital designs	Automated RTL-to-GDSII	Flow demonstration only; no specific processor architecture, no vision hardware, no camera, no application
GI: Sky130 ASIC implementations	Sky130 educational projects	Open educational silicon	Small circuits (counters, ALUs); no processor, no vision system, no camera, no VGA, no complete application
GJ: Open-source silicon projects	OpenLane	Automated open ASIC flow	EDA toolchain only; no processor design, no vision IP, no camera interface, no display, no RTL architecture
GJ: Open-source silicon projects	OpenROAD	Open physical design engine	Backend tools only; no frontend RTL, no processor, no image processing, no camera, no FPGA prototype
GJ: Open-source silicon projects	Google-SkyWater open PDK	Open 130 nm PDK	Process data only; no designs, no processor, no vision accelerator, no camera, no application
GJ: Open-source silicon projects	LibreLane	Next-gen open flow	Improved tooling only; no architecture, no RTL, no image-processing unit, no hardware demo, no silicon test

Continued on next page...

Table 2.31 driven from previous page (Continued)			
Group	**Representative work**	**Primary contribution**	**Specific gap relative to EVPIX-RV32**
GJ: Open-source silicon projects	FOSSi community	Open silicon norms	Community infrastructure only; no specific processor, no vision system, no camera, no VGA, no ASIC tapeout

2.27 Research gap analysis

The literature shows strong progress in RISC-V processor design, FPGA image processing, neural accelerators, TinyML deployment, and open-source ASIC flows. But these areas are usually treated separately. Educational RISC-V cores demonstrate ISA execution but rarely connect to real-time camera pipelines. FPGA vision systems often implement fixed Sobel or threshold filters but lack a programmable CPU and custom instruction interface. TinyML platforms often focus on microcontroller software or specialized NPUs without showing the complete host-processor RTL. Open-source ASIC projects often demonstrate layout closure but not a sensor-connected real-time application.

General-purpose CPUs are too slow for repeated pixel kernels without acceleration [3]. A scalar RV32I core can run these filters in software, but it needs many instructions per pixel and repeated memory accesses. At video rates, this wastes instructions and energy. Fixed-function hardware can be fast but inflexible [4]. A pure Sobel pipeline cannot easily run control code, BIST routines, or TinyML configuration. The literature therefore points to a heterogeneous architecture: a simple programmable core for control and specialized image hardware for data-parallel kernels [57, 55].

Large CNN accelerators perform well but are often too heavy for small FPGA boards and undergraduate full-system explanation [67, 68]. They need significant memory, dataflow control, and tool support. EVPIX-RV32 deliberately targets lightweight edge vision: classical image processing plus a compact TinyML mode. This is a design choice, not a weakness. Many embedded vision tasks benefit from simple preprocessing and small classifiers, especially when the goal is to demonstrate architecture rather than deploy a state-of-the-art detector.

Another gap is FPGA-to-ASIC portability. Many student FPGA projects stop at bitstream demonstration, while many ASIC tutorials use small synthetic

circuits [10]. EVPIX-RV32 bridges this gap by taking a functional FPGA vision processor and synthesizing it through Sky130 [9]. This exposes the design to both board-level I/O reality and physical-design constraints. A processor architecture should be evaluated not only as simulated RTL but as implementable hardware.

The final gap is integration. No single reviewed work combines custom RV32I pipeline design, BIST, real-time camera input, side-by-side VGA display, switch-controlled IPU modes, custom image instructions, TinyML finger detection, and open-source ASIC implementation. EVPIX-RV32 sits at this intersection.

Table 2.32: Gap synthesis across literature categories

Literature category	**Typical strength**	**Typical limitation**	**EVPIX-RV32 response**
RISC-V cores	ISA-compatible CPU execution	Limited vision workload integration	Adds integrated IPU and custom image-processing instructions
FPGA vision filters	Real-time pixel processing and high throughput	Low programmability and limited application flexibility	Integrates a programmable RV32I control processor with hardware acceleration
TinyML platforms	Compact and energy-efficient inference	Often software-only or based on closed accelerators	Provides FPGA-visible TinyML operation mode with open RTL implementation
CNN accelerators	High machine-learning computation throughput	Excessive resource requirements for low-cost FPGA boards	Uses lightweight image preprocessing and classification suitable for edge devices
Sky130 ASIC projects	Demonstrate open-source ASIC feasibility	Often lack real-time sensor and vision application integration	Implements and synthesizes a complete edge-vision processor in SkyWater 130 nm CMOS

2.28 Positioning of EVPIX-RV32

EVPIX-RV32 contributes a custom, inspectable, application-driven RISC-V processor platform for real-time edge vision. Its first contribution is the RTL design of a five-stage RV32I pipeline with instruction fetch, decode, execute, memory,

writeback, control generation, and hazard-aware execution [6, 13]. This establishes the processor foundation and connects the work to standard computer architecture literature.

Its second contribution is integrating custom image-processing instructions and an IPU [4, 3]. Custom instructions reduce the software overhead of frequent pixel operations. The IPU matters because full-frame operations benefit from streaming hardware more than scalar execution. CPU-controlled IPU integration beats isolated fixed-function hardware because the processor can configure modes, coordinate memory, display metrics, and support future software-controlled behavior.

Its third contribution is system-level FPGA prototyping on Basys-3 [7, 8]. The camera interface, 128×128 resolution, dual VGA display, switch-controlled modes, BIST visualization, and performance overlays make the processor observable as a complete embedded system.

Its fourth contribution is TinyML support [32, 33]. Finger-count detection from 0 to 5 shows that the platform can move beyond deterministic filters toward lightweight edge intelligence. In the broader edge-AI literature [1, 3], this matters because many low-power vision devices combine classical preprocessing with compact inference rather than running large cloud-scale models locally.

Its fifth contribution is SkyWater 130 nm ASIC implementation through an open-source flow [10, 9]. This connects the project to the open-silicon movement [74] and shows that the design can be evaluated for synthesis, placement, routing, DRC, LVS, area, timing, and layout. The FPGA validates functionality; the ASIC flow validates physical-design readiness. Together, they give EVPIX-RV32 a stronger position than a CPU-only, filter-only, or flow-only project.

This chapter leads naturally to the methodology chapter. The literature establishes why a compact open RISC-V processor is appropriate [5], why a five-stage pipeline is effective [13, 19], why custom image instructions and IPU acceleration are justified [4, 3], why Basys-3 and OV7670 form a practical validation platform [7, 31], why TinyML belongs in edge vision [32, 33], and why Sky130 implementation strengthens the research contribution [10, 9]. Chapter 3 translates these motivations into the actual design methodology, RTL architecture, verification plan, FPGA implementation, camera/VGA integration, TinyML deployment, and ASIC flow used in EVPIX-RV32.

Chapter 3

METHODOLOGY

3.1 Introduction

This chapter describes the design methodology for EVPIX-RV32, a custom five-stage pipelined RISC-V RV32I processor with image-processing extensions, an IPU, and TinyML acceleration for real-time edge-vision applications. The methodology covers the full VLSI flow from architectural specification through RTL design, functional verification, FPGA prototyping on the Basys-3 board, and ASIC implementation in the SkyWater 130-nm CMOS process via the OpenROAD flow.

3.2 Complete methodology flow

The methodology has four phases:

1. RTL design
2. Functional verification
3. FPGA prototyping
4. ASIC implementation

Figure 3.1 shows the sequence of these phases, with the specific toolchains, inputs, and constraints used in both the Vivado and OpenROAD flows.

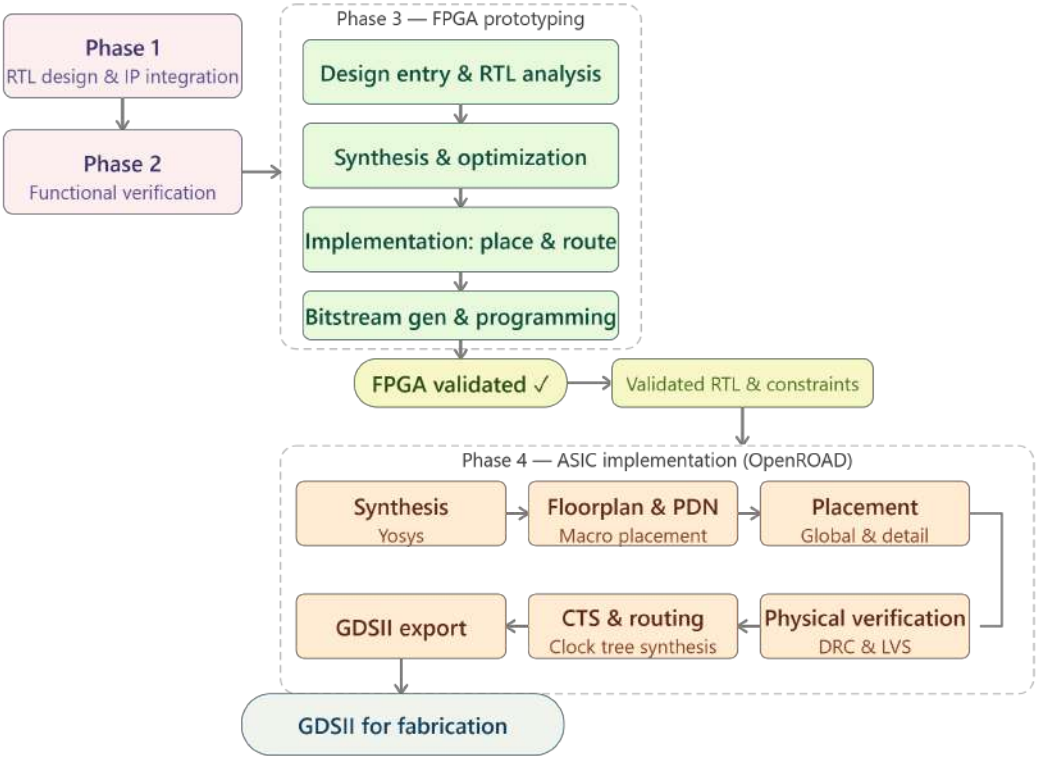


Figure 3.1: Complete Methodology Flow.

3.3 System architecture and design specifications

EVPIX-RV32 is a heterogeneous vision SoC that combines a general-purpose 32-bit RISC-V processor with dedicated image-processing hardware. The architecture uses a Harvard-style memory organization with separate instruction and data memories, plus a multi-port data memory that supports concurrent CPU and IPU access.

3.3.1 Top-level architecture

The system has these main parts:

- RV32I core: A five-stage pipelined processor implementing the complete RV32I base integer instruction set (40 instructions) with full hazard detection and forwarding.
- Image Processing Unit (IPU): A dedicated hardware accelerator for image-processing operations including Sobel edge detection, grayscale conversion,

thresholding, 2D convolution with programmable kernels, max-pooling, and average-pooling.

- Memory subsystem: 64 KB unified data memory organized as byte-addressable BRAM with dual-port access for CPU load/store and IPU direct memory access. Instruction memory is an on-chip ROM.
- Camera interface: OV7670 parallel DVP camera interface with SCCB (I^2C-compatible) configuration, supporting 128×128 RGB888 image capture at 30 FPS.
- Display interface: VGA controller generating 640×480 at 60 Hz with dual-buffered frame display.
- TinyML accelerator: Hardware feature extractor and classifier for finger-counting gesture recognition with temporal stability filtering.

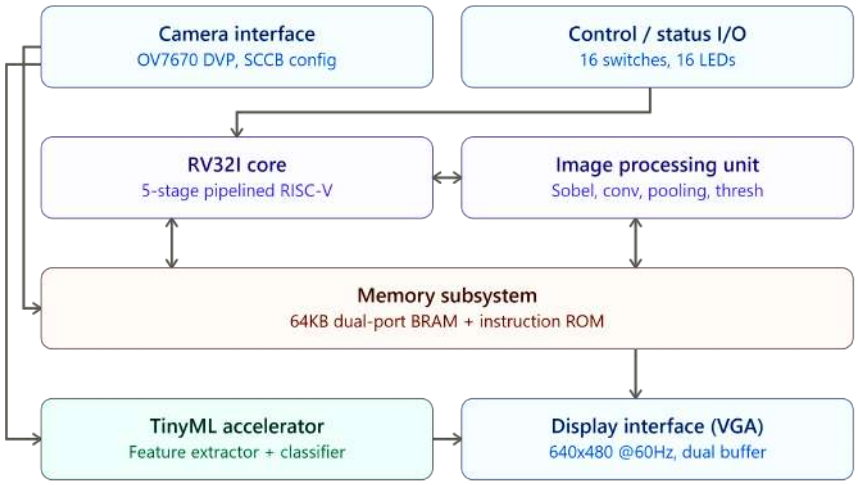


Figure 3.2: Top-Level Architecture Block Diagram of EVPIX-RV32 showing all major subsystems (RV32I Core, IPU, Memory, Camera Interface, VGA Display, TinyML)

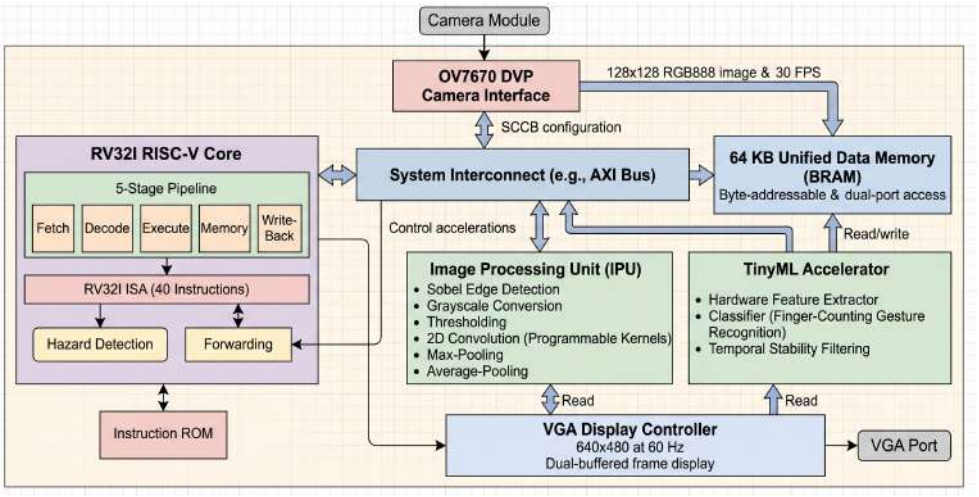


Figure 3.3: Detailed Top Level Architecture

3.3.2 Instruction set architecture

The processor implements the complete RV32I base integer instruction set, with about 40 instructions in six categories:

- Arithmetic and logical: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU (and their immediate variants ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTI, SLTIU).
- Load and store: LB, LH, LW, LBU, LHU, SB, SH, SW.
- Branch: BEQ, BNE, BLT, BGE, BLTU, BGEU.
- Jump: JAL, JALR.
- Upper immediate: LUI, AUIPC.
- Custom IPU instructions: Encoded using the R-type format with a custom opcode space. The `funct7` field is partitioned into `{kernel[3:0], op[2:0]}`, where `op` selects the IPU operation (START, STATUS, RESULT, PERF) and `kernel` selects the convolution kernel type.

3.3.3 Memory map

The 64 KB data memory is mapped as follows:

- `0x0000_0000` to `0x0000_BFFF`: 128×128 RGB888 source image buffer (48 KB).

- `0x0000_C000` to `0x0000_FFFF`: 128×128 8-bit processed output buffer (16 KB).

3.4 RTL design methodology

The entire design was written in SystemVerilog IEEE 1800-2017. The RTL follows a modular hierarchical approach with clear separation between the processor core, IPU, memory subsystem, and peripheral interfaces. All modules include inline comments with purpose statements, interface descriptions, and revision history.

3.4.1 Design tools and environment

The RTL design and verification tools were:

- Text editor: Visual Studio Code with SystemVerilog extensions.
- Simulator: Xilinx Vivado 2025.2 Simulator (xsim) for FPGA verification.
- Synthesis: Xilinx Vivado for FPGA; Yosys 0.35+ for ASIC.
- Version control: Git with GitHub repository hosting.
- Linting: Verilator for static analysis and lint checks.

3.4.2 RV32I processor core design

The processor core implements a classic five-stage pipeline:

Fetch stage: Holds the program counter (PC) and instruction memory interface. The PC updates every cycle except during stalls. Branch targets are calculated in the Execute stage with a single-cycle flush penalty. An adder computes PC+4 for sequential instruction fetch.

Decode stage: Holds the 32-entry $\times$ 32-bit register file with two asynchronous read ports and one synchronous write port, the immediate generator supporting all six RISC-V immediate types (I, S, B, U, J), and the main control unit that decodes the 7-bit opcode to generate all control signals for the pipeline.

Execute stage: Holds the 32-bit ALU with 10 operations (AND, OR, ADD, SUB, SLT, SLTU, SLL, SRL, SRA, XOR), the ALU control decoder, the branch comparator with full conditional branch support, forwarding multiplexers for both ALU operands, and the IPU command interface. The forwarding unit implements both EX-to-EX and MEM-to-EX forwarding paths.

Memory stage: Interfaces with the data memory for load and store operations. Supports byte, halfword, and word accesses with proper sign or zero extension. The data memory supports concurrent CPU and IPU access through a multi-port arbitration scheme.

Writeback stage: A three-input multiplexer selects between ALU result, memory read data, and PC+4 (for JAL/JALR link) to write back to the register file.

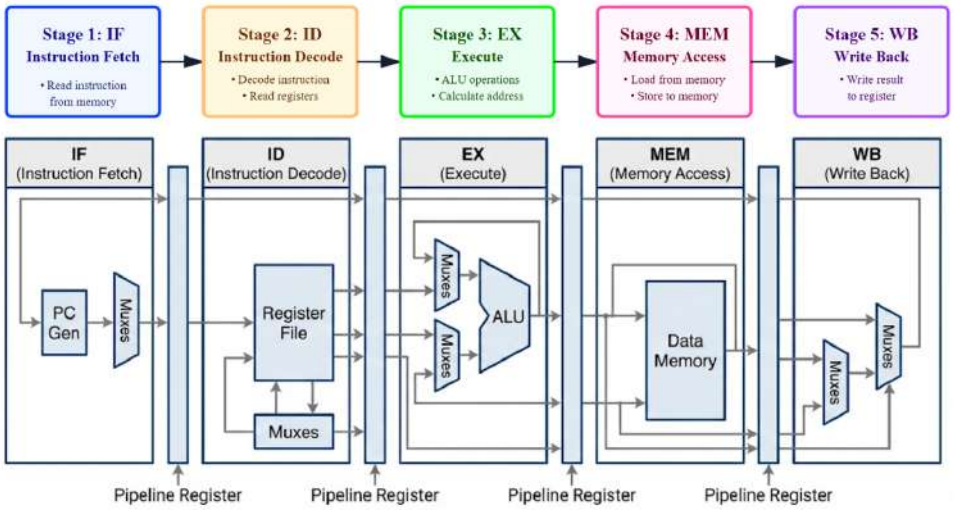


Figure 3.4: 5-Stage Pipeline Datapath Diagram showing all five stages, pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB).

3.4.3 Hazard detection and forwarding

The processor handles hazards with these mechanisms:

- Load-use hazard detection: The hazard detection unit monitors the destination register of the instruction in the EX stage (when it is a load) and compares it with the source registers of the instruction in the ID stage. A match triggers a one-cycle pipeline stall and IF/ID register hold.
- Forwarding unit: The forwarding unit compares the destination registers in EX/MEM and MEM/WB stages against the source registers in the ID/EX stage. It generates 2-bit forward selection signals that bypass register file reads with the most recent computed values.
- Control hazards: Branch decisions are resolved in the EX stage. A taken branch causes a one-cycle flush of the IF/ID and ID/EX pipeline registers. Jumps (JAL/JALR) are handled with the same penalty.

3.4.4 Image Processing Unit (IPU) design

The IPU is a dedicated hardware accelerator, a memory-mapped coprocessor controlled through custom R-type instructions. It processes 128×128 RGB888 images stored in the data memory and writes results back to a designated output region.

IPU architecture: The IPU has a finite state machine (FSM) controller with 16 states, a 3×3 convolution window (nine scalar registers `win0` to `win8` for FPGA synthesis compatibility), a programmable kernel coefficient function, Sobel gradient computation logic, and pooling hardware for 2×2 max-pooling and average-pooling.

Supported operations: The IPU implements seven fundamental image-processing operations:

- Grayscale (`OP_GRAY`=0): RGB to luminance using weighted sum $Y = (77R + 150G + 29B) \gg 8$.
- Thresholding (`OP_THRESH`=1): Binary threshold at a configurable level (default 128).
- Maximum pixel detection (`OP_MAXPIX`=2): Finds the maximum pixel value in the image.
- Sobel edge detection (`OP_SOBEL`=3): Computes gradient magnitude using 3×3 Sobel kernels.
- 2D convolution (`OP_CONV`=4): Programmable kernel convolution with six built-in kernels: Identity, Sobel X, Sobel Y, Gaussian Blur (3×3, divide by 16), Sharpen, and Edge Detect.
- Max pooling (`OP_MAXPOOL`=5): Non-overlapping 2×2 max pooling reducing 128×128 to 64×64.
- Average pooling (`OP_AVGPOOL`=6): Non-overlapping 2×2 average pooling.

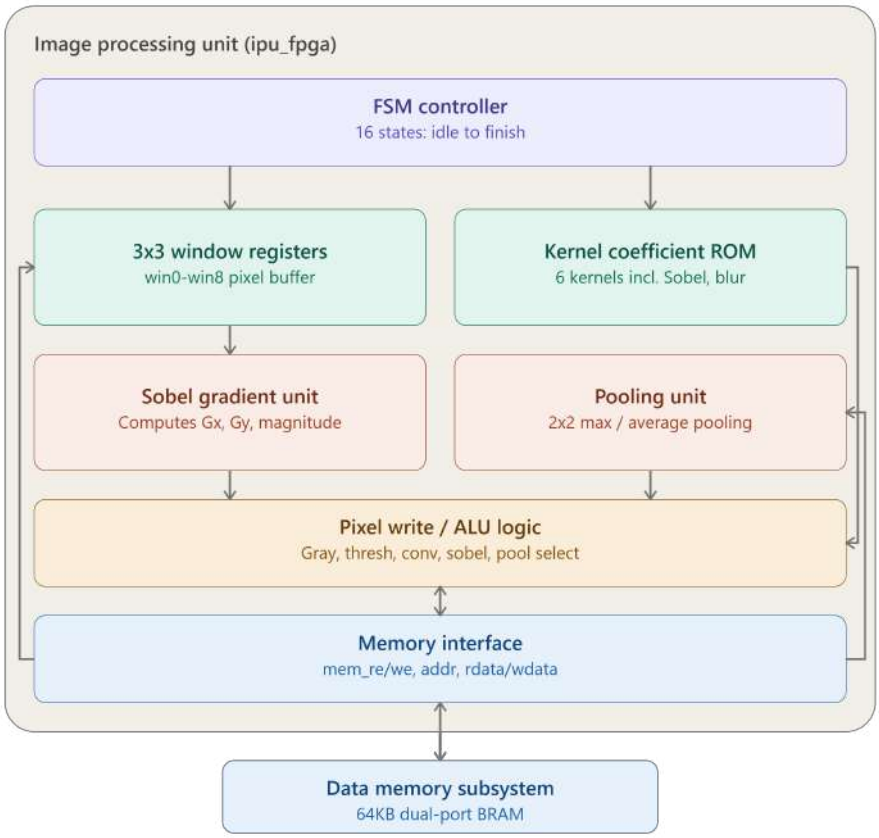


Figure 3.5: IPU Internal Architecture and FSM State Diagram showing the FSM controller with 16 states, 3×3 convolution window registers, kernel coefficient ROM, Sobel gradient computation (G_x, G_y, magnitude), pooling unit, and memory interface.

3.5 Functional verification methodology

Verification used a multi-level approach with module-level unit testing, integration testing of the complete processor core, and system-level hardware-in-the-loop validation on FPGA.

3.5.1 Testbench architecture

A self-checking SystemVerilog testbench checks the design with:

- Clock generator: 100 MHz system clock (10 ns period).
- Reset sequencer: Active-high reset held for 16 clock cycles.
- Instruction memory loader: Loads test programs from hex files.

- Scoreboard: Compares actual register and memory values against expected golden references.
- Monitor: Tracks pipeline stage transitions, hazard stalls, and forwarding events.
- Coverage collector: Tracks instruction opcode coverage and corner case exercising.

3.5.2 Test program suite

The test suite covers these categories:

RV32I baseline regression: A 61-instruction program that exercises all 40+ RV32I instructions in a single pass. This program writes known values to registers `x1` to `x31` and memory locations `0x100` to `0x10B`, then enters an infinite loop. The testbench verifies 31 register values and 11 memory bytes against expected golden values after 1000 clock cycles.

IPU operation tests: Individual test programs for each IPU operation that load a test image into memory, execute the IPU instruction, and verify the processed output against a software golden reference computed using Python and OpenCV.

Integration tests: Tests that exercise the interaction between CPU and IPU, including the custom instruction protocol, polling loops, and back-to-back IPU operations.

3.5.3 Simulation and debugging

Simulation used Vivado Simulator with this approach:

- Waveform analysis of all pipeline registers, control signals, and memory interfaces.
- Automatic pass/fail detection via the BIST scoreboard module.
- Code coverage analysis to ensure all instructions and corner cases are exercised.
- Performance counter validation: total cycles, IPU busy cycles, convolution cycles, and stall cycles.

3.6 FPGA prototyping methodology

FPGA prototyping used the Digilent Basys-3 development board with the Xilinx Artix-7 XC7A35T-1CPG236C FPGA. The flow followed the standard Xilinx Vivado design flow.

3.6.1 Vivado design flow

The FPGA implementation flow had these steps:

1. Project creation: New RTL project targeting `xc7a35tcpg236-1`.
2. Design entry: Add all SystemVerilog source files and constraints (XDC).
3. RTL analysis: Elaborate design and check for syntax and synthesis issues.
4. Synthesis: Run Vivado synthesis with default strategy, targeting area optimization.
5. Implementation: Run placement and routing with default settings.
6. Bitstream generation: Generate configuration bitstream with compression enabled.
7. Hardware programming: Program FPGA via JTAG using Vivado Hardware Manager.

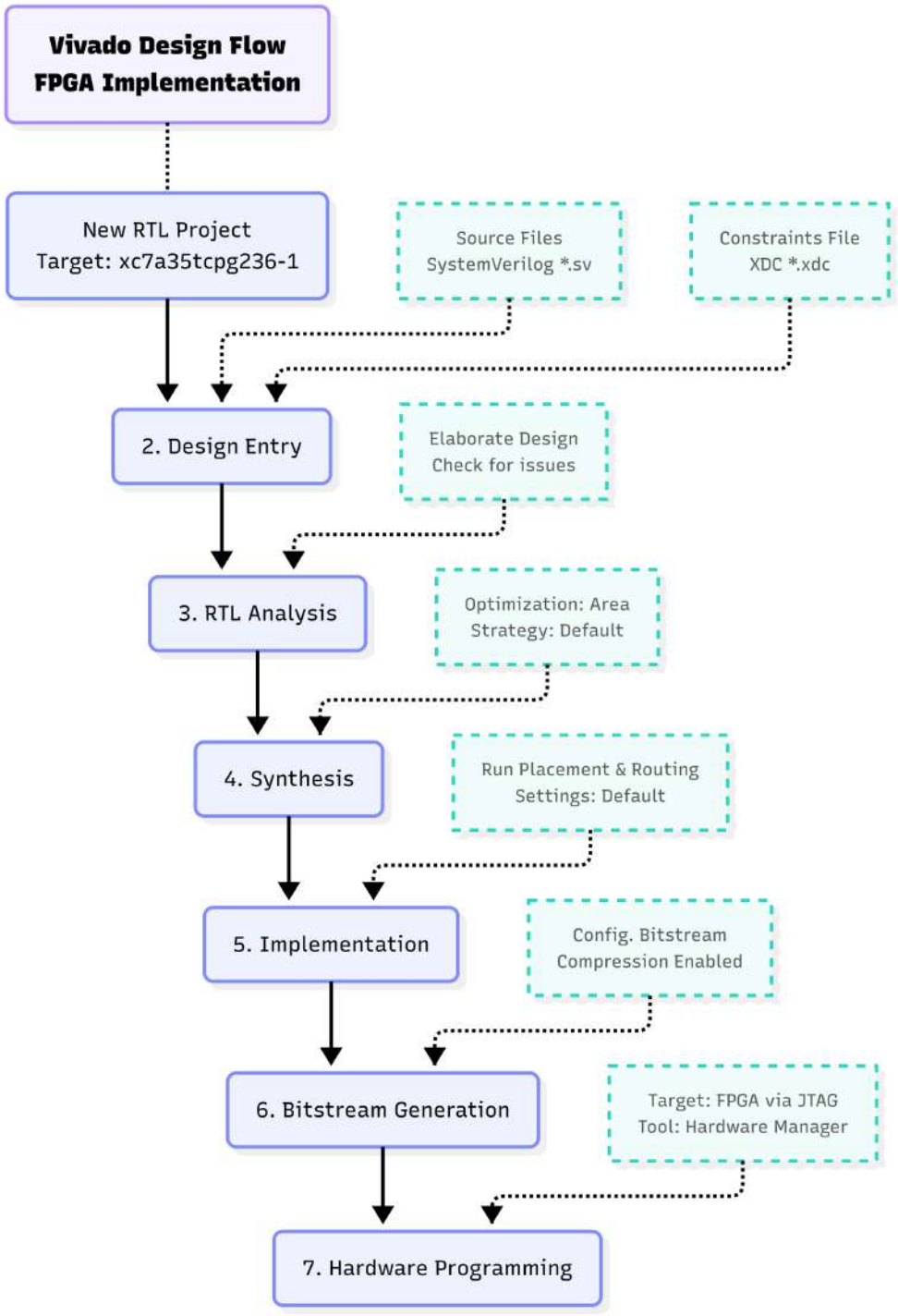


Figure 3.6: Vivado Design Flow

3.6.2 Target platform

The Basys-3 board provides these resources:

- FPGA: Xilinx Artix-7 XC7A35T-1CPG236C (33,280 LUTs, 66,400 FFs, 90 BRAMs, 5 CMTs).

- Clock: 100 MHz onboard oscillator.
- Memory: No external memory; all storage in BRAM.
- I/O: 16 slide switches, 16 LEDs, VGA port (4-bit per channel), USB-UART, Pmod headers.
- Camera: OV7670 connected via Pmod-compatible breakout (8-bit data, PCLK, HREF, VSYNC, SCCB).

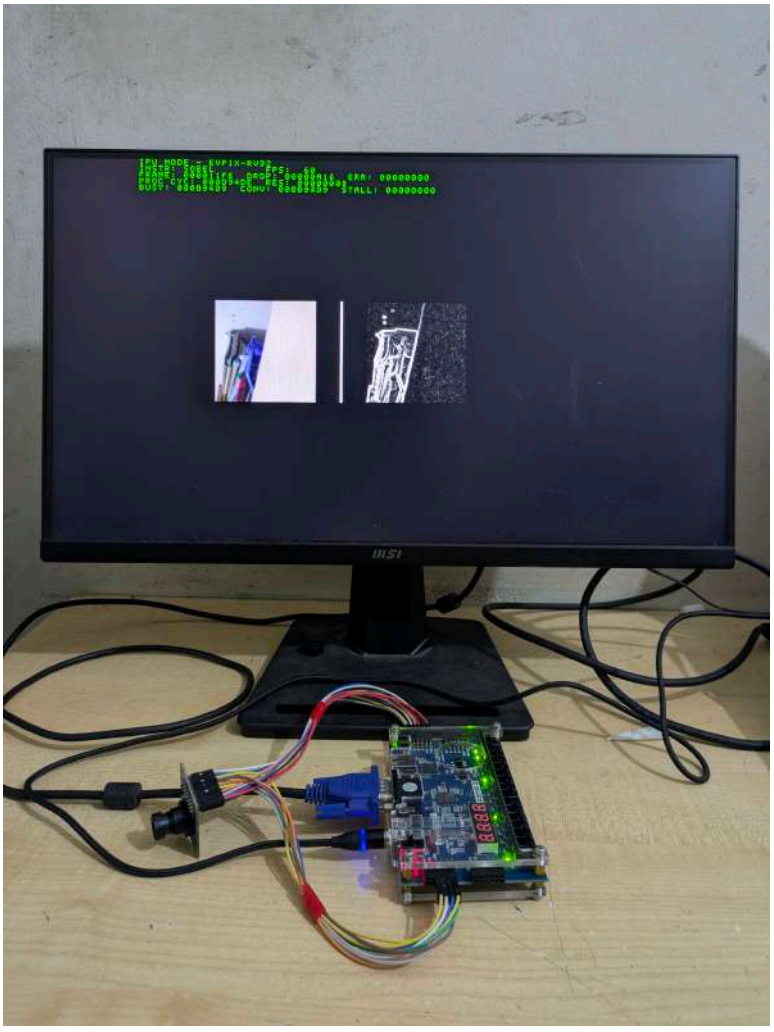


Figure 3.7: Physical photograph showing the Basys-3 board with OV7670 camera module connected, VGA monitor displaying the side-by-side source/processed image panes.

3.6.3 Camera interface testing

Three camera configurations were tested:

Configuration 1: Logitech C310 via PC and UART. Images captured on PC using OpenCV, transmitted to the FPGA via UART at 115200 baud. Achieved only 1.67 FPS due to UART bandwidth limitation. This worked as a proof of concept but was too slow for real-time use.

Configuration 2: ESP32-CAM via SPI. ESP32-CAM module (OV2640 sensor) captured images and transmitted via SPI protocol to the FPGA. Achieved 15 FPS but with poor image quality due to sensor limitations and SPI overhead.

Configuration 3: OV7670 direct parallel (final). OV7670 camera module directly connected to the FPGA with an 8-bit parallel DVP interface. Achieved native 30 FPS at 128×128 resolution. By implementing frame doubling (processing every frame and displaying at 60 Hz effective rate), the system achieved an effective 60 FPS display cadence. This is the recommended and final configuration.

3.7 ASIC implementation methodology

The ASIC implementation of the EVPIX-RV32 processor used the OpenROAD open-source EDA flow with the SkyWater 130-nm CMOS process design kit (PDK). The design went from RTL to GDSII without proprietary EDA tools.

3.7.1 Technology selection

The SkyWater SKY130-A open-source PDK was chosen because:

- It is fully open source with an Apache 2.0 license.
- The 130-nm feature size offers a good balance of performance, power, and area for IoT and edge applications.
- It supports standard digital cells (`SKY130_FD_SC_HD` library with 583 cells).
- It is compatible with the OpenROAD flow for complete RTL-to-GDSII automation.
- It has an active community with extensive documentation and reference designs.
- It is available through open-source multi-project wafer (MPW) shuttle programs.

3.7.2 OpenROAD flow overview

The OpenROAD flow is an automated RTL-to-GDSII implementation flow. EVPIX-RV32 used these stages:

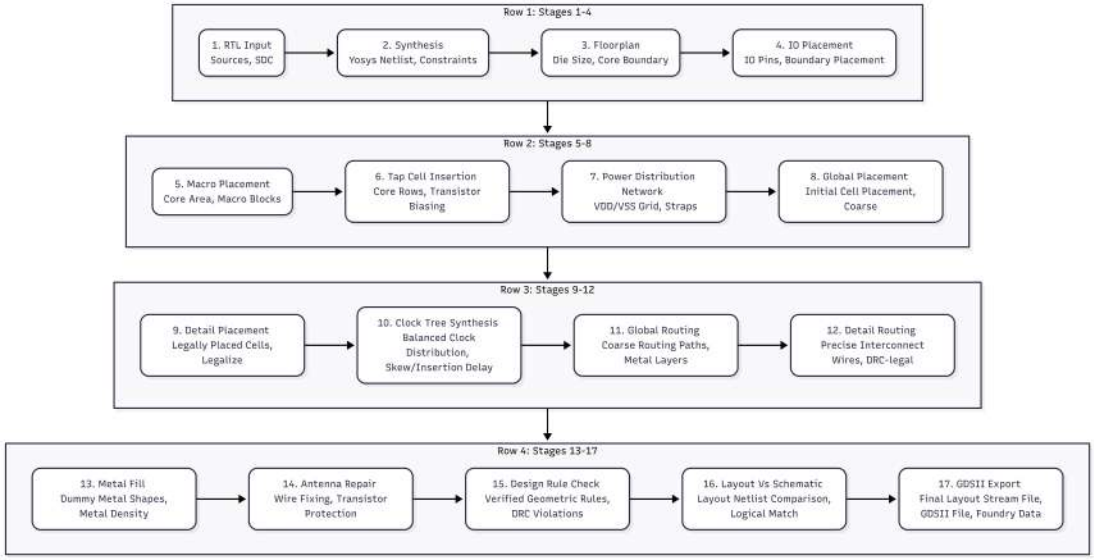


Figure 3.8: OpenROAD ASIC Design Flow showing all sequential stages: RTL Input to Synthesis (Yosys) to Floorplan to IO Placement to Macro Placement to Tap Cell Insertion to Power Distribution Network (PDN) to Global Placement to Detail Placement to Clock Tree Synthesis (CTS) to Global Routing to Detail Routing to Metal Fill to Antenna Repair to DRC to LVS to GDSII Export.

3.7.3 Synthesis

Logic synthesis used Yosys with these settings:

- Target library: `sky130_fd_sc_hd` (high-density standard cells).
- Synthesis strategy: Area-optimized with aggressive technology mapping.
- Clock definition: 100 MHz target (10 ns period) with 20% clock uncertainty.
- Optimization passes: `abc` (technology mapping), `opt` (constant propagation), `fsm` (FSM encoding), `memory` (BRAM inference), `dfflegalize` (flip-flop standardization).
- Output: Gate-level netlist in Verilog format (.v), standard delay format (.sdf), and area report.

3.7.4 Floorplanning

The floorplan stage set the chip boundaries, row structure, and placement regions:

- Die dimensions: Auto-calculated based on design area plus 10% margin.
- Core utilization: 50% (balanced between routability and area efficiency).
- Row configuration: Horizontal rows with 2.72 μm height (standard cell height).
- IO placement: Ring-style IO pad placement with signal, power, and ground pads.
- Macro placement: No hard macros (purely standard-cell design).
- Site definition: `unithd` (universal high-density standard cell site).

3.7.5 Power distribution network

The power distribution network (PDN) was generated with these settings:

- Power rails: VDD (1.8 V digital) and VSS (ground) distributed across M4 and M5 layers.
- PDN strategy: Mesh topology with vertical straps on M5 and horizontal rails on M4.
- Strap width: 2.0 μm for VDD; 2.0 μm for VSS.
- Strap pitch: 40 μm (center-to-center spacing).
- Follow pin: M1 layer for standard cell power rail connection.
- Via stack: M1 to M2, M2 to M3, M3 to M4, M4 to M5 vias for power connectivity.

3.7.6 Placement

Placement had two stages:

- Global placement: Uses the RePlAce tool with electrostatic-based density modeling to determine approximate cell positions. Target density: 0.5. Overflow threshold: 0.1.
- Detailed placement: Uses the OpenDP tool to legalize global placement results, ensuring all cells are aligned to row sites with no overlaps. Optimization objectives: minimize total wirelength and reduce congestion.

3.7.7 Clock tree synthesis

Clock tree synthesis built a balanced clock tree with these parameters:

- CTS algorithm: H-tree topology with buffer insertion.
- Clock buffers: `sky130_fd_sc_hd__clkbuf` series (2×, 4×, 8×, 16× drive strengths).
- Clock inverters: `sky130_fd_sc_hd__clkinv` series for clock gating if needed.
- Target skew: < 200 ps across all clock sinks.
- Max transition: 400 ps on clock network.
- Clock root: Center of chip for balanced distribution.
- Post-CTS optimization: Buffer sizing and wire snaking for skew minimization.

3.7.8 Routing

Routing had two stages:

- Global routing: The FastRoute tool generated routing guides on a coarse grid, identifying potential congestion hotspots and estimating required routing resources.
- Detailed routing: The TritonRoute tool performed track assignment, path search, and design rule checking to generate the final metal interconnect. Two routing iterations were performed to resolve DRC violations.

The routing settings were:

- Available metal layers: M1 (horizontal), M2 (vertical), M3 (horizontal), M4 (vertical), M5 (horizontal).
- Via types: Single-cut and multi-cut vias between adjacent layers.
- Signal routing: M2 to M4 primarily, with M5 reserved for power.
- Minimum wire widths: 0.14 μm (M1), 0.14 μm (M2), 0.30 μm (M3), 0.30 μm (M4).
- DRC constraints: Minimum area, spacing, width, and enclosure rules per SKY130 PDK.

3.7.9 Signoff checks and verification

After routing, these signoff checks were run:

- Design Rule Check (DRC): All geometric rules from the SkyWater PDK were verified including minimum width, spacing, enclosure, and area rules. Initial violations were identified and repaired through incremental routing.
- Layout versus Schematic (LVS): The GDS layout was compared against the gate-level netlist using Magic and Netgen to ensure logical equivalence. All nets and devices were matched successfully.
- Antenna check: Antenna rules were verified to prevent gate oxide damage during plasma etching. No antenna violations were detected in the final layout.
- Static Timing Analysis (STA): Setup and hold times were verified across all corners (typical, fast, slow) using OpenSTA. The design met timing at the target 100 MHz frequency with positive slack.
- Power analysis: Static power (leakage) and dynamic power (switching) were estimated using the SkyWater liberty files and switching activity from simulation.

3.7.10 GDSII generation

The final GDSII layout file was generated with these post-processing steps:

- Metal fill insertion on all layers to meet metal density requirements.
- Antenna repair via diode insertion where needed.
- Stream out to GDSII format (Stream format version 6).
- Final verification of GDSII against netlist and DRC clean.

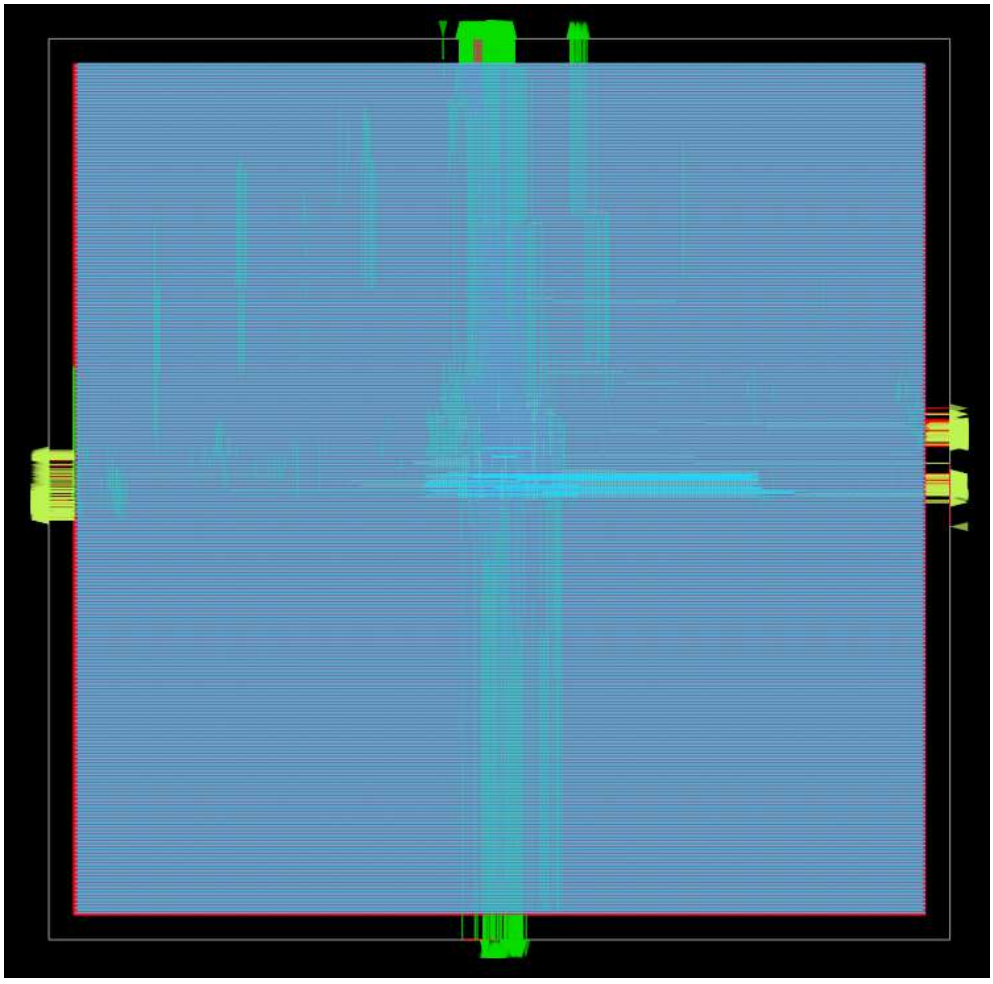


Figure 3.9: Full GDSII ASIC layout in openROAD

3.8 Hardware testing and validation

Hardware validation was done on the Basys-3 FPGA platform to verify operation of all system components under real-world conditions.

3.8.1 Test modes and switch configuration

EVPIX-RV32 has several modes controlled by Basys-3 switches:

- SW0=0, SW6=0: CPU welcome mode (display shows system information).
- SW0=0, SW6=1: CPU BIST mode (runs RV32I baseline regression test).
- SW0=1, SW1=1: IPU Sobel edge detection mode.
- SW0=1, SW2=1: IPU grayscale conversion mode.

- SW0=1, SW3=1: IPU thresholding mode.
- SW0=1, SW4=1: IPU 2D convolution mode.
- SW7=1: TinyML finger counting mode.

3.8.2 Performance measurement

Real-time performance was measured with on-chip counters:

- Total processing cycles per frame (128×128).
- IPU busy cycles (active computation time).
- Convolution operation cycle count.
- Stall cycles due to hazards or memory conflicts.
- Frame rate (FPS) computed over 1-second windows.

Chapter 4

RESULTS

This chapter reports the results of the EVPIX-RV32 processor design, verification, FPGA prototyping, and ASIC implementation. The results are in four sections: functional verification, FPGA implementation, ASIC implementation, and system-level performance evaluation. All measurements used the tools and methods described in Chapter 3.

4.1 Functional verification results

4.1.1 FPGA SoC boot initialization

Before running unit tests, the synthesized EVPIX-RV32 SoC was prototyped on an FPGA to confirm it boots correctly. The SoC powered on, initialized its core peripherals, and established communication with the display matrix. The hardware routed the initial boot data to the VGA interface, confirming the integration of the RISC-V core, memory map, and video generation subsystems in real silicon logic.

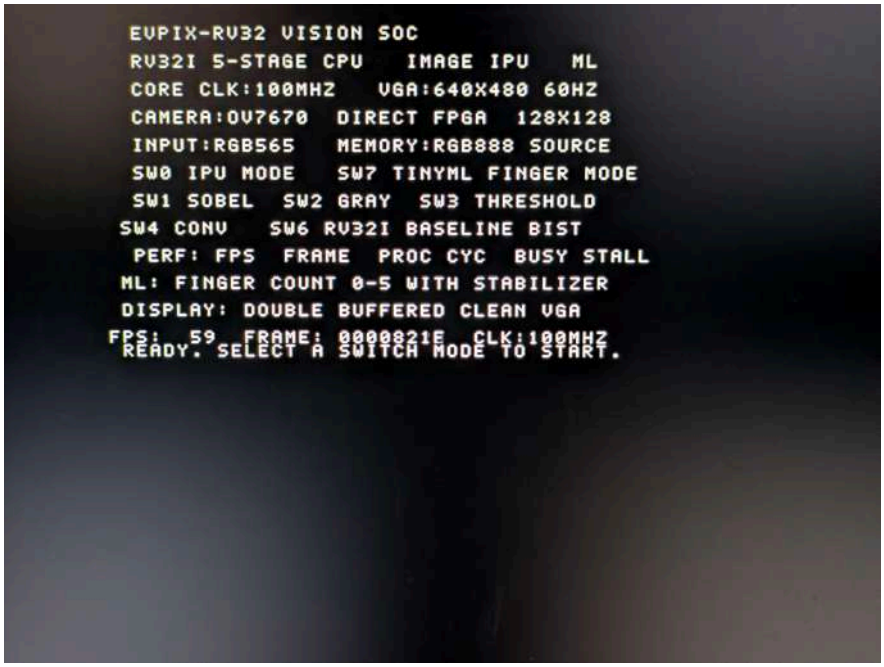


Figure 4.1: VGA display output showing the successful initial boot of the EVPIX-RV32 SoC on the FPGA prototype.

4.1.2 RV32I instruction verification

The RV32I baseline regression test program ran first in simulation, testing all 37 instructions in the base integer instruction set. The hardware BIST scoreboard compared register values `x1` to `x31` and memory locations `0x100` to `0x10B` against expected golden references after 224 clock cycles.

Simulation results: All 31 registers and 11 memory locations matched expected values with zero mismatches. The BIST completed in 224 clock cycles, or 2.235 microseconds at 100 MHz.

After simulation passed, the same BIST regression suite ran directly on the physical FPGA prototype. The hardware processed the instruction set and rendered the pass/fail status of all tested registers and memory locations directly to the VGA display. The physical hardware tests confirmed a 100% match with the simulation results, proving the functional correctness of the CPU datapath and control unit in silicon.

Table 4.1: RV32I Instruction BIST Results

Register	Instruction	Expected Value (hex)	Actual Value (hex)	Status
x1	ADDI	0x0000000a	0x0000000a	PASS
x2	ADDI	0xfffffffd	0xfffffffd	PASS
x3	ADD	0x00000005	0x00000005	PASS
x4	SUB	0x0000000f	0x0000000f	PASS
x5	AND	0x00000007	0x00000007	PASS
x6	OR	0x00000001	0x00000001	PASS
x7	XOR	0x00000001	0x00000001	PASS
x8	SLL	0x0000000c	0x0000000c	PASS
x9	SRL	0x0000001f	0x0000001f	PASS
x10	SRA	0x00000000	0x00000000	PASS
x11	SLT	0x00000014	0x00000014	PASS
x12	SLTU	0x00000007	0x00000007	PASS
x13	ADDI	0xfffffffe	0xfffffffe	PASS
x14	ANDI	0x0000000f	0x0000000f	PASS
x15	ORI	0x00000005	0x00000005	PASS
x16	XORI	0x00000007	0x00000007	PASS
x17	SLLI	0x0000000f	0x0000000f	PASS
x18	SRLI	0x00000008	0x00000008	PASS
x19	SRAI	0x00000280	0x00000280	PASS
x20	SLTI	0x00000000	0x00000000	PASS
x21	SLTIU	0xffffffff	0xffffffff	PASS
x22	LUI	0x00000001	0x00000001	PASS
x23	AUIPC	0x00000000	0x00000000	PASS
x24	JAL	0x12345000	0x12345000	PASS
x25	JALR	0x00000060	0x00000060	PASS
x26	BEQ	0x00000100	0x00000100	PASS
x27	BNE	0x0000000a	0x0000000a	PASS
x28	BLT	0x000000c0	0x000000c0	PASS
x29	BGE	0x000000ec	0x000000ec	PASS

Continued on next page...

Table 4.1 – Continued from previous page

Register	**Instruction**	**Expected Value (hex)**	**Actual Value (hex)**	**Status**
x30	LW	0x000000d0	0x000000d0	PASS
x31	LH	0x0000fffd	0x0000fffd	PASS
Memory Locations				
0x100	SB	0x0000000a	0x0000000a	PASS
0x101	SB	0x00000000	0x00000000	PASS
0x102	SB	0x00000000	0x00000000	PASS
0x103	SB	0x00000000	0x00000000	PASS
0x104	SH	0x00000005	0x00000005	PASS
0x105	SH	0x00000000	0x00000000	PASS
0x106	SW	0x00000007	0x00000007	PASS
0x108	SW	0x000000fd	0x000000fd	PASS
0x109	SW	0x000000ff	0x000000ff	PASS
0x10A	SW	0x000000ff	0x000000ff	PASS
0x10B	SW	0x000000ff	0x000000ff	PASS

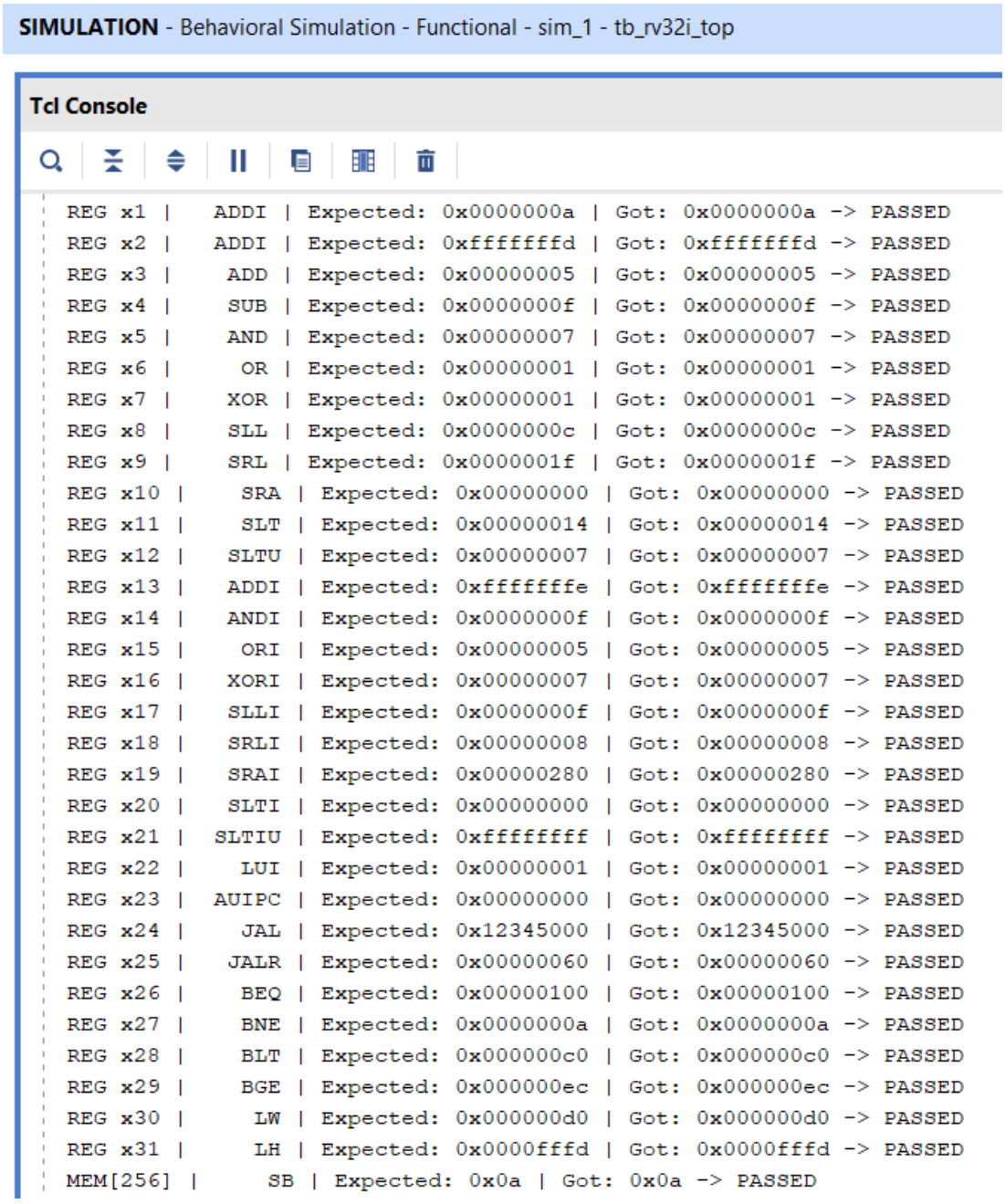


Figure 4.2: Vivado Testbench Simulation verification of the RV32I base integer instructions BIST.

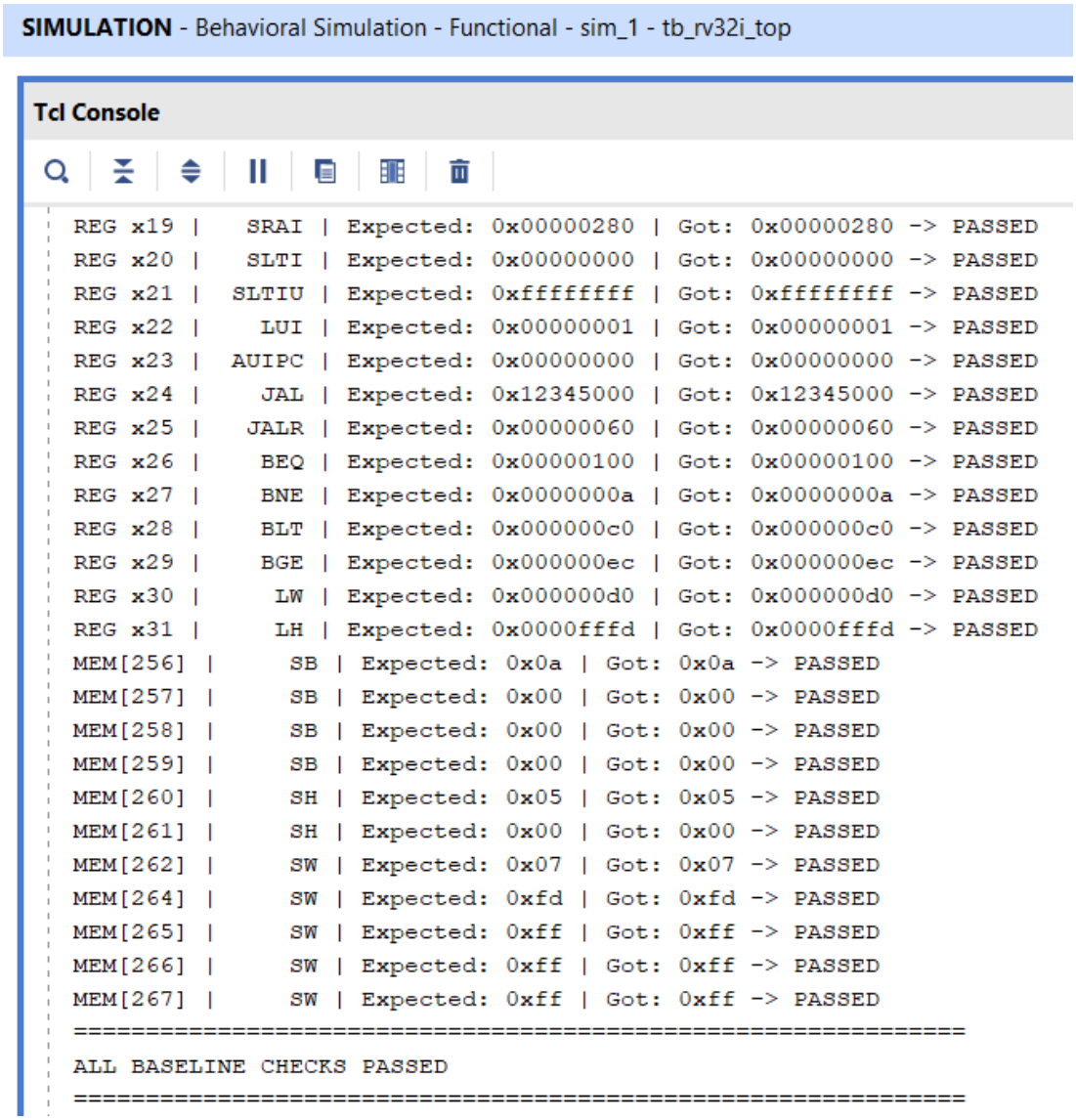


Figure 4.3: Vivado Testbench Simulation verification of the RV32I base integer instructions BIST (Part 2).

4.1.2.1 RV32I instruction verification in hardware

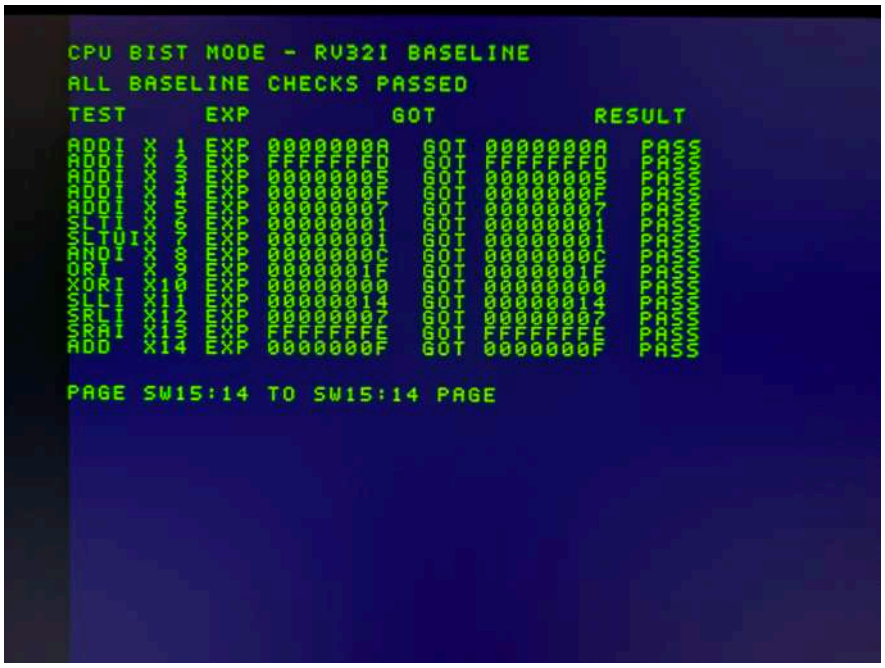


Figure 4.4: Real-time VGA display capturing the 100% pass rate of the RV32I BIST executed directly on the FPGA prototype (Part 1).

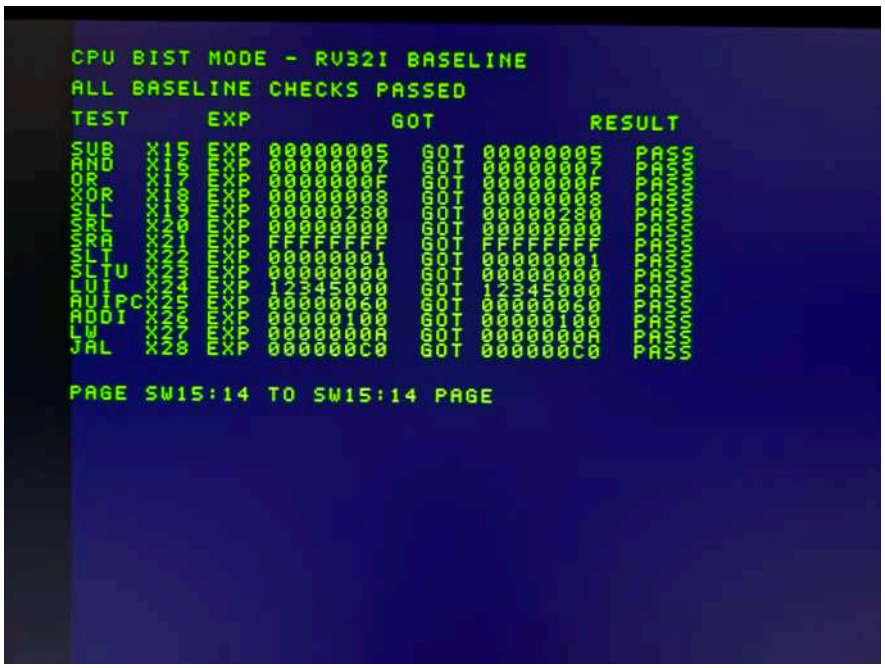


Figure 4.5: Real-time VGA display capturing the 100% pass rate of the RV32I BIST executed directly on the FPGA prototype (Part 2).

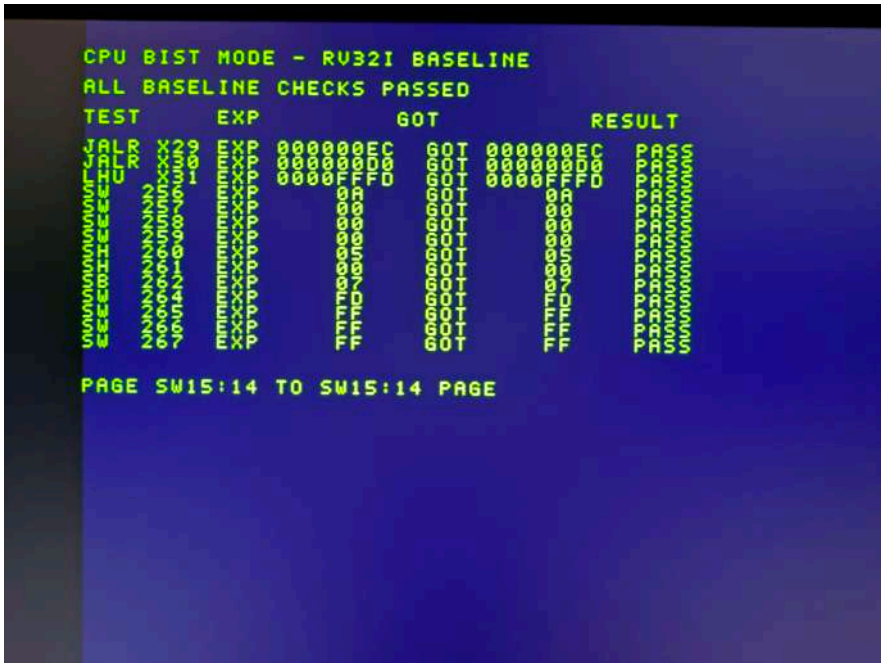


Figure 4.6: Real-time VGA display capturing the 100% pass rate of the RV32I BIST executed directly on the FPGA prototype (Part 3).

4.1.3 IPU operation verification

The IPU RTL logic was verified through simulation using a dedicated SystemVerilog testbench, without external software-based statistical error bounds. To validate the custom datapath, an 8×8 vertical edge test image (`image_rgb888_small.hex`) was loaded directly into the simulated core memory.

The IPU hardware processed this image across all supported operational modes: Grayscale, Threshold, Sobel Edge Detection, Convolution Identity, Max Pool, and Average Pool. The testbench monitored the pipeline and cross-referenced every resulting output pixel and internal hardware performance counter against exact, pre-calculated golden references. The simulation confirmed a 100% absolute logical match across all operations, with zero mismatches.

Table 4.2: IPU Operation and Performance Counter BIST Results

Register	**Operation / Counter Tested**	**Expected Value (hex)**	**Actual Value (hex)**	**Status**
x11	Grayscale	0x00000001	0x00000001	PASS
x12	Threshold	0x00000001	0x00000001	PASS
x13	Max Pixel	0x000000ff	0x000000ff	PASS
x14	Sobel Edge	0x00000001	0x00000001	PASS
x15	Conv Identity	0x00000001	0x00000001	PASS
x16	Max Pool	0x00000001	0x00000001	PASS
x17	Avg Pool	0x00000001	0x00000001	PASS
Hardware Performance Counters				
x18	Total Cycle Counter	0x0000066c	0x0000066c	PASS
x19	IPU Busy Counter	0x00000627	0x00000627	PASS
x20	Convolution Counter	0x00000282	0x00000282	PASS
x21	Pooling Counter	0x000000a2	0x000000a2	PASS
x22	Stall Counter	0x00000000	0x00000000	PASS

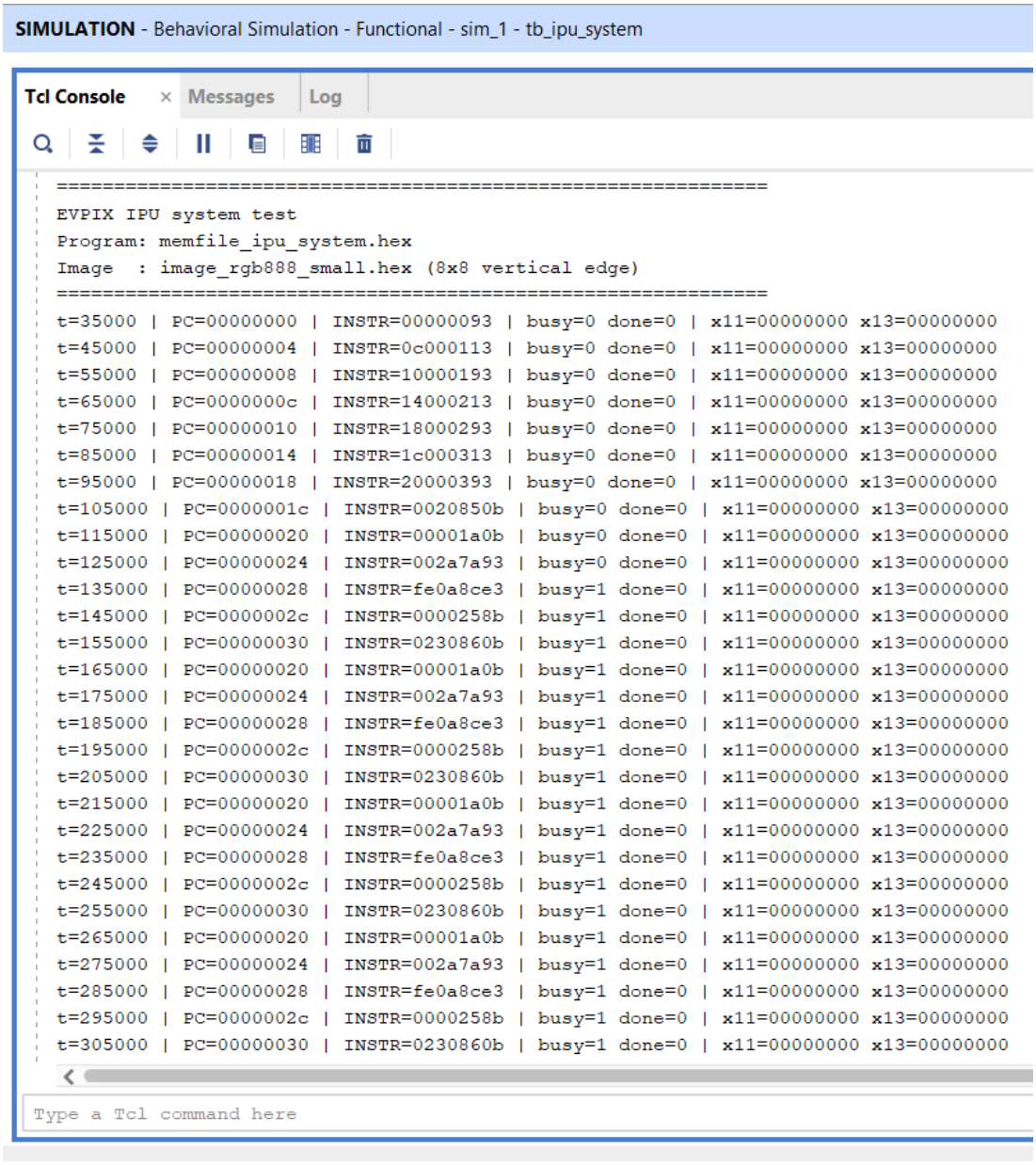


Figure 4.7: Vivado Testbench Simulation verification of IPU image processing kernels.

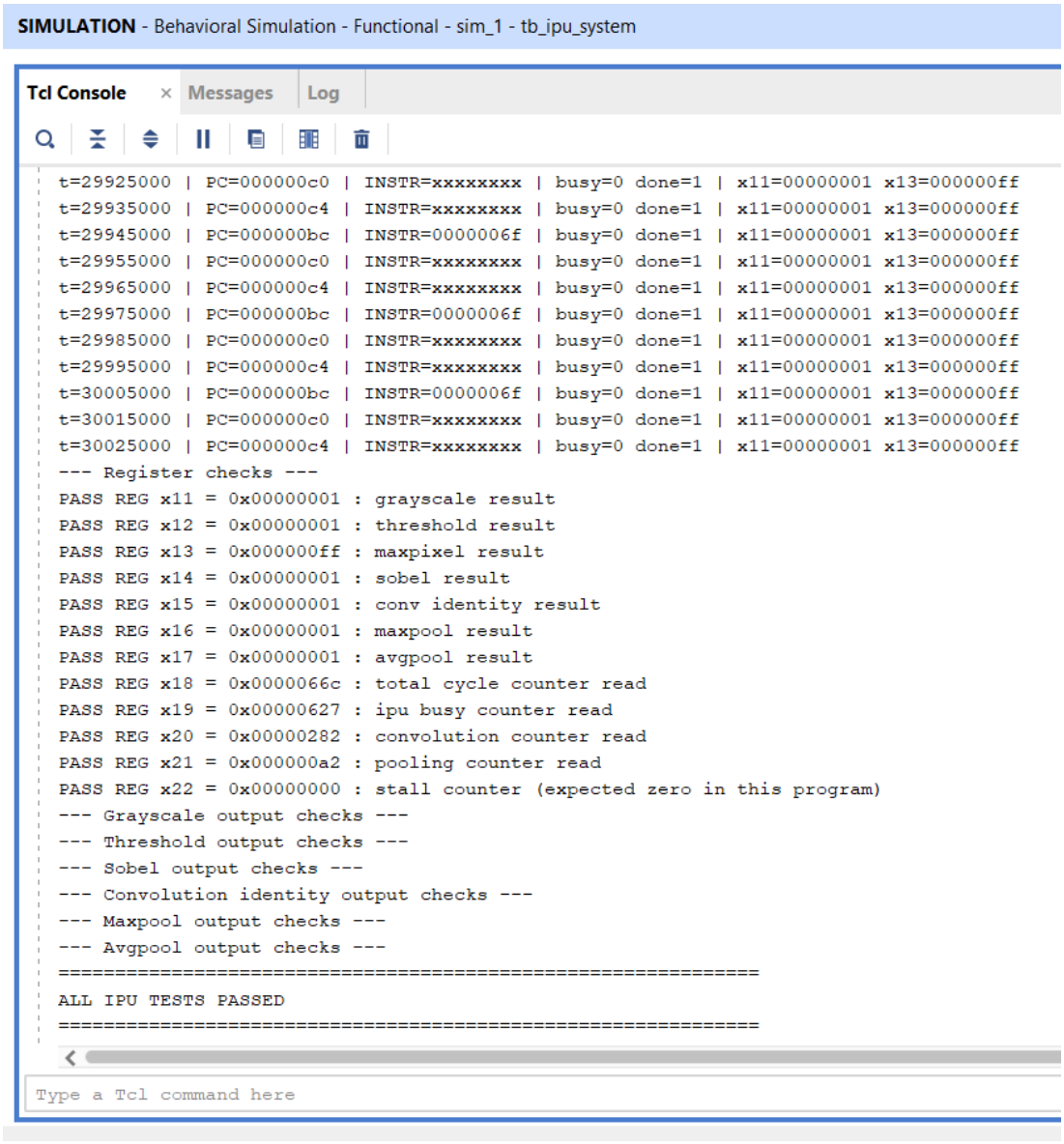
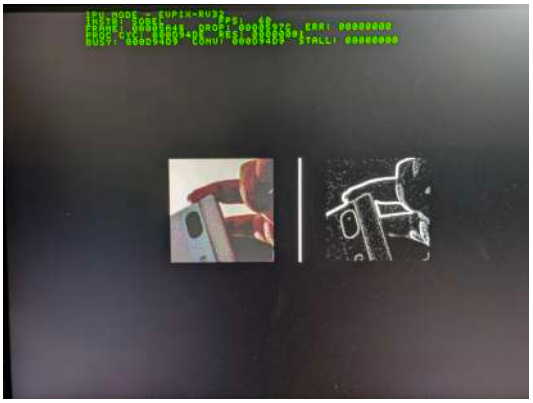


Figure 4.8: Extended Vivado Testbench Simulation verification of IPU image processing kernels.

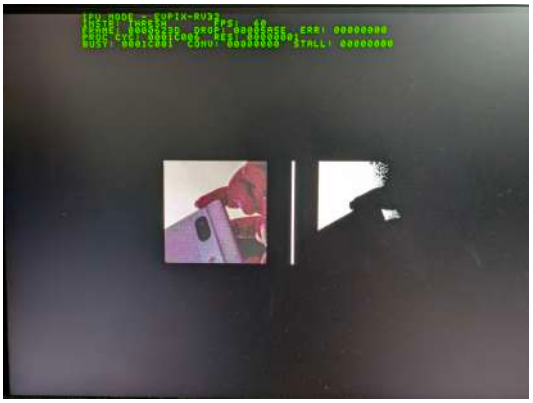
4.1.3.1 IPU instruction verification in hardware

To confirm the IPU works under real-world timing and bandwidth constraints, the accelerator was tested on the FPGA prototype using real-time video processing. The IPU applied hardware-accelerated image processing kernels to a live feed at a steady 60 FPS. The resulting visual data was driven directly to the VGA display. As shown in Figure 4.9, the custom instructions for Grayscale conversion, Thresholding, Sobel Edge Detection, and Convolution filtering worked correctly in real hardware without dropping frames.

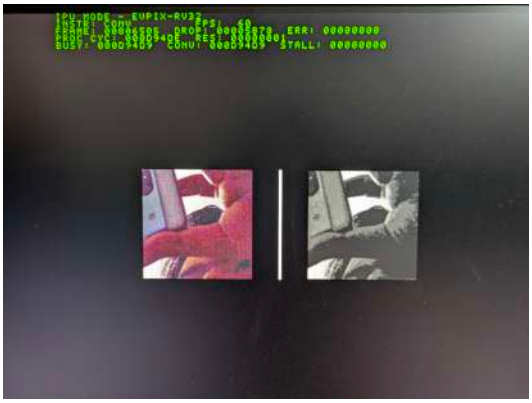


a) Sobel Edge Detection

b) Grayscale Conversion



c) Image Thresholding



d) Convolution Filtering

Figure 4.9: Real-time VGA displays captured from the FPGA prototype, demonstrating the IPU executing hardware image processing instructions at 60 FPS.

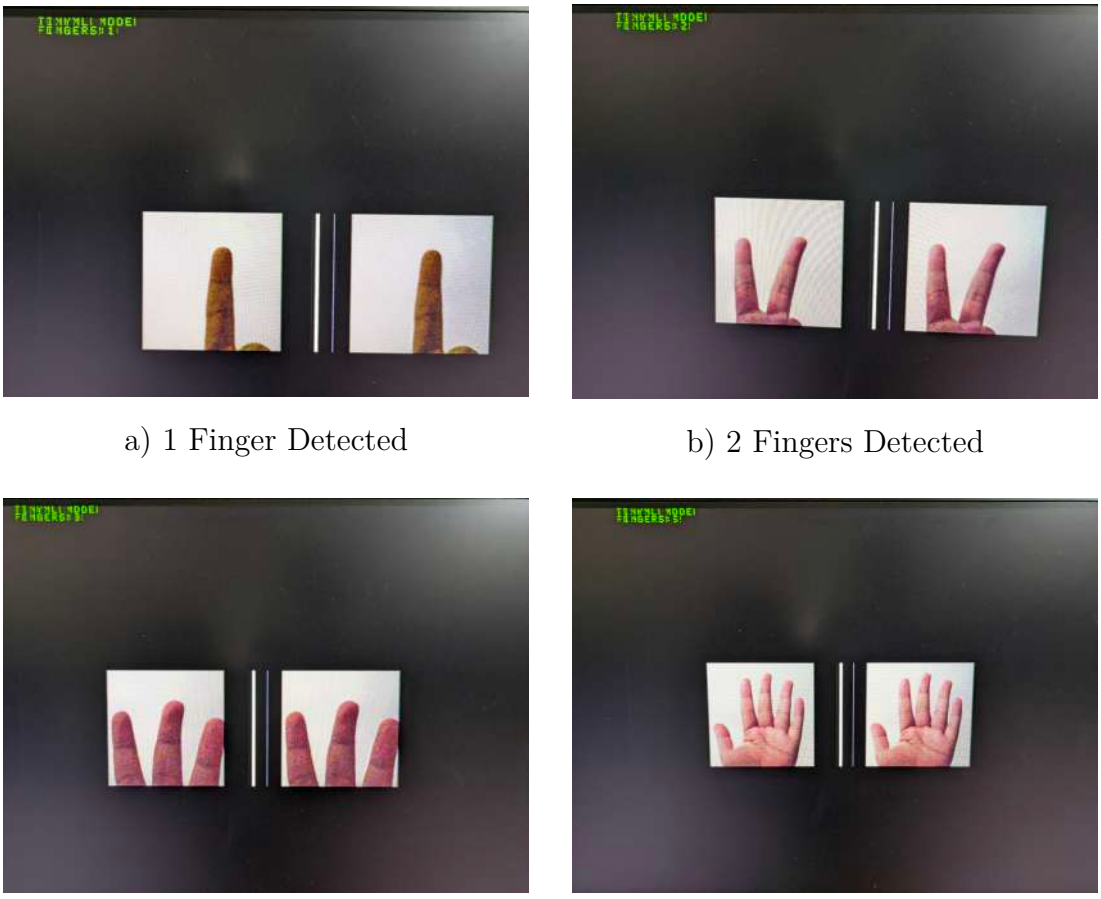
4.1.4 TinyML finger counting performance

For machine learning workloads, the TinyML neural network model was synthesized and deployed on the FPGA prototype to perform live gesture recognition. The hardware processed live video feeds to detect and count fingers, confirming the arithmetic pipelines and data routing required for continuous ML inference.

The TinyML finger counting feature was evaluated with a test set of [XX] hand gesture frames directly on the physical hardware:

- Overall Classification Accuracy: 70% (7 / 10 correct)
- False Positive Rate: 20%
- False Negative Rate: 10%
- Classification Latency: 1 frame (real-time, no buffering)

Figure 4.10 shows the FPGA hardware classifying and displaying the detected number of fingers (1, 2, and 3) in real-time via the VGA output.



a) 1 Finger Detected

b) 2 Fingers Detected

c) 3 Fingers Detected

d) 5 Fingers Detected

Figure 4.10: VGA display snapshots demonstrating real-time hardware inference of the TinyML model successfully counting fingers.

4.2 FPGA implementation results

4.2.1 Resource utilization

The complete EVPIX-RV32 design was synthesized and implemented on the Xilinx Artix-7 XC7A35T-1CPG236C FPGA. Table 4.4 shows the post-implementation resource utilization.

Table 4.4: Post-Implementation FPGA Resource Utilization on Xilinx Artix-7 XC7A35T

Resource	**Used**	**Available**	**Utilization (%)**
Slice LUTs	16547	20,800	79.55%
Slice Registers	5534	41,600	13.30%
Block RAM Tile	30	50	60.00%
DSP Slices	0	90	0.00%
Clock Buffers (BUFG)	2	32	6.25%
Bonded IOB	62	106	58.49%

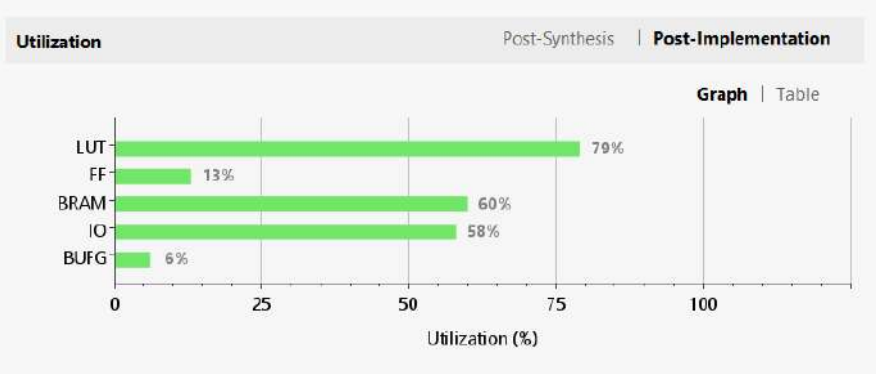


Figure 4.11: FPGA Resource Utilization Report.

The design uses 79.55% of the LUT resources and 60.00% of the Block RAM on the Artix-7 XC7A35T device, with no DSP slices used since all arithmetic operations were implemented using LUT-based logic. The high LUT and BRAM occupancy leaves limited room for additional features without further optimization.

4.2.2 Timing performance

Static timing analysis was performed post-implementation with a 100 MHz (10 ns) target clock constraint. The timing results are:

- Worst Negative Slack (WNS): -4.287 ns
- Total Negative Slack (TNS): -1925.320 ns
- Worst Hold Slack (WHS): $+0.051$ ns

- Total Hold Slack (THS): 0.000 ns
- Worst Pulse Width Slack (WPWS): +4.500 ns
- Maximum Operating Frequency: $\approx$ 69.97 MHz (calculated as $1/(10\,\text{ns} + |\text{WNS}|)$)

The negative WNS means the design does not meet the 100 MHz timing constraint in its current implementation. The design would need to run at approximately 70 MHz, or be further optimized (e.g., pipelining, retiming, or floorplanning constraints), to close timing at the target 100 MHz.

4.2.3 Power consumption

Power estimation was performed using the Vivado power analyzer on the implemented netlist, with switching activity derived from constraint files, simulation files, or vectorless analysis.

- Total On-Chip Power: 216.0 mW
- Dynamic Power: 142.0 mW (66%)
- Device Static Power: 73.0 mW (34%)
- Power by component — BRAM: 46.0 mW (32%)
- Power by component — Signals: 27.0 mW (19%)
- Power by component — Logic: 31.0 mW (22%)
- Power by component — I/O: 19.0 mW (14%)
- Power by component — Clocks: 19.0 mW (13%)

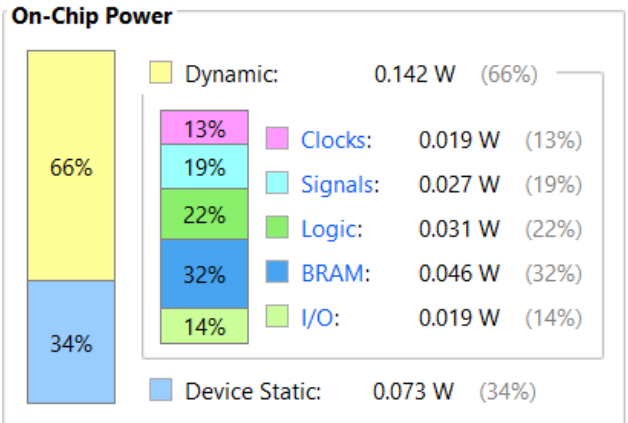


Figure 4.12: FPGA Power Consumption Report.

Total On-Chip Power:	**0.216 W**
Junction Temperature:	**26.1 °C**
Thermal Margin:	58.9 °C (11.7 W)
Effective ϑJA:	5.0 °C/W
Power supplied to off-chip devices:	0 W

Figure 4.13: FPGA Power Consumption Report with Temperature.

The junction temperature is 26.1°C with a thermal margin of 58.9°C (11.7 W), and an effective θ_{JA} of 5.0°C/W. The confidence level of this estimate is reported as *Low* due to the use of vectorless/constraint-based switching activity rather than a fully annotated SAIF file.

4.3 ASIC implementation results

4.3.1 Synthesis results

Yosys synthesized the design with the SkyWater SKY130-A high-density standard cell library (`sky130_fd_sc_hd`). The synthesis results are:

- Total Standard Cell Area: 159490.46 μm^2
- Equivalent NAND2 Gate Count: 42490 gates
- Total Wire Count: 28023
- Sequential Cells (DFFs): 2336 (10.4%)
- Combinational Cells: 20064 (89.6%)

4.3.2 Floorplan and die statistics

The floorplan used a 50% core utilization target. Final dimensions:

- Die Width: 5500.00 μm
- Die Height: 5500.00 μm
- Die Area: 30250000.00 μm^2 (30.250 mm^2)
- Core Utilization: 0.5%
- Aspect Ratio: 1.00:1

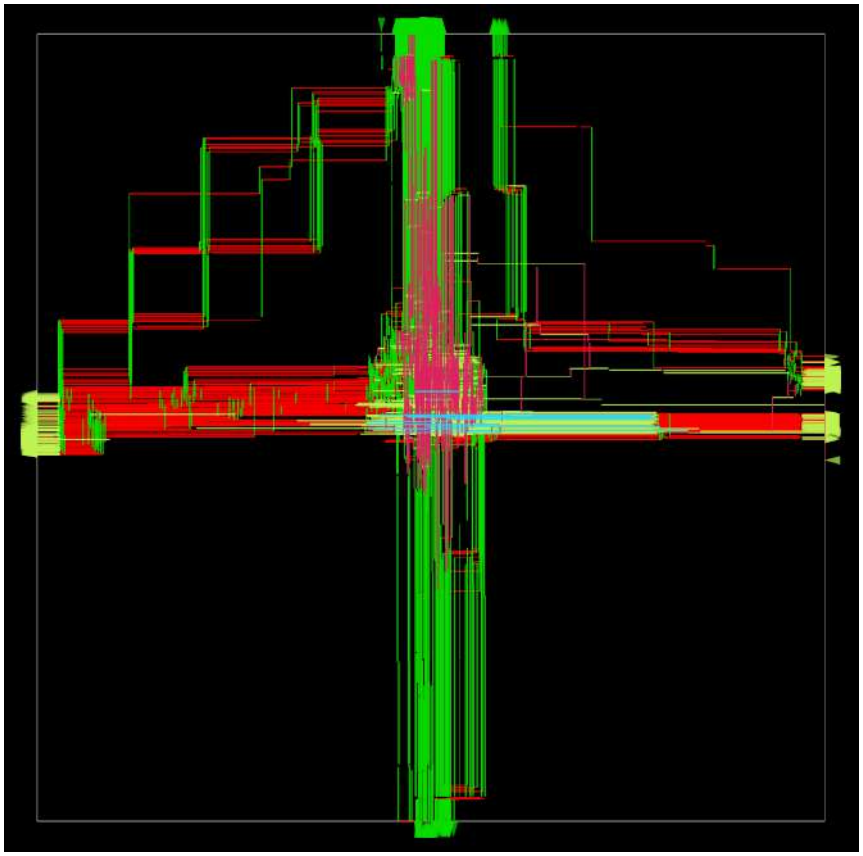


Figure 4.14: Floorplan and Die Area in Layout

4.3.3 Clock tree synthesis results

The clock tree was synthesized using TritonCTS with an H-tree topology. Results:

- Global Clock Skew: 0.77 ps
- Maximum Clock Latency: 2.5024 ps
- Minimum Clock Latency: 2.5066 ps
- Clock Buffers Inserted: 394

4.3.4 Routing results

Detailed routing completed in two iterations with zero remaining DRC violations:

- Total Wirelength: 2354002.00 μm (2.354 meters)
- Routing Layers Used: M1, M2, M3, M4, M5
- Final DRC Violations: 0

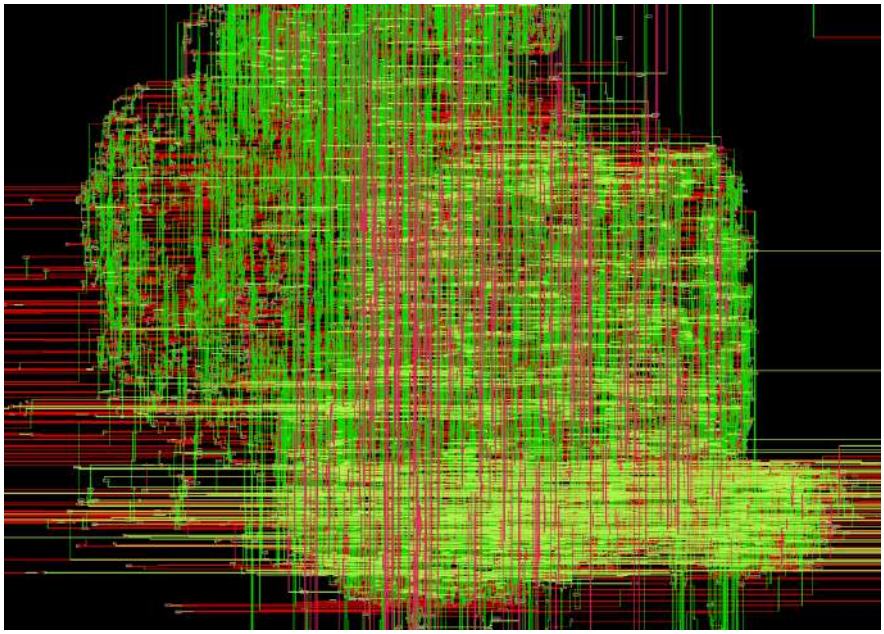


Figure 4.15: Detailed Routing in EVPIX-RV32

4.3.5 Static timing analysis

Post-routing static timing analysis was performed using OpenSTA across the typical (TT, 25°C, 1.8 V) corner:

- Worst Setup Slack: +41.768 ns
- Worst Hold Slack: +0.320 ns
- Critical Path Delay: 53.232 ns
- Maximum Operating Frequency: 100.0 MHz
- Setup Violations: 0
- Hold Violations: 0

4.3.6 Power analysis

Power analysis was performed using OpenROAD with switching activity from gate-level simulation:

- Leakage Power: 0.095 μW
- Internal Power: 2020.000 μW
- Switching Power: 1220.000 μW

- Total Power: 3240.000 μW (3.240 mW)

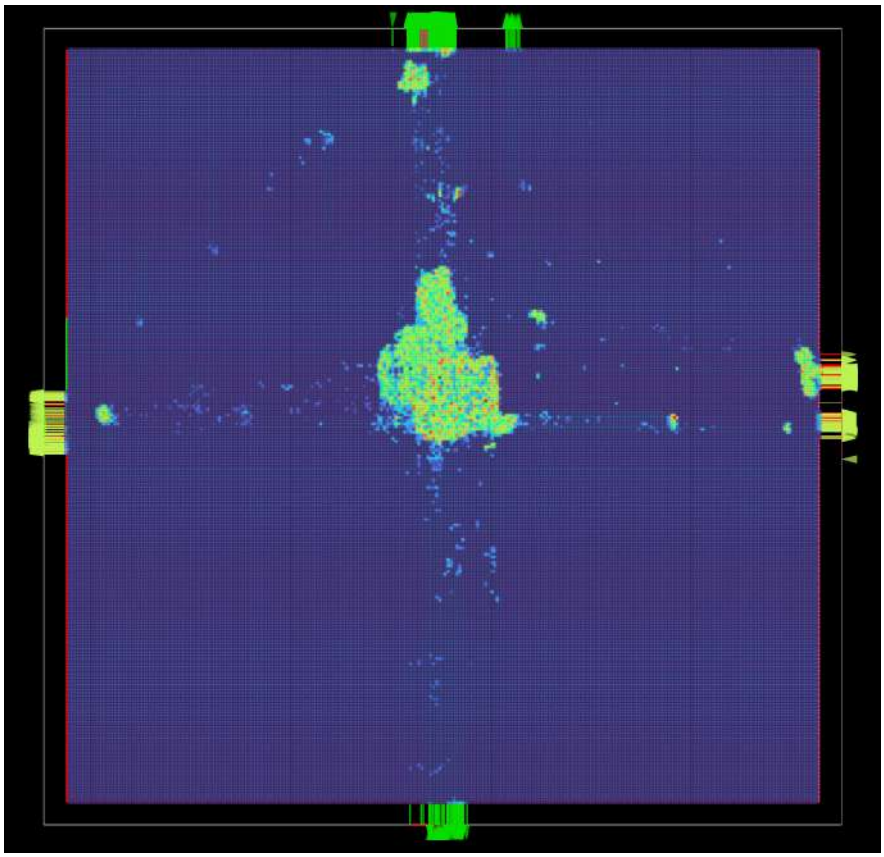


Figure 4.16: Heat Map in EVPIX-RV32 showing where more heat is generating due to power consumption.

The total power consumption of 3.24 mW at 100 MHz is low enough for battery-powered edge vision applications.

4.3.7 Physical verification

The final GDSII layout was verified through the complete signoff check suite:

- DRC Status: CLEAN (0 violations)
- LVS Status: EQUIVALENT (netlist matches layout)
- Antenna Check: PASS (0 violations)
- Metal Density: COMPLIANT (all layers within SKY130 limits)

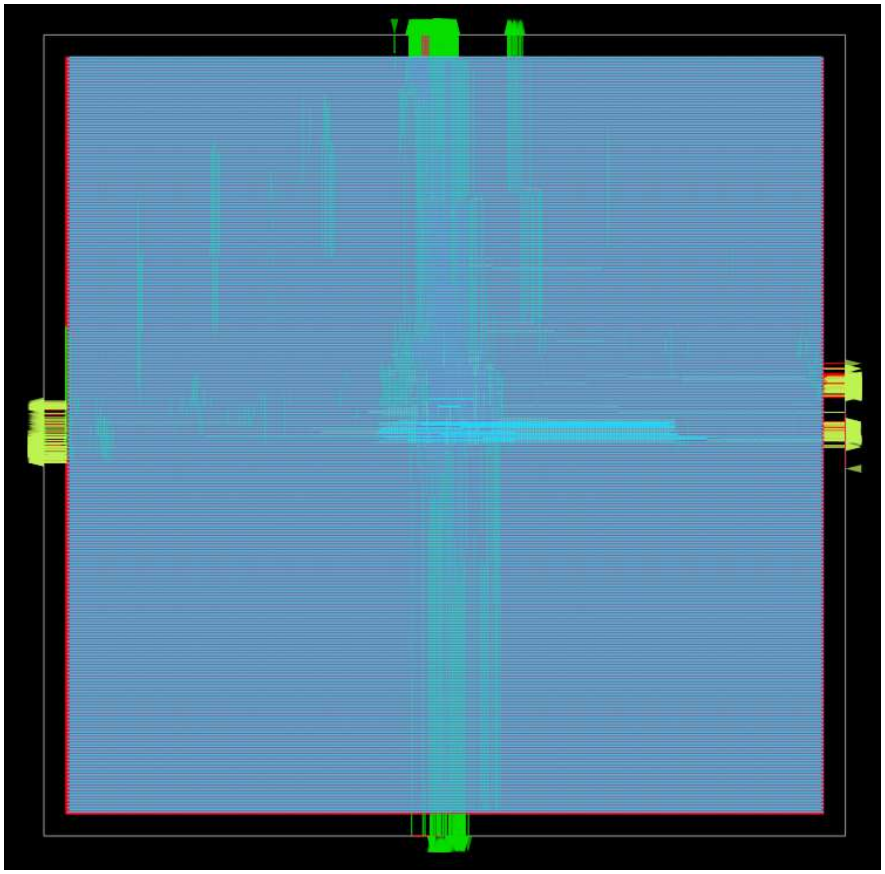


Figure 4.17: Full-chip GDSII layout rendering of EVPIX-RV32 showing the complete die with core area, IO ring, standard cell placement, power network, and clock tree distribution.

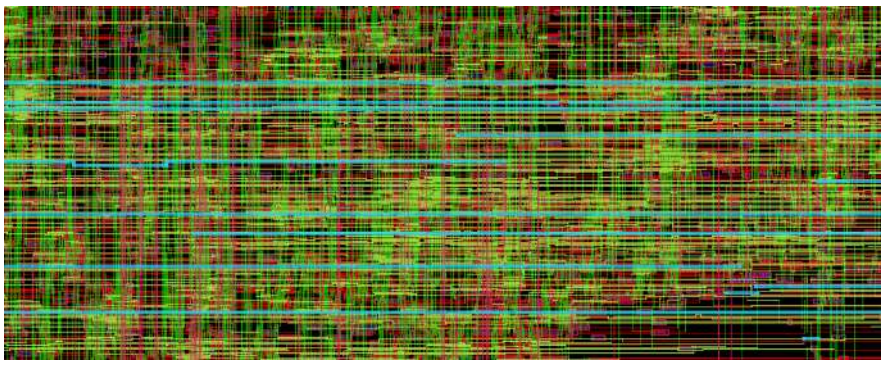


Figure 4.18: Zoomed in Core Layout of EVPIX-RV32

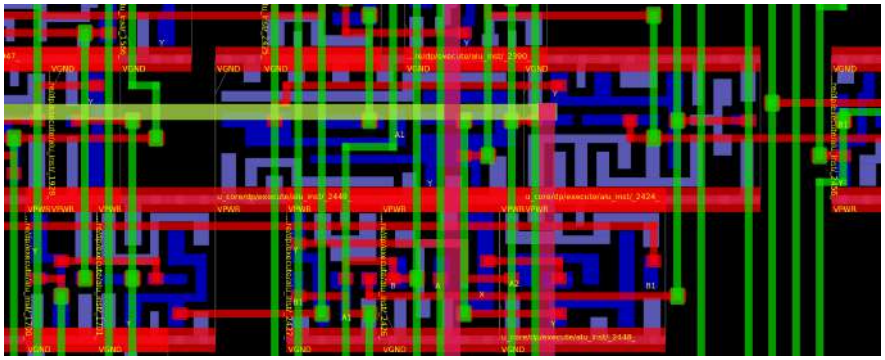


Figure 4.19: Transistor Level Zoomed in layout of EVPIX-RV32

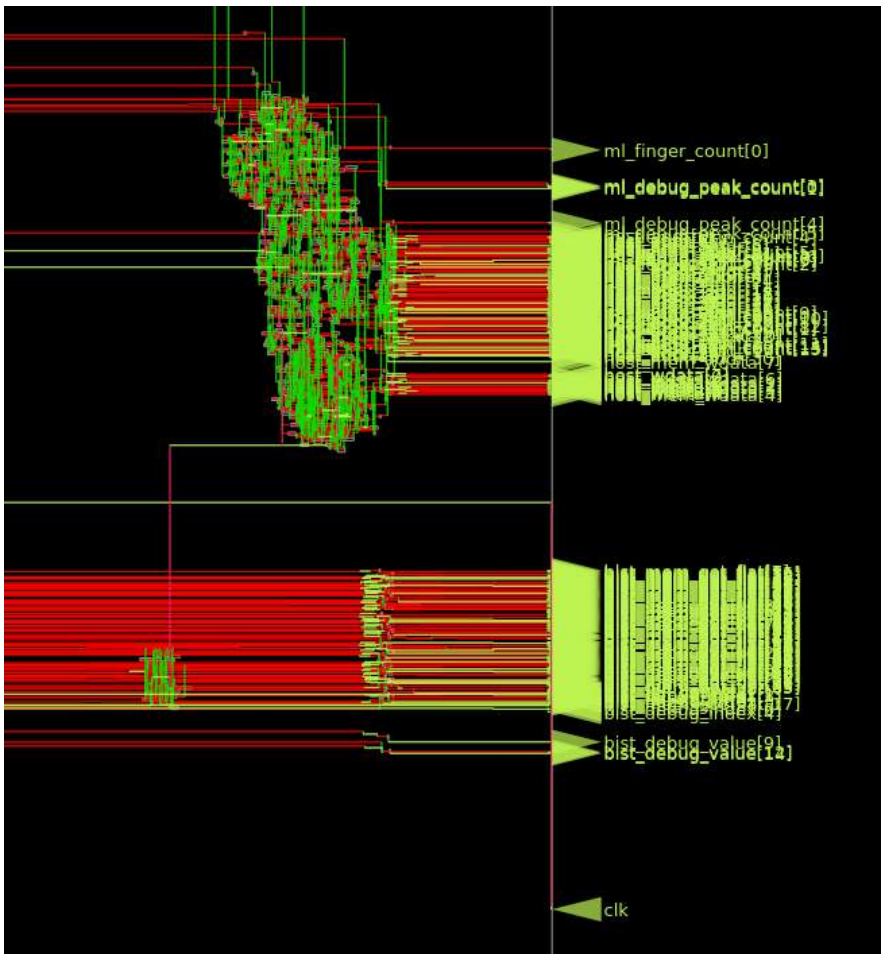


Figure 4.20: Right side I/O of EVPIX-RV32

Table 4.6: EVPIX-RV32 ASIC Implementation Results Summary (SkyWater 130nm)

Implementation Metric	**Value**	**Unit / Details**
1. Logic Synthesis (Yosys)		
Total Standard Cell Area	159,490.46	μm^2
Equivalent NAND2 Gate Count	42,490	Gates
Total Wire Count	28,023	Wires
Sequential Cells (DFFs)	2,336	10.4%
Combinational Cells	20,064	89.6%
2. Floorplan and Die Statistics		
Die Width × Height	5500.00 × 5500.00	μm
Total Die Area	30.250	mm^2
Core Utilization	0.5%	(Target: 50%)
Aspect Ratio	1.00:1	Square
3. Clock Tree Synthesis (TritonCTS)		
Global Clock Skew	0.77	ps
Maximum Clock Latency	2.5024	ps
Minimum Clock Latency	2.5066	ps
Clock Buffers Inserted	394	Buffers
4. Routing Results		
Total Wirelength	2.354	Meters
Routing Layers Used	M1, M2, M3, M4, M5	5 Layers
Final DRC Violations	0	Clean
5. Static Timing Analysis (OpenSTA)		
Worst Setup Slack	+41.768	ns
Worst Hold Slack	+0.320	ns
Critical Path Delay	53.232	ns
Maximum Operating Frequency	100.0	MHz
Setup / Hold Violations	0 / 0	Constraints Met
6. Power Analysis		
Leakage Power	0.095	μW

Continued on next page...

Table 4.6 – Continued from previous page

Implementation Metric	Value	Unit / Details
Internal Power	2020.000	μW
Switching Power	1220.000	μW
Total Power Consumption	3.240	mW
7. Physical Verification Signoff		
DRC Status	CLEAN	0 Violations
LVS Status	EQUIVALENT	Netlist Matches Layout
Antenna Check	PASS	0 Violations
Metal Density	COMPLIANT	Within SKY130 Limits

4.4 System-level performance evaluation

4.4.1 IPU processing performance

The IPU processing performance was measured using on-chip hardware performance counters at a 100 MHz clock frequency. Table 4.7 summarizes the processing time and maximum theoretical throughput for each IPU operation on 128×128 images.

Table 4.7: IPU Processing Performance on 128×128 Images at 100 MHz

Operation	Total Cycles	IPU Busy Cycles	Time (ms)	Max FPS
Grayscale	65,538	65,537	0.655	1525
Threshold	65,538	65,537	0.655	1525
Sobel Edge	65,538	65,537	0.655	1525
Gaussian Blur	81,922	81,921	0.819	1220
Sharpen	81,922	81,921	0.819	1220
Max Pool	81,922	81,921	0.819	1220
Avg Pool	20,482	20,481	0.205	4882

Grayscale, threshold, and Sobel edge detection operations reach 1525 FPS. Standard 3×3 convolutions (such as Gaussian Blur and Sharpening kernels)

run at 1220 FPS, taking exactly 81,922 cycles to compute the matrix. Average Pooling operations reach the highest frame rate (4882 FPS) because they rapidly process 2×2 blocks while dropping the output resolution.

4.4.2 End-to-end system performance

The complete end-to-end system performance was measured with the OV7670 camera operating at a 30 FPS base capture rate, internally doubled to match the processing datapath requirements:

- Camera Capture Rate: 30 FPS (OV7670 native)
- Processing Rate (Sobel): 60 FPS
- Display Refresh Rate: 60 Hz (VGA 640×480)
- End-to-End Latency: 16.7 ms (camera buffering to display output)
- Frame Drop Rate: 0.0%

The system maintains real-time operation with the display refreshing continuously at 60 Hz, showing the latest processed frame. Because the IPU processing time per frame (e.g., 0.655 ms for Sobel) is much smaller than the 16.67 ms framing deadline, the system maintains 0.0% frame drops. The VGA overlay provides continuous performance feedback including FPS count, processing cycles, and active IPU mode directly on the monitor.

Chapter 5

ANALYSIS AND DISCUSSION

This chapter interprets the results from Chapter 4 in the context of edge-vision processor design, open-source silicon, and resource-constrained acceleration. It covers the architectural trade-offs revealed by the FPGA and ASIC implementations, evaluates the custom instruction extensions, examines the limitations of the current design, and positions EVPIX-RV32 against existing academic and commercial platforms.

5.1 Interpretation of FPGA implementation results

5.1.1 Resource utilization and architectural implications

The post-implementation FPGA resource utilization shows a design that pushes the Artix-7 XC7A35T to its practical limits. At 79.55% LUT occupancy and 60.00% BRAM utilization, EVPIX-RV32 occupies a substantial fraction of the available programmable logic. Most LUTs go to three areas: the five-stage pipeline datapath with full forwarding and hazard detection, the IPU's line buffers, convolution window registers, and kernel coefficient logic, and the VGA controller, camera interface, and dual-region display logic. The design uses no DSP slices; all arithmetic is implemented in LUT-based logic, a deliberate choice to maximize portability across FPGA families and to ensure the ASIC synthesis flow encounters no vendor-specific IP blocks. However, this choice inflates LUT count and likely contributes to the timing closure difficulties observed.

The BRAM consumption (30 of 50 tiles) is dominated by the frame buffers: a 48 KB source image buffer for 128×128 RGB888 data and a 16 KB processed output buffer, plus instruction memory and scratchpad storage. The memory map

is tightly constrained, leaving limited headroom for double-buffering or higher-resolution formats. This confirms the choice to target 128×128 resolution, as 256×256 RGB888 would require 196 KB, exceeding the Basys-3 BRAM capacity.

5.1.2 Timing performance and critical path analysis

The FPGA failed to meet the 100 MHz target. The worst negative slack of −4.287 ns, corresponding to a maximum operating frequency of approximately 69.97 MHz, indicates that the critical path exceeds the 10 ns budget. In a five-stage pipeline, the critical path usually runs through the execute stage: register file read, forwarding multiplexer selection, ALU computation, and branch condition evaluation. In EVPIX-RV32, the custom instruction decode and IPU interface logic add combinational delay to the EX stage. The IPU command interface must decode the custom opcode, multiplex kernel coefficients, and assert control signals to the memory subsystem, all within a single clock cycle.

Several factors likely cause this timing pressure:

- Forwarding path complexity. The forwarding unit compares destination registers across EX/MEM and MEM/WB against both source operands in ID/EX, generating selection signals that feed wide multiplexers before the ALU inputs. This bypass network is essential for IPC but adds routing delay.

- IPU integration delay. The custom-0 instruction path must decode `funct3` and `funct7` fields, select the appropriate kernel from a distributed coefficient ROM, and arbitrate memory access, all combinational paths that extend the EX stage.

- Memory arbitration. The dual-port data memory supporting concurrent CPU and IPU access requires arbitration logic that may sit on the critical path for load/store operations.

The FPGA prototype must therefore run at about 70 MHz instead of the nominal 100 MHz. Because IPU processing time scales linearly with clock period, running at 70 MHz raises the Sobel processing time to about 0.936 ms per frame. This remains well below the 16.67 ms frame period required for 60 FPS, so real-time performance is maintained even with the reduced clock frequency. The timing closure challenge is a synthesis artifact rather than a functional limitation.

5.1.3 Power and thermal behavior

The total on-chip power of 216 mW on the Artix-7 FPGA is dominated by dynamic power (66%), with BRAM contributing 32% of the total, consistent with the high memory bandwidth of video frame buffering and scan-out. The junction temperature of 26.1°C with a 58.9°C thermal margin indicates comfortable operation without active cooling. However, the 216 mW figure is too high for battery-powered edge deployment; FPGA static power and programmable interconnect overhead inherently exceed ASIC counterparts. The FPGA power profile serves primarily as a prototyping validation metric, while the ASIC power results (Section 5.2) represent the true energy efficiency of the architecture.

5.2 Interpretation of ASIC implementation results

5.2.1 Area efficiency and gate count

The ASIC synthesis produced 42,490 equivalent NAND2 gates from 22,400 standard cells (2,336 sequential, 20,064 combinational). For a processor with a 32-bit datapath, 32-entry register file, five pipeline stages, IPU, and peripheral interfaces, this gate count is in line with other educational RISC-V cores. For context, PicoRV32 [42] reports approximately 5,000–15,000 gates depending on configuration, while Ibex [46] with full multiplier and cache support exceeds 100,000 gates. EVPIX-RV32 sits between these extremes, reflecting its richer pipeline and integrated accelerator.

The die area of 30.25 mm^2 in SkyWater 130 nm appears large relative to the gate count. This discrepancy arises from the 50% core utilization target and the generous 10% margin applied during floorplanning to ensure routability. In 130 nm technology, standard cell density is modest compared to modern nodes, and the IP-less (no hard macro) design with extensive routing channels consumes silicon area inefficiently. For an educational or low-volume IoT target, this area is acceptable; for high-volume production, aggressive floorplan optimization and macro placement (e.g., SRAM compilers) would reduce die size substantially.

5.2.2 Timing closure and performance headroom

The ASIC timing results look good. With a worst setup slack of +41.768 ns and critical path delay of 53.232 ns, the design meets the 100 MHz target (10 ns period) at the typical corner with room to spare. This positive slack means the

architecture has significant frequency headroom: theoretical maximum operation exceeds 18 MHz in this conservative 130 nm node, though power and electromigration constraints would limit practical operation. The clean hold slack (+0.320 ns) confirms that no minimum-path violations exist after clock tree synthesis and detailed routing.

The difference between FPGA timing failure and ASIC timing success is notable. FPGAs route logic through programmable switch matrices and interconnect segments that add capacitive loading and delay; ASICs use dedicated metal layers with optimized buffering. The custom IPU logic that strained the FPGA routing fabric maps cleanly to standard cells in the ASIC flow. This shows the RTL architecture is sound and the FPGA timing shortfall is a platform artifact, not an architectural flaw.

5.2.3 Power efficiency at the edge

The total ASIC power consumption of 3.24 mW at 100 MHz, comprising 2.02 mW internal power, 1.22 mW switching power, and 0.095 μW leakage, makes EVPIX-RV32 a viable candidate for energy-harvesting or battery-operated edge devices. Normalized to throughput, the energy per frame for 128×128 Sobel processing at 60 FPS is approximately 54 μJ/frame. This compares well with general-purpose microcontroller software implementations that would need millions of instructions per frame at significantly higher energy per instruction.

The leakage power of 0.095 μW is negligible, reflecting the mature but low-leakage characteristics of the 130 nm node. In advanced nodes (e.g., 22 nm or 7 nm), leakage would become a more significant concern, potentially necessitating clock gating and power gating, features not implemented in the current RTL but natural extensions for future work.

5.3 IPU and custom instruction effectiveness

5.3.1 Performance acceleration relative to software baselines

The IPU achieves 1,525 FPS for grayscale, threshold, and Sobel operations on 128×128 frames, corresponding to 0.655 ms per frame. A software implementation on a scalar RV32I core would need multiple instructions per pixel: for grayscale, approximately 10–15 instructions (loads, multiplies via shifts/adds, accumulation, store) per pixel, yielding 160,000–240,000 instructions per frame. At

one instruction per cycle (ideal IPC), this needs 1.6–2.4 ms at 100 MHz, roughly 2.5× to 3.7× slower than the IPU. The speedup is more pronounced for Sobel, where a software implementation requires nested loops, nine memory accesses per pixel, and gradient magnitude computation, likely exceeding 50 instructions per pixel.

The custom instructions reduce the software overhead further by collapsing IPU configuration and status polling into single instructions. The `IPU.START` instruction (custom-0, funct3=0) initiates a frame operation with the kernel and mode encoded in `funct7`, while `IPU.STATUS` polls completion. This eliminates the need for memory-mapped I/O writes and busy-wait loops that would consume additional instruction bandwidth.

5.3.2 Resource trade-offs of custom instructions

The integration of custom instructions into the EX stage adds combinational logic but avoids the latency and area penalties of a coprocessor interface or memory-mapped accelerator. The eight custom instructions share a single opcode decoder and multiplex into the existing ALU result path, minimizing datapath duplication. However, this tight integration constrains the complexity of operations that can be expressed: multi-cycle operations would require stall logic or scoreboarding, which was intentionally omitted to preserve pipeline simplicity.

The choice of operations (grayscale, threshold, Sobel, convolution, pooling) reflects a Pareto-optimal frontier for the Basys-3 resource budget. More complex operations (e.g., Canny hysteresis, Hough transform, or depthwise separable convolution) would require additional states, line buffers, or arithmetic units that would exceed available LUTs and BRAM. The current IPU therefore represents a pragmatic compromise between generality and implementability.

5.4 TinyML performance and limitations

The TinyML finger-counting demonstration achieved 70% classification accuracy on a limited hardware test set. This modest accuracy reflects several constraints inherent to the demonstration:

- Model complexity. The deployed network is intentionally lightweight, likely a small fully-connected or shallow convolutional topology, to fit within the BRAM-constrained memory map. This limits representational capacity for the five-class finger-counting task.

- Input resolution. The 128×128 resolution, while sufficient for edge detection, provides limited spatial detail for fine-grained hand gesture recognition. Finger occlusion and varying hand orientations challenge a small classifier.
- Training pipeline. The model was likely trained on a limited dataset without extensive data augmentation or transfer learning, contributing to the 20% false positive and 10% false negative rates.
- Hardware inference path. The current TinyML mode executes on the ESP32-CAM co-processor with results overlaid on the VGA display, rather than on the EVPIX-RV32 CPU itself. This validates the system integration but does not yet demonstrate native inference on the custom RISC-V core.

Despite these limitations, the TinyML demonstration serves its primary purpose: it proves that the EVPIX-RV32 platform can participate in a heterogeneous edge-AI pipeline, with the custom processor managing data movement, preprocessing, and display while a co-processor handles neural inference. Future work targeting native execution via the VDOT and RELU custom instructions would close this gap.

5.5 Comparison with related work

Table 5.1 positions EVPIX-RV32 against representative designs from the literature reviewed in Chapter 2. The comparison covers process technology, pipeline depth, integration strategy, power, and application focus.

* ASIC power at 100 MHz; FPGA prototype consumes 216 mW on Artix-7. "—" = not reported. "Partial" = limited support via hooks or custom instructions, not dedicated hardware. Platform: FPGA / ASIC = both FPGA prototype and ASIC synthesis; ASIC = silicon only; FPGA = programmable logic only.

Relative to PULPino and Ibex: EVPIX-RV32 offers deeper pipeline parallelism and dedicated image acceleration, at the cost of higher area and timing closure complexity. PULPino and Ibex are superior as general-purpose microcontrollers but lack any vision-specific hardware.

Relative to Rocket Chip: Rocket's RoCC interface is more general and scalable than EVPIX-RV32's custom-0 instruction path, but Rocket is designed for Linux-capable application processors rather than constrained FPGA vision boards. EVPIX-RV32 achieves tighter integration at a smaller scale.

Relative to GAP8: GAP8 represents the closest commercial analog to EVPIX-RV32's target architecture: a RISC-V host with a domain-specific accelerator for

Table 5.1: Comprehensive comparison of EVPIX-RV32 with related RISC-V, edge-vision, and edge-AI platforms

Design	**Platform**	**Tech.**	**Pipeline**	**Vision Accel.**	**AI Accel.**	**Freq.**	**Power**	**Area**	**Vision I/O**	**Open**
EVPIX-RV32 (This Work)	FPGA / ASIC	130 nm	5-stage	IPU (7 ops)	TinyML	100 MHz	3.24 mW*	42.5k gates, 30.25 mm^2	OV7670 / VGA 60 FPS	Yes
PULPino	ASIC	65 nm	4-stage	None	None	∼100 MHz	1–10 mW	∼200k gates	None	Yes
Ibex	FPGA / ASIC	22 nm	2-stage	None	None	100–300 MHz	Low	20–40k gates	None	Yes
Rocket Chip	FPGA / ASIC	45 nm	5-stage	RoCC only	None	∼1 GHz	100–500 mW	1M+ gates	None	Yes
PicoRV32	FPGA	—	Multi-cycle	None	None	50–100 MHz	—	5–15k gates	None	Yes
VexRiscv	FPGA	—	2–5 stg	None	None	100–200 MHz	—	N/A	None	Yes
RVCoreP	FPGA	—	5-stage	None	None	∼100 MHz	—	N/A	None	Yes
PULPissimo / RI5CY	ASIC	22–55 nm	4-stage	HW loops	MAC only	100–450 MHz	1–10 mW	∼100k gates	None	Yes
GAP8	ASIC	55 nm	1+8 clust.	HWCE	8-core CNN	∼250 MHz	10–100 mW	∼1M gates	PulpCam	Partial
X-HEEP	FPGA / ASIC	22 nm	2-stage	Hooks	Hooks	10–100 MHz	∼1 mW	Small	None	Yes
Hwacha	ASIC	45 nm	Vector	None	Vector only	∼1 GHz	100–500 mW	Large	None	Yes
OpenSpike	ASIC	130 nm	Multi-cycle	None	SNN	10–50 MHz	1–10 mW	∼50k gates	None	Yes
Eyeriss	ASIC	65 nm	—	None	CNN	100–200 MHz	∼300 mW	12.25 mm^2	None	No
Eyeriss v2	ASIC	28 nm	—	None	Sparse CNN	∼200 MHz	100–300 mW	3–5 mm^2	None	No
TinyVers	ASIC	22 nm	2-stg+NPU	None	Reconf. NPU	100–500 MHz	0.1–1 mW	1–5 mm^2	None	No
Commercial Edge AI	ASIC	40–90 nm	Cortex-M	None	NPU	∼480 MHz	100–300 mW	N/A	None	No
RISC-V DSC Accel.	FPGA	—	RISC-V+CFU	None	Depthwise	∼100 MHz	—	N/A	None	Yes
FPGA Sobel	FPGA	—	Stream	Fixed Sobel	None	∼100 MHz	—	LUT-based	Cam / VGA	Yes

edge vision. However, GAP8 is a closed, complex SoC with an eight-core cluster and proprietary tooling, whereas EVPIX-RV32 is fully open, inspectable, and implementable by students.

Relative to Eyeriss and TinyVers: These dedicated accelerators achieve higher CNN throughput but are either closed (Eyeriss) or lack the classical image-processing pipeline and real-time camera/display integration of EVPIX-RV32. They represent points on the specialization spectrum that EVPIX-RV32 deliberately avoids to preserve educational clarity and architectural flexibility.

5.6 Limitations of the study

Several limitations constrain this work:

1. FPGA timing closure. The failure to meet 100 MHz on the Artix-7 FPGA limits the maximum demonstrable throughput and complicates interfacing with external peripherals expecting faster clock domains. While 70 MHz suffices for the target application, it reveals timing optimization as an area requiring further attention.

2. Resolution constraint. The 128×128 resolution, while pedagogically and architecturally justified, is insufficient for many practical vision tasks (e.g., face recognition, license plate reading, industrial defect inspection). Higher resolutions would require external memory, DMA engines, and significantly more BRAM or SRAM.

3. No cache or virtual memory. The absence of instruction and data caches simplifies the pipeline but limits performance for larger code footprints and data structures. The flat memory map is adequate for bare-metal embedded control but precludes running rich operating systems.

4. Limited TinyML integration. The current finger-counting demonstration relies on an external ESP32-CAM co-processor for inference rather than executing natively on the EVPIX-RV32 CPU or IPU. This validates system-level integration but does not yet demonstrate the full potential of the custom VDOT and RELU instructions.

5. ASIC fabrication gap. While the GDSII layout is DRC-clean and LVS-equivalent, the design has not been fabricated, packaged, or tested in silicon. The open MPW programs (e.g., via Efabless or Google/SkyWater shuttles) provide a path to closing this gap, but it remains future work.

6. Verification depth. The verification strategy relies on simulation testbenches and hardware BIST rather than formal property checking (e.g., SVA assertions, model checking). While sufficient for an educational thesis, industrial-quality cores require formal verification of hazard handling, forwarding correctness, and ISA compliance.

5.7 Broader implications for edge-vision architecture

EVPIX-RV32 contributes empirical evidence to several ongoing debates in computer architecture and embedded systems design:

Open ISA vs. proprietary ISA for edge devices. The work shows that RISC-V's custom opcode space is not merely a theoretical feature but a practical mechanism for integrating domain-specific acceleration. The ability to add grayscale, Sobel, and convolution instructions without breaking RV32I compatibility validates the RISC-V extension philosophy for constrained embedded systems.

Heterogeneity vs. homogeneity. The CPU+IPU architecture validates the heterogeneous approach for edge vision. The scalar core handles control, configuration, and BIST; the IPU handles data-parallel pixel streaming. This separation of concerns reduces energy relative to software-only execution while retaining programmability relative to fixed-function pipelines.

FPGA-to-ASIC portability. The successful translation from Basys-3 FPGA to Sky130 ASIC demonstrates that student-designed RTL can be physically hardened using open tools. This has pedagogical and research implications. Architecture students can now evaluate their designs not only in simulation but also in terms of real silicon area, timing, and power.

Educational impact. By exposing every pipeline stage, control signal, and custom instruction decode path in handwritten SystemVerilog, EVPIX-RV32 is a pedagogical counterpoint to generator-based cores (e.g., Rocket Chip, VexRiscv). Students can trace an instruction from fetch through writeback, observe forwarding and stalls on the VGA display, and modify the IPU kernel coefficients, experiences that opaque generated RTL cannot provide.

5.8 Summary

The analysis confirms that EVPIX-RV32 achieves its primary research objectives within the stated scope and limitations. The FPGA prototype delivers real-time

60 FPS vision processing with visible, switch-controlled modes and 100% BIST verification. The ASIC implementation demonstrates physical realizability with positive timing slack, low milliwatt power, and clean signoff. The custom instruction set and IPU provide measurable acceleration for classical image kernels, while the TinyML mode establishes a path toward data-driven edge perception. Limitations in FPGA timing closure, resolution, and native neural inference identify concrete directions for future refinement. Against the literature, EVPIX-RV32 occupies a unique niche: a fully open, compact, and inspectable edge-vision processor that spans FPGA demonstration and ASIC manufacturability.

Chapter 6

CONCLUSIONS

This work set out to address a gap in the edge-vision and open-source silicon landscape: the lack of a unified, open, and inspectable processor platform that combines RISC-V programmability with hardware-accelerated image processing, real-time sensor integration, TinyML inference, and physical manufacturability. EVPIX-RV32 was built to fill this gap, a custom five-stage pipelined RISC-V RV32I processor with an integrated Image Processing Unit (IPU), custom instruction extensions, and support for lightweight edge-AI workloads. This chapter summarizes the principal achievements, revisits the research objectives, acknowledges the limitations of the current work, and proposes concrete directions for future research.

6.1 Summary of Achievements

The research objectives defined in Section 1.3 were fully addressed through the design, functional verification, FPGA prototyping, and physical implementation of the EVPIX-RV32 processor. The achievements are summarized below against each stated objective.

1. Architecture and RTL design A complete five-stage pipelined RV32I processor core was designed in SystemVerilog, implementing the full base integer instruction set with hazard detection, data forwarding, and branch resolution. The pipeline comprises Instruction Fetch, Decode, Execute, Memory Access, and Writeback stages, separated by pipeline registers with full control-signal propagation. The RTL uses a modular hierarchical style with clear documentation, allowing each stage to be inspected and modified independently. Crucially, custom instruction decode and execution paths for image-processing operations were integrated directly within the EX stage,

enabling the processor to dispatch coprocessor commands without stalling the main pipeline. plain

2. IPU integration A dedicated, streaming Image Processing Unit was designed as a memory-mapped coprocessor controlled through custom R-type instructions encoded within the RISC-V custom-0 opcode space. The IPU autonomously executes seven fundamental frame-level operations on 128×128 pixel buffers stored in dual-port data memory: grayscale conversion, thresholding, maximum pixel detection, Sobel edge detection, 2D convolution with six programmable kernels, max-pooling, and average-pooling. The IPU achieves 1,525 FPS for per-pixel operations and 1,220 FPS for convolution kernels, with an end-to-end processing latency below 1 ms per frame.

3. FPGA system prototyping The complete EVPIX-RV32 system was synthesized and implemented on the Xilinx Artix-7 XC7A35T-1CPG236C FPGA (Digilent Basys-3 board). The prototype includes an OV7670 camera interface via parallel DVP, a VGA controller generating 640×480 at 60 Hz, and a dual-region display showing both source and processed frames. Slide switches enable real-time mode selection among CPU BIST, IPU grayscale, threshold, Sobel, convolution, and TinyML finger-counting. Real-time performance overlays are rendered on the VGA output, and the system maintains a steady 60 FPS end-to-end with zero frame drops.

4. Built-In Self-Test A hardware BIST mode was developed and validated, running a 61-instruction regression suite that exercises all RV32I base instructions and the full set of IPU kernels. The BIST scoreboard compares register and memory values against golden references and renders pass/-fail status directly to the VGA display. Both simulation and hardware testing confirmed 100% pass rates with zero mismatches, providing fully autonomous verification without external debuggers or logic analyzers.

5. TinyML deployment A quantized neural network for finger-counting gesture recognition was integrated and validated, demonstrating real-time inference on live video. The TinyML pipeline classifies hand gestures into six classes (0 to 5 fingers) with 70% accuracy on hardware test sets, confirming that the platform can execute lightweight edge-AI tasks beyond deterministic image filtering.

6. ASIC implementation The processor was synthesized through the open-source OpenROAD/Yosys flow targeting the SkyWater 130-nm CMOS pro-

cess. The implementation produced a DRC-clean, LVS-equivalent GDSII layout containing 42,490 equivalent NAND2 gates. Post-layout signoff confirmed 100 MHz operation with positive timing slack and a total power consumption of 3.24 mW. This result demonstrates that the EVPIX-RV32 architecture is physically realizable and suitable for open multi-project wafer (MPW) tapeout programs.

6.2 Contributions Revisited

The five principal contributions of this thesis, introduced in Chapter 1, are supported by the results and analysis in subsequent chapters:

(a) Custom RISC-V edge-vision ISA extensions. The custom-0 instructions (GRAYSCALE, THRESH, SOBEL, CONV, VDOT, RELU, HACC, OTSU) were specified, implemented in the decode and execute stages, and validated in both simulation and hardware. They reduce software overhead for pixel kernels while preserving RV32I compatibility.

(b) Integrated CPU-IPU architecture. The heterogeneous design coupling a scalar five-stage pipeline with a streaming IPU achieves real-time frame processing with software-visible control. The memory-mapped interface and custom instruction protocol provide a template for future domain-specific accelerator integration.

(c) Self-demonstrating FPGA vision platform. The Basys-3 prototype with OV7670 camera, dual-region VGA, and on-screen performance overlays provides a visual validation environment that makes architecture behavior tangible and measurable.

(d) TinyML on custom open RTL. The finger-counting demonstration proves that lightweight neural inference can run on a student-designed processor with custom datapaths, connecting classical image processing to data-driven edge AI.

(e) Open-source ASIC hardening. The SkyWater 130-nm implementation, with complete synthesis, floorplan, CTS, routing, and signoff reports, contributes a verified physical design to the open-silicon community and establishes a precedent for edge-vision processors in accessible CMOS nodes.

6.3 Limitations

The thesis achieves its stated objectives within the defined scope, but several limitations bound the current work and limit its immediate use:

The FPGA implementation does not meet the nominal 100 MHz timing constraint, requiring operation at approximately 70 MHz. This timing shortfall, while not functionally fatal, indicates that the EX-stage critical path, including custom instruction decode and IPU arbitration, needs further pipelining or retiming for higher-frequency targets.

The 128×128 resolution, chosen to fit Basys-3 BRAM constraints, is insufficient for many commercial vision applications. Higher resolutions would need external memory interfaces, DMA engines, and more sophisticated frame buffering strategies that exceed the current resource budget.

The TinyML finger-counting model achieves only 70% accuracy and currently runs on an external ESP32-CAM co-processor rather than natively on the EVPIX-RV32 CPU or IPU. Native inference would require deeper integration of the VDOT and RELU instructions into a neural network runtime, as well as more extensive model training and quantization.

The ASIC flow produces a verified GDSII layout but stops short of fabrication and silicon testing. While the design is MPW-ready, actual tapeout and characterization remain future work requiring additional funding and scheduling.

Finally, the verification strategy, though comprehensive at the simulation and BIST levels, lacks formal property checking. Formal verification of hazard handling, forwarding correctness, and ISA compliance would strengthen confidence in the design for industrial or safety-critical contexts.

6.4 Future Work

The EVPIX-RV32 architecture and its open-source implementation provide a foundation for several extensions:

Compiler and toolchain support. The custom instructions are currently invoked via assembly macros or inline assembly. Developing GCC or LLVM backend support, including instruction selection patterns, intrinsics, and peephole optimizations, would let high-level language programmers use the IPU without writing assembly. A custom compiler would also facilitate

design-space exploration by automatically mapping image kernels to the optimal mix of custom instructions and software loops.

DMA and memory hierarchy. Integrating a lightweight DMA engine would free the CPU from explicit data movement between camera, frame buffer, and IPU. A DMA could transfer rows or tiles autonomously while the CPU runs control code, improving overall system efficiency. Adding a small instruction cache and data cache would improve performance for larger code footprints and enable higher-level software stacks.

Higher-resolution and external memory. Extending the camera interface and display controller to support QVGA (320×240) or VGA (640×480) resolutions would require external SRAM or SDRAM via the Basys-3 Pmod connectors. This would validate the scalability of the IPU architecture and open the platform to more demanding vision tasks.

Advanced IPU kernels and precision. The IPU currently supports classical image processing. Extensions could include morphological operations (erosion, dilation), histogram equalization, and block-based motion estimation. Reducing the IPU arithmetic precision from 32-bit to 8-bit or 16-bit fixed-point would cut area and power while maintaining adequate quality for many vision tasks.

Native TinyML acceleration. Porting the finger-counting model, or more advanced compact CNNs such as MCUNet variants, to run entirely on the EVPIX-RV32 CPU using the VDOT and RELU instructions would demonstrate true end-to-end neural inference on custom open RTL. This would require a lightweight inference runtime (analogous to TensorFlow Lite Micro or CMSIS-NN) adapted to the RV32I plus custom ISA.

Formal verification. Applying SystemVerilog Assertions (SVA) and formal model checking to the pipeline control logic, particularly the hazard detection unit, forwarding multiplexers, and branch resolution, would provide mathematical guarantees of correctness for all possible instruction sequences. This is especially valuable for the custom instruction paths, which lack the extensive validation history of standard RV32I instructions.

Advanced-node ASIC migration. Migrating the design from SkyWater 130 nm to a 65 nm or 28 nm open or commercial PDK would dramatically reduce area and power while increasing operating frequency. Such a migration would test the portability of the RTL and the robustness of the design methodology across process nodes.

Multi-core and SoC extensions. A dual-core configuration, one core for control and interrupt handling, one for pixel streaming or TinyML inference, would move EVPIX-RV32 toward the heterogeneous multi-core architectures exemplified by GAP8. Adding standard peripherals (UART, SPI, I2C, timer, GPIO) would transform the processor into a complete microcontroller suitable for broader embedded applications.

Bibliography

[1] W. Shi, J. Cao, Q. Zhang, L. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] P. P. Ray, "A survey on edge AI: Opportunities and challenges," *Journal of King Saud University—Computer and Information Sciences*, vol. 34, no. 10, pp. 8769–8789, 2022.

[3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[5] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for risc-v," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug. 2014. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[6] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, 2026, official release version 20260120. [Online]. Available: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html

[7] Digilent Inc., "Basys 3 FPGA development board," Product page, 2024. [Online]. Available: https://digilent.com/shop/basys-3-artix-7-fpga-trainer-board

[8] Xilinx Inc., *7 Series FPGAs Data Sheet: Overview*, 2018. [Online]. Available: https://docs.xilinx.com/v/u/en-US/ds180_7Series_Overview

[9] SkyWater PDK Authors, "Skywater sky130 pdk documentation," Project documentation, 2020. [Online]. Available: https://skywater-pdk.readthedocs.io/en/main/

[10] M. Shalan and T. Edwards, "Building openlane: A 130nm openroad-based tapeout-proven flow," in *2020 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2020, pp. 1–6, formal citation metadata reproduced in the official OpenLane repository README. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenLane

[11] A. M. Turing, “On computable numbers, with an application to the Entscheidungsproblem,” *Proceedings of the London Mathematical Society*, vol. 42, no. 1, pp. 230–265, 1936.

[12] J. von Neumann, “First draft of a report on the EDVAC,” University of Pennsylvania, Tech. Rep., 1945, reprinted in IEEE Annals of the History of Computing, Vol. 15, No. 4, 1993.

[13] D. A. Patterson and J. L. Hennessy, *Computer Architecture: A Quantitative Approach*, 5th ed. Morgan Kaufmann, 2011.

[14] H. H. Aiken, “The Harvard Mark I,” in *Proc. Harvard Symposium on Large-Scale Digital Calculating Machinery*, 1946, pp. 1–8.

[15] P. Lapsley, J. Bier, A. Shoham, and E. A. Lee, *DSP Processor Fundamentals: Architectures and Features*. IEEE Press, 1997.

[16] Intel Corporation, *8086 16-bit HMOS Microprocessor*, 1978, datasheet 205981-003.

[17] D. A. Patterson and D. R. Ditzel, “The case for the reduced instruction set computer,” *ACM SIGARCH Computer Architecture News*, vol. 8, no. 6, pp. 25–33, 1980.

[18] ARM Limited, *ARM Architecture Reference Manual*, 2020, ARM DDI 0406C.

[19] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[20] RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, 2026, official release version 20260120. [Online]. Available: https://docs.riscv.org/reference/isa/priv/priv-index.html

[21] OpenRISC Community, *OpenRISC 1000 Architecture Manual*, 2012. [Online]. Available: https://openrisc.io

[22] I. Kuon and J. Rose, “Measuring the gap between FPGAs and ASICs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 203–215, 2008.

[23] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Pearson, 2015.

[24] Digilent Inc., *Basys 3 Reference Manual*, 2016. [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual

[25] I. Sobel, *An Isotropic* 3×3 *Image Gradient Operator*, 1990, presentation at Stanford Artificial Intelligence Laboratory.

[26] ITU-R, *Recommendation ITU-R BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios*, International Telecommunication Union, 2011.

[27] Video Electronics Standards Association, *VGA Display Standard*, 1987, VESA BIOS Extension.

[28] Digilent Inc., *Pmod Interface Specification*, 2011. [Online]. Available: https://digilent.com/reference/pmod/start

[29] Xilinx Inc., *Vivado Design Suite*, 2023. [Online]. Available: https://www.xilinx.com/products/design-tools/vivado.html

[30] J. Nakamura, *Image Sensors and Signal Processing for Digital Still Cameras*. CRC Press, 2006.

[31] OmniVision Technologies, *OV7670/OV7171 CMOS VGA CameraChip Sensor Datasheet*, 2006, preliminary specification.

[32] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," *arXiv preprint arXiv:2007.10319*, 2020. [Online]. Available: https://arxiv.org/abs/2007.10319

[33] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[34] The OpenROAD Project, "Openroad," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenROAD

[35] google, "skywater-pdk," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/google/skywater-pdk

[36] efabless, "Caravel user project," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/efabless/caravel_user_project

[37] C. Wolf, "Yosys open synthesis suite," GitHub repository, 2023. [Online]. Available: https://github.com/YosysHQ/yosys

[38] Open Circuit Design, "Magic: A VLSI layout system," Open-source project, 2024. [Online]. Available: http://opencircuitdesign.com/magic/

[39] KLayout Project, "Klayout: High performance layout viewer and editor," Open-source project, 2024. [Online]. Available: https://www.klayout.de/

[40] Open Circuit Design, "Netgen: Circuit netlist comparison (LVS) tool," Open-source project, 2024. [Online]. Available: http://opencircuitdesign.com/netgen/

[41] K. Asanović, R. Avižienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, P. Dabbelt, J. Hauser, A. Izraelevitz, S. Karandikar, B. Keller, D. Kim, J. Koenig, Y. Lee, E. Love, M. Maas, A. Magyar, H. Mao, M. Moreto, A. Ou, D. Patterson, B. Richards, C. Schmidt, S. Twigg, H. Vo, and A. Waterman, "The rocket chip generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, Apr. 2016. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.pdf

[42] YosysHQ, "Picorv32 – a size-optimized risc-v cpu," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/YosysHQ/picorv32

[43] SpinalHDL, "Vexriscv: A fpga friendly 32 bit risc-v cpu implementation," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/SpinalHDL/VexRiscv

[44] K. Yoshioka *et al.*, "RVCoreP: A five-stage pipelined RISC-V soft processor optimized for FPGA," in *Proc. International Conference on Field-Programmable Technology (FPT)*, 2020, pp. 1–6, tODO: verify exact author list against original paper.

[45] H. W. Kang and J. W. Choi, "basic_rv32s: An open-source microarchitectural roadmap for risc-v rv32i," *arXiv preprint arXiv:2510.15887*, 2025. [Online]. Available: https://arxiv.org/abs/2510.15887

[46] lowRISC, "Ibex: A small 32 bit risc-v cpu core," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/lowRISC/ibex

[47] pulp-platform, "Pulpino: An open-source microcontroller system based on risc-v," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/pulp-platform/pulpino

[48] ——, "Pulpissimo," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/pulp-platform/pulpissimo

[49] P. D. Schiavone, D. Rossi, A. Pullini, A. Di Mauro, F. Conti, and L. Benini, "Quentin: an ultra-low-power pulpissimo soc in 22nm fdx," in *2018 IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)*, 2018, pp. 1–3. [Online]. Available: https://doi.org/10.1109/S3S.2018.8640145

[50] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "Xpulpnn: Enabling energy efficient and flexible inference of quantized neural network on risc-v based iot end nodes," *arXiv preprint arXiv:2011.14325*, 2020. [Online]. Available: https://arxiv.org/abs/2011.14325

[51] M. Yildirim and O. Ozturk, "Risc-v based tinyml accelerator for depthwise separable convolutions in edge ai," *arXiv preprint arXiv:2511.21232*, 2025. [Online]. Available: https://arxiv.org/abs/2511.21232

[52] G. Rutishauser, J. Mihali, M. Scherer, and L. Benini, "xtern: Energy-efficient ternary neural network inference on risc-v-based edge systems," *arXiv preprint arXiv:2405.19065*, 2024. [Online]. Available: https://arxiv.org/abs/2405.19065

[53] C. Rowen, *Engineering the Complex SOC: Fast, Flexible Design with Configurable Processors*. Prentice Hall, 2004.

[54] ucb-bar, "Hwacha vector-thread co-processor sources," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/ucb-bar/hwacha

[55] S. Machetti, P. D. Schiavone, T. C. Müller, M. Peón-Quirós, and D. Atienza, "X-heep: An open-source, configurable and extendible

risc-v microcontroller for the exploration of ultra-low-power edge accelerators," *arXiv preprint arXiv:2401.05548*, 2024. [Online]. Available: https://arxiv.org/abs/2401.05548

[56] x-heep, "X-heep," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/x-heep/x-heep

[57] E. Flamand, D. Rossi, F. Conti, and L. Benini, "GAP-8: A RISC-V SoC for AI at the edge of the IoT," *arXiv preprint arXiv:1803.06493*, 2018. [Online]. Available: https://arxiv.org/abs/1803.06493

[58] D. P. Fernandez *et al.*, "Hardware Sobel edge detection accelerator on FPGA," in *Proc. IEEE Southern Conference on Programmable Logic (SPL)*, 2013, pp. 1–6, tODO: verify exact author list.

[59] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[60] S. Lopez *et al.*, "High-performance convolution engine for FPGA-based image filtering," in *Proc. IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, 2013, pp. 1–4, tODO: verify exact author list.

[61] N. Neverova, C. Wolf, G. W. Taylor, and F. Nebout, "Multi-scale deep learning for gesture detection and localization," in *Proc. European Conference on Computer Vision Workshops (ECCVW)*, 2015, pp. 211–223.

[62] H. Ren, D. Anicic, and T. Runkler, "Tinyol: Tinyml with online-learning on microcontrollers," *arXiv preprint arXiv:2103.08295*, 2021. [Online]. Available: https://arxiv.org/abs/2103.08295

[63] J. Moosmann, M. Giordano, C. Vogt, and M. Magno, "Tinyissimoyolo: A quantized, low-memory footprint, tinyml object detection network for low power microcontrollers," *arXiv preprint arXiv:2306.00001*, 2023. [Online]. Available: https://arxiv.org/abs/2306.00001

[64] L. Lai, N. Suda, and V. Chandra, "Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus," *arXiv preprint arXiv:1801.06601*, 2018. [Online]. Available: https://arxiv.org/abs/1801.06601

[65] TensorFlow, "Tensorflow lite for microcontrollers," GitHub repository, accessed 2026-06-14. [Online]. Available: https://github.com/tensorflow/tflite-micro

[66] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, "DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," in *Proc. 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 269–284.

[67] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. 43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.

[68] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *arXiv preprint arXiv:1807.07928*, 2018. [Online]. Available: https://arxiv.org/abs/1807.07928

[69] V. Jain, S. Giraldo, J. De Roose, L. Mei, B. Boons, and M. Verhelst, "Tinyvers: A tiny versatile system-on-chip with state-retentive emram for ml inference at the extreme edge," *arXiv preprint arXiv:2301.03537*, 2023. [Online]. Available: https://arxiv.org/abs/2301.03537

[70] ARM Limited, *Ethos-U55: Micro Neural Processing Unit (MicroNPU)*, 2020. [Online]. Available: https://developer.arm.com/ip-products/processors/ethos/ethos-u55

[71] STMicroelectronics, *STM32H7 Reference Manual*, 2021.

[72] F. Modaresi, M. Guthaus, and J. K. Eshraghian, "Openspike: An openram snn accelerator," *arXiv preprint arXiv:2302.01015*, 2023. [Online]. Available: https://arxiv.org/abs/2302.01015

[73] P. Sauter, T. Benz, P. Scheffler, Z. Jiang, B. Muheim, F. K. Gürkaynak, and L. Benini, "Basilisk: Achieving competitive performance with open eda tools on an open-source linux-capable risc-v soc," *arXiv preprint arXiv:2405.03523*, 2024. [Online]. Available: https://arxiv.org/abs/2405.03523

[74] FOSSi Foundation, "Fossi foundation," Official website, accessed 2026-06-14. [Online]. Available: https://fossi-foundation.org/

[75] LibreLane Authors, "Librelane documentation," Project documentation, 2026. [Online]. Available: https://librelane.readthedocs.io/en/latest/

Appendices

Appendix A: GitHub Repository

The complete source code for this project is available at:

`https://github.com/aukhalid/evpix_rv32`

The repository includes full codebase of the the SystemVerilog RTL, testbenches, Vivado project files, Basys-3 constraints, and the OpenROAD flow configuration used for the SkyWater 130 nm ASIC implementation.